ENERGY-EFFICIENT TIME SERIES CLASSIFICATION ON IOT DEVICES WITH SENSOR-AWARE EARLY-EXIT MACHINE LEARNING

By

LUBAH NELSON

A thesis submitted in partial fulfillment of
the requirements for the degree of

MASTER OF SCIENCE

WASHINGTON STATE UNIVERSITY
School of Electrical Engineering and Computer Science

MAY 2025

© Copyright by LUBAH NELSON, 2025
All Rights Reserved

© Copyright by LUBAH NELSON, 2025
All Rights Reserved

To the Faculty of Washington State University:

The members of the Committee appointed to examine the thesis of LUBAH NELSON find it satisfactory and recommend that it be accepted

Ganapati Bhat, Ph.D., Chair

Assefaw Gebremedhin, Ph.D.

Nghia Hoang, Ph.D.

ACKNOWLEDGMENT

I acknowledge my committee members, Dr. Ganapati Bhat, Dr. Assefaw Gebremedhin, and Dr. Nghia Hoang, for serving on my thesis committee.

ENERGY-EFFICIENT TIME SERIES CLASSIFICATION ON IOT DEVICES WITH SENSOR-AWARE EARLY-EXIT MACHINE LEARNING

Abstract

by Lubah Nelson, M.S.
Washington State University
May 2025

Chair: Ganapati Bhat

Time-series data processing is critical in several high-impact applications including mobile health, environmental monitoring, and digital agriculture. Energy efficiency represents a fundamental bottleneck for continuous time-series classification on edge IoT devices, particularly in wearable health monitoring where battery constraints severely limit deployment. Traditional machine learning models for time-series processing wait for complete sensor data windows before classification, resulting in excessive energy consumption regardless of task difficulty.

This thesis introduces Sensor-Aware Early-Exit (SEE) classifiers that jointly optimize sensing and computational energy through progressive inference with partial data windows. The proposed framework includes multiple architectural innovations: (1) SEE-CNN, a convolutional neural network with late input blocks that preserve temporal information while enabling early exits; (2) SEE-vRF, a vertically-split random forest architecture; and (3) SEE-hRF, a sequential random forest with cascaded sub-forests. These architectures dynamically terminate inference and transition sensors to low-power mode once prediction confidence exceeds learned thresholds.

A comprehensive evaluation methodology was developed encompassing six diverse health and activity recognition datasets (PAMAP2, WESAD, UCI-HAR, OPPORTUNITY, EMG, and Epilepsy), systematic ablation studies, and comparative analysis against state-of-the-art baselines including NMEC and computational-only early-exit methods. Threshold optimization was performed through Design Space Exploration combined with Bayesian Optimization, balancing accuracy-energy trade-offs across the solution space.

Experimental results demonstrate that SEE classifiers achieve 50-70% sensing energy savings while maintaining accuracy within 2% of standard models that use complete data windows. The sequential RF architecture (SEE-hRF) achieves up to 70% energy reduction on health monitoring datasets. End-to-end validation on Raspberry Pi hardware with real sensors confirms 29-42% system-level energy savings, validating the practical feasibility of the approach. Calibration analysis demonstrates that entropy-based confidence metrics provide reliable exit decisions with Expected Calibration Error below 0.05.

This work establishes that joint sensor-compute optimization through early-exit architectures fundamentally outperforms approaches targeting only computational or only sensing efficiency. The proposed methods enable longer battery lifetime for wearable devices, potentially improving user compliance and expanding the viability of continuous health monitoring applications.

TABLE OF CONTENTS

Page

ACKNOWLEDGMENT iii

ABSTRACT iv

LIST OF TABLES xiv

LIST OF FIGURES xv

1 INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Statement 2

1.3 Research Contributions 4

1.4 Research Questions 6

1.5 Thesis Organization 8

2 BACKGROUND AND RELATED WORK 9

2.1 Time-Series Classification on Edge Devices 9

2.1.1 Fundamentals of Time-Series Classification 9

2.1.2 Edge Deployment Constraints and Trade-offs 10

2.1.3 Existing Edge Optimization Strategies 11

2.1.4 Adaptive Sensing Approaches 13

2.2 Early-Exit Neural Networks 15

2.2.1 BranchyNet and MSDNet 15

2.2.2 HarvNet for Energy Harvesting 16

2.2.3 Limitations: No Sensing Energy Optimization 18

2.3 Early Time-Series Classification Methods 19

Page

2.3.1 TEASER: Two-Tier Early Classification 19

2.3.2 NMEC: Non-Myopic Early Classification 21

2.3.3 Broader ECTS Literature and Evaluation Metrics 23

2.3.4 Limitations: No Real-Time Sensor Control or Energy Focus 24

2.4 Random Forests for Edge Deployment 26

2.4.1 Advantages for Resource-Constrained Devices 27

2.4.2 Prior Random Forest Optimization Work 29

2.4.3 Daghero RF: Computational Early Stopping Only 31

2.5 Design Space Exploration and Bayesian Optimization 33

2.5.1 Design Space Exploration for Threshold Selection 33

2.5.2 Bayesian Optimization for Efficient Search 35

2.5.3 Applications in Machine Learning 37

2.6 Summary and Gap Analysis 39

2.6.1 Summary of Related Work 39

2.6.2 The Critical Gap 40

2.6.3 Gap Analysis Table 41

2.6.4 Positioning the Contribution 43

3 PROPOSED METHODOLOGY 46

3.1 System Overview and Problem Formulation 47

3.1.1 Training and Deployment Pipeline 47

3.1.2 Energy Model 47

3.1.3 Problem Formulation 48

3.2 Sensor-Aware Early-Exit Framework 49

3.2.1 Core Concept: Progressive Inference 49
3.2.2 Confidence-Based Exit Decisions 50
3.2.3 Entropy as Uncertainty Metric 51
3.3 SEE-CNN Architecture 52
3.3.1 Network Architecture Design 52
3.3.2 Late Input Blocks: The Key Architectural Innovation 54
3.3.3 Training Procedure 55
3.4 SEE-vRF: Vertical Random Forest Architecture 56
3.4.1 Tree Splitting Strategy 57
3.4.2 Progressive Inference with Partial Trees 57
3.4.3 Aggregation and Confidence Estimation 58
3.5 SEE-hRF: Sequential Random Forest Architecture 59
3.5.1 Cascaded Sub-Forest Design 59
3.5.2 Prediction Refinement Mechanism 60
3.5.3 Sequential Decision Process 61
3.6 Threshold Optimization Methodology 61
3.6.1 Design Space Exploration with Coarse-to-Fine Refinement 61
3.6.2 Bayesian Optimization for Efficient Search 62
3.6.3 Multi-Objective Optimization 63
4 EXPERIMENTAL DESIGN AND EVALUATION FRAMEWORK 65
4.1 Evaluation Framework Design 65
4.1.1 Cross-Domain Evaluation Strategy 65
4.1.2 Multi-Dimensional Performance Assessment 66
4.2 Datasets 67

4.2.1 Health Monitoring Applications 67
4.2.2 Activity Recognition Applications 69
4.2.3 Preprocessing Protocol 70
4.3 Evaluation Metrics 71
4.3.1 Classification Accuracy and Error Analysis 71
4.3.2 Energy Consumption Modeling 72
4.3.3 Confidence Calibration Assessment 73
4.3.4 Computational Efficiency Characterization 74
4.4 Baseline Models and Comparison Framework 75
4.4.1 Standard Full-Window Baselines 75
4.4.2 BranchyNet: Early-Exit Neural Networks 75
4.4.3 Early Time-Series Classification Methods 77
4.4.4 Fair Comparison Protocol 78
4.5 Hardware Platforms and Measurement Protocol 79
4.5.1 Development Platform: Nvidia AGX Xavier 79
4.5.2 Deployment Platform: Raspberry Pi 4 80
5 Results and Analysis 83
5.1 Overall Performance: Accuracy-Energy Trade-offs 83
5.1.1 Statistical Validation 85
5.2 Architecture Comparison: CNN vs. vRF vs. hRF 86
5.2.1 Computational Cost Analysis 87
5.3 Ablation Studies 88
5.3.1 Impact of Exit Point Configuration 88
5.3.2 Optimization Method Comparison 89

5.3.3 Confidence Measure Selection 90
5.4 Comparative Analysis with Baselines 91
5.4.1 Comparison with Early Time-Series Classification 91
5.4.2 Comparison with Compute-Only Optimization 92
5.5 Calibration and Confidence Reliability 93
5.5.1 Calibration Quality Assessment 93
5.5.2 Threshold Sensitivity Analysis 94
5.6 Cross-Domain Performance Analysis 95
5.6.1 Performance by Application Domain 95
5.6.2 Factors Predicting Energy Savings Potential 96
5.7 Real-World Validation: Raspberry Pi Measurements 97
5.7.1 Hardware Testbed Configuration 97
5.7.2 Measured Energy Savings 98
5.7.3 Component-Level Energy Profiling 99
5.8 Summary 99
6 Conclusion 101
6.1 Summary of Contributions 101
6.2 Key Findings and Insights 104
6.3 Limitations and Future Research Directions 107
6.4 Broader Impact and Future Vision 110
6.5 Closing Remarks 113
APPENDIX
A Hyperparameter Configurations 118
A.1 CNN Architecture Hyperparameters 118

A.2 Random Forest Hyperparameters 118
A.3 Bayesian Optimization Configuration 118
B Detailed Dataset Statistics 122
B.1 Dataset Characteristics Summary 122
B.2 Class Distribution Details 123
C Energy Model Derivations 125
C.1 Sensing Energy Model 125
C.2 Early-Exit Sensing Energy 125
C.3 Computational Energy Model 126
C.3.1 CNN Inference Energy 126
C.3.2 Random Forest Inference Energy 127
C.4 Early-Exit Overhead 127
C.5 Total System Energy 128
C.6 Energy Savings Calculation 128
D Author Contribution Statements 129
D.1 ESWEEK 2025 Paper Contributions 129
D.1.1 Individual Contributions 129
D.2 MLSys 2026 Paper Contributions 130
D.2.1 Individual Contributions 130
D.2.2 Clarification on Thesis Scope 132
E Additional Experimental Results 133
E.1 Per-Dataset Ablation Studies 133
E.2 Confidence Metric Detailed Comparison 134
E.3 Exit Point Distribution Histograms 134

F Implementation Details 135
F.1 Software Dependencies 135
F.2 Pseudocode for Key Algorithms 135
F.2.1 Early-Exit Inference Algorithm 135
F.2.2 Bayesian Optimization for Threshold Selection 135
F.3 Hardware Setup Details 135
F.3.1 Raspberry Pi Configuration 135
F.3.2 Power Measurement Setup 137
REFERENCES 117

LIST OF TABLES

TABLE Page

2.1 Gap Analysis: Existing Approaches vs. Required Capabilities for Joint Sensing-Computation Optimization 42

4.1 Dataset characteristics spanning health monitoring and activity recognition domains 70

5.1 Performance across datasets and architectures. Energy savings show reduction in sensing energy relative to full-window baselines. Average exit indicates mean window fraction at classification. 84

5.2 Architecture selection guide by dataset. Bold values indicate optimal accuracy-energy balance for each application. 86

5.3 Computational overhead on Nvidia AGX Xavier platform. Exit logic adds minimal burden to base model inference. 87

5.4 Ablation study on number of exits, averaged across all datasets and architectures. Diminishing returns appear beyond three exits. 88

5.5 Threshold optimization strategy comparison. Bayesian optimization provides near-optimal performance with order-of-magnitude speedup. 89

5.6 Expected Calibration Error for different confidence metrics. Lower values indicate better alignment between predicted confidence and observed accuracy. 90

5.7 Comparison with early classification baselines. SEE achieves 2–3× greater savings through real-time sensor control versus retrospective analysis. 91

5.8 System energy breakdown showing sensing dominance. Compute-only optimization (Daghero) achieves limited savings compared to sensor control (SEE-RF). 92

5.9 Expected Calibration Error across architectures and datasets. Lower values indicate better alignment between predicted confidence and observed accuracy. 94
5.10 False positive rates at different confidence thresholds. Higher thresholds reduce errors at the cost of later exits and decreased energy savings. 95
5.11 Cross-domain performance analysis. Energy savings vary by domain characteristics, with activity recognition showing highest potential. 96
5.12 End-to-end energy measurements on Raspberry Pi 4. Measured savings closely track simulated estimates, validating the energy model. 98

A.1 Hyperparameters for SEE-CNN architecture (ESWEEK 2025) 119
A.2 Hyperparameters for SEE-RF, SEE-vRF, and SEE-hRF architectures 120
A.3 Bayesian optimization hyperparameters for threshold selection (MLSys 2026) 121

B.1 Complete dataset characteristics 122
B.2 Per-class sample counts and balance ratios 124

E.1 Complete ablation study results: number of exits per dataset 133
E.2 Detailed calibration metrics for different confidence measures 134

F.1 Software environment and dependencies 136

LIST OF FIGURES

FIGURE Page

Dedication

CHAPTER ONE

INTRODUCTION

1.1 Motivation

The proliferation of Internet of Things (IoT) and wearable devices has transformed continuous monitoring across healthcare, environmental sensing, and industrial applications. Billions of sensors now stream physiological and contextual data that enable applications ranging from glucose monitoring and seizure detection to fall detection and activity recognition (**lara2012survey**; **vasisht2017farmbeats**; **hughes2023wearable**). Clinically, remote monitoring reduces hospital readmissions by 30–50% for chronic conditions and enables earlier intervention in disease progression (**maetzler2016clinical**; **hughes2023wearable**). Yet one constraint systematically limits deployment at scale: energy efficiency.

In real systems, *sensing* rather than computation dominates the energy budget. Sensor energy consumption accounts for 60–80% of total device energy in typical IoT time-series applications (**bhat2019energy**; **nakhkash2019energy**; **martinez2015power**). Pipelines trained on fixed windows of length T acquire all T timesteps for every instance, regardless of difficulty. A static sitting posture becomes separable within approximately 1 second of accelerometer data—yet the system continues collecting 9 additional seconds, wasting 90% of the sensing energy. Walking exhibits clear periodic gait patterns within 2–3 seconds, again rendering 70% of data collection unnecessary. Only complex transitional activities genuinely require the full window duration for confident classification. This one-size-fits-all approach creates massive preventable energy waste. When devices require daily charging, longitudinal studies report patient compliance falling from approximately 80% in week one to below 20% by week four (**ozanne2018wearables**; **daneault2018measurement**; **espay2016technology**). Monitoring gaps translate directly into missed clinical events—cardiac episodes, falls, and seizure patterns go undetected because users abandon energy-

hungry devices.

Two observations motivate this work. First, time-series classification tasks exhibit pronounced heterogeneity in temporal information content. Some instances are confidently classifiable after observing only early timesteps, while others genuinely require the full window. Second, existing efficiency methods optimize only one dimension of the system. Compute-centric early-exit neural networks reduce computational costs by 20–40% but require complete sensor data before inference begins, leaving the dominant sensing energy (60–80% of total) unaddressed and achieving only 4–16% total system energy savings. Sensing-centric techniques such as duty cycling reduce acquisition energy by 30–40% but operate independently of machine learning models, risking the loss of discriminative temporal features. Early time-series classification methods demonstrate that partial data often suffices for accurate predictions but operate on pre-collected datasets without real-time sensor control, focusing on latency reduction rather than energy optimization. What is missing is a *joint* approach that uses prediction confidence from progressively arriving data to decide both when to stop acquiring and when to stop computing, achieving multiplicative rather than additive energy savings.

1.2 Problem Statement

Traditional machine learning models for time-series classification operate under a fundamental architectural constraint: they require complete data windows before inference can begin. This limitation stems from architectural design. A convolutional neural network trained on windows of length T timesteps has its first convolutional layer designed to accept input tensors of fixed dimension $X \in \mathbb{R}^{T \times C}$, where C represents the number of channels. Similarly, a random forest classifier expects feature vectors of predetermined dimensionality computed over complete T-length windows. These models cannot simply skip missing timesteps—the architecture provides no mechanism to process partial data segments. Dur-

ing training, models learn feature representations and decision boundaries under the explicit assumption that all T timesteps will be available at inference time. This design creates a rigid coupling between data acquisition and inference: sensors must collect the entire window $X = \{x_0, x_1, \ldots, x_{T-1}\}$ before the model $f : X \to y$ can produce a prediction, even when early timesteps contain sufficient information for accurate classification.

Consider a wearable activity recognition system deployed for continuous monitoring of an elderly patient at risk for falls. The system uses a standard 10-second sliding window for classification, with a tri-axial accelerometer sampling at 50 Hz, yielding 500 data points per window. Across three representative instances, the missed opportunity for energy savings becomes concrete. For a sitting posture, the accelerometer readings stabilize to near-zero movement within the first second—50 samples reveal a consistent absence of motion that enables 99% confident classification as "sitting." Yet the system continues collecting data for the remaining 9 seconds, consuming 90% of the sensing energy unnecessarily. For walking, clear periodic gait patterns emerge by 3 seconds into the window—the characteristic frequency and amplitude of normal ambulation become unambiguous in 150 samples, enabling 95% confident classification. Again, the system unnecessarily collects 7 additional seconds of redundant data, wasting 70% of the sensing energy. Only for complex transitional activities such as sit-to-stand transfers does the full 10-second window prove necessary: the pattern remains ambiguous at 5 seconds, requiring 8–9 seconds of observation before sufficient discriminative features accumulate for 92% confident classification. Current systems treat all three scenarios identically, collecting complete 10-second windows regardless of instance-specific information content. Averaged across typical activity distributions, this represents a potential 60–80% energy savings opportunity that remains entirely unexploited.

The objective of this work is to reduce sensing energy and inference latency while maintaining accuracy within a small bound Δ (typically $\leq 2\%$) of full-window baselines. The core challenge is to perform *progressive* inference on partial windows and to trigger reliable early exits whose confidence can safely drive sensor power transitions in real time. This

requires innovations across multiple dimensions: architectural designs that accommodate variable-length input windows, training methodologies that prepare models for progressive inference, confidence quantification approaches that provide calibrated early predictions, and system-level integration that couples inference confidence to sensor power state transitions. This thesis addresses this gap by investigating the following central question: *How can we design machine learning architectures that progressively process partial sensor data, make confidence-calibrated early exit decisions, and dynamically transition sensors to low-power states—thereby achieving substantial sensing-energy savings while maintaining classification accuracy?*

1.3 Research Contributions

This thesis contributes a joint sensing–computation optimization framework for energy-efficient time-series classification on resource-constrained edge devices. The work is based on two publications: (1) "Energy-Efficient Time Series Applications on IoT Devices with Sensor-Aware Early-Exit Classifiers" (accepted to ESWEEK 2025), and (2) "Sensor-Aware Random Forests With Early Exits For Energy-Efficient Time Series Applications on IoT Devices" (submitted to MLSys 2026). These publications represent different stages of research maturity: the ESWEEK paper establishes foundational concepts through individual work, while the MLSys paper presents sophisticated architectural variants and optimization methodologies developed collaboratively with labmates Sirine Belhaj and Negar Haghpanah. The contributions are organized to clearly distinguish individual versus collaborative work.

The primary contributions from the ESWEEK 2025 paper represent individual work. The Sensor-Aware Early Exit (SEE) framework introduces a unified approach for joint optimization of sensing and computational energy through progressive inference with partial data windows. The framework introduces early exit points corresponding to increasing data availability, couples exit decisions to sensor power control through hardware interfaces, and

uses entropy-based confidence thresholds to determine when to stop data acquisition. This framework addresses a gap in prior work: computational early-exit methods optimize only inference energy, while sensing methods operate independently of model predictions. SEE achieves multiplicative savings by jointly controlling both dimensions based on progressive inference confidence. The SEE-CNN architecture implements this framework using late input blocks that merge newly arriving sensor data with previously extracted features (detailed in Section 3.3). Experimental results across diverse application domains demonstrate 50–60% average sensing energy savings while maintaining accuracy within 2% of full-window baselines. The initial SEE-RF design presents the first random forest architecture incorporating early exits for time-series classification, proving that progressive inference generalizes beyond neural networks to decision tree ensembles. Random forests offer distinct advantages for ultra-resource-constrained deployments: 10–100× lower memory footprint compared to CNNs, faster inference through tree traversal, and inherent interpretability. The preliminary RF design achieves energy savings comparable to CNN variants while enabling deployment on microcontroller-class hardware. Cross-domain empirical validation provides systematic evaluation across six diverse datasets spanning health monitoring, activity recognition, and brain-computer interfaces, demonstrating consistent 50–60% average sensing energy savings with accuracy maintained within 2% of standard models and latency reduced by 40–50%.

Following the successful demonstration of SEE-CNN, observation revealed that random forests offer several advantages for resource-constrained deployments—particularly memory efficiency and fast inference—motivating exploration of RF-specific early-exit architectures. In collaboration with Sirine Belhaj and Negar Haghpanah, the MLSys 2026 work developed two complementary architectural approaches exploiting different structural properties of random forests. Contributions to this collaborative work included the conceptual motivation, evaluation methodology design, and comprehensive experimental analysis. The architectural designs were led by collaborators. The SEE-vRF architecture employs vertical tree splitting with depth-based progressive traversal (Section 3.4), excelling on datasets

with early discriminative features and achieving up to 70% sensing energy savings on health monitoring applications. The SEE-hRF architecture uses horizontal sub-forest cascades with progressive refinement (Section 3.5), proving particularly effective for datasets with temporal pattern evolution. Systematic threshold optimization introduces a two-stage approach combining coarse-grained Design Space Exploration with fine-grained Bayesian Optimization using Gaussian Process surrogates (Section 3.6). This methodology achieves 90% reduction in optimization time compared to exhaustive search while producing solutions within 2% of global optima, making threshold tuning tractable for practical deployments. A comprehensive evaluation framework provides reproducible methodology spanning accuracy–energy–latency trade-offs with careful baseline selection and Expected Calibration Error analysis ensuring confidence estimates are reliable for sensor control decisions (Chapter 4). Real-world hardware validation implements end-to-end systems on Raspberry Pi 4 with physical accelerometer sensors, actual sensor power state control through GPIO interfaces, and direct energy measurements using external power meters. System-level measurements demonstrate 29–42% energy savings across different activity scenarios, validating that simulation-based estimates are accurate within 5% and computational overhead remains below 3%, proving practical feasibility beyond theoretical simulation results.

1.4 Research Questions

This thesis investigates four primary research questions that address fundamental challenges in designing energy-efficient time-series classifiers for resource-constrained edge devices.

RQ1: Can partial sensor windows maintain classification accuracy in time-series tasks? Traditional machine learning models assume all temporal information is necessary for accurate classification, but it is not immediately obvious that early timesteps contain sufficient discriminative information to enable confident predictions. This question validates the core premise of progressive inference by analyzing the temporal distribution

of class-discriminative features, measuring information gain as a function of time within windows, and quantifying accuracy degradation curves as data availability increases from partial to complete windows.

RQ2: How should early-exit architectures differ between convolutional neural networks and random forests? CNNs perform hierarchical feature extraction through successive convolutional layers, while random forests employ ensemble voting across independently trained decision trees. For CNNs, the challenge lies in introducing partial data at intermediate layers through late input blocks that merge new timesteps with previously extracted features. For random forests, the question becomes whether to split trees vertically (by depth) or horizontally (by trees, creating sub-forest cascades). This question explores design principles specific to each model family and characterizes which architectures suit which deployment scenarios based on memory constraints, computational budgets, and dataset properties.

RQ3: How can we efficiently search the threshold design space for early-exit decisions? Each early exit requires a confidence threshold that determines when to stop inference, creating a K-dimensional continuous optimization space for K exits that grows exponentially and exhibits non-linear, non-convex accuracy-energy trade-offs. This question investigates whether structured search strategies (coarse-grained Design Space Exploration, Bayesian Optimization with Gaussian Process surrogates) can achieve near-optimal solutions with drastically reduced computational budgets compared to exhaustive grid search, making threshold optimization tractable for practical deployments.

RQ4: What are the real-world energy savings on embedded hardware platforms? Simulation-based energy estimation makes simplifying assumptions about sensor wake-up and sleep transition latencies, computational overhead, memory access patterns, and operating system scheduling that may not hold in practice. This question requires building end-to-end systems on actual embedded platforms with physical sensors, implementing complete sensor control through hardware interfaces, measuring actual power draw

using external power meters, and validating that published savings claims are grounded in actual hardware measurements rather than optimistic simulation assumptions.

1.5 Thesis Organization

The remainder of this thesis is organized as follows. Chapter 2 reviews compute-only early exits, sensing-only strategies, and offline early-classification methods, and formalizes the joint-optimization gap that motivates this work. Chapter 3 presents the SEE framework and progressive architectures for CNNs and random forests, together with the two-stage threshold optimization procedure combining Design Space Exploration and Bayesian Optimization. Chapter 4 details datasets, preprocessing, evaluation metrics, baseline configurations, and embedded hardware setup, emphasizing calibration analysis and reproducibility protocols. Chapter 5 reports experimental results organized by research question, including overall performance comparisons, architecture-specific analysis, systematic ablation studies, comparative evaluation against compute-only and sensing-only baselines demonstrating synergistic benefits of joint optimization, and hardware validation on Raspberry Pi with measured energy savings. Chapter 6 concludes with a summary of key contributions, discussion of fundamental insights about joint sensing-computation optimization, practical implications for practitioners deploying early-exit classifiers, acknowledgment of limitations including assumptions about sensor power gating and static threshold policies, and directions for future research such as online threshold adaptation and extensions to other model families.

CHAPTER TWO

BACKGROUND AND RELATED WORK

This chapter surveys related work across five key areas. Section 2.1 examines time-series classification on edge devices and existing optimization strategies. Sections 2.2 and 2.3 analyze computational and temporal optimization approaches respectively, revealing that existing methods optimize only one dimension of the system. Section 2.4 reviews random forest architectures for resource-constrained devices. Section 2.5 covers optimization methodologies for threshold selection. Finally, Section 2.6 synthesizes these areas to identify the critical gap that motivates this thesis.

2.1 Time-Series Classification on Edge Devices

Time-series classification involves assigning discrete labels to temporal sequences of sensor data. The standard approach processes fixed-length windows through feature extraction and classification stages. While effective in unconstrained environments, this pipeline faces fundamental challenges when deployed on battery-powered edge devices with limited compute, memory, and energy budgets.

2.1.1 Fundamentals of Time-Series Classification

Time-series classification assigns labels to sequences of temporally ordered observations. Formally, given an input sequence $X = \{x_1, x_2, \ldots, x_T\}$ where each $x_t \in \mathbb{R}^C$ represents a C-channel sensor measurement at timestep t, the goal is to learn a mapping $f : X \to y$ where $y \in \{1, 2, \ldots, K\}$ denotes one of K discrete classes. Common applications include human activity recognition from accelerometer and gyroscope data, health monitoring through electrocardiogram (ECG), electroencephalogram (EEG), and electromyography (EMG) signals, and environmental event detection from distributed sensor networks (**lara2012survey**; **bagnall2017great**; **fawaz2019deep**).

The standard processing pipeline operates in five stages. First, data acquisition collects sensor readings at a fixed sampling rate, typically ranging from 50 Hz for activity recognition to 700 Hz for fine-grained physiological monitoring. Second, windowing segments the continuous stream into fixed-length windows, often using sliding windows with 50% overlap to ensure no events are missed at window boundaries. Third, preprocessing applies normalization to handle sensor calibration differences and filtering to remove high-frequency noise. Fourth, feature extraction computes statistical descriptors (mean, variance, energy), frequency-domain representations (Fourier coefficients, spectral entropy), or learned features through neural networks. Finally, classification assigns labels using machine learning models including convolutional neural networks for hierarchical feature learning, recurrent neural networks for sequential dependencies, random forests for ensemble predictions, or support vector machines for margin-based separation (**lara2012survey**; **fawaz2019deep**).

This architecture proves effective when computational resources are abundant, but deployment on resource-constrained edge devices introduces severe limitations that challenge every stage of the pipeline.

2.1.2 Edge Deployment Constraints and Trade-offs

Deploying time-series classification on edge devices requires navigating fundamental trade-offs between cloud and edge processing. Cloud-based systems offer unlimited computational resources, supporting arbitrarily complex models without memory or latency constraints. However, streaming raw sensor data to the cloud incurs substantial costs: privacy concerns arise from transmitting sensitive physiological or behavioral data, communication latency typically ranges from 50–200 milliseconds for cellular networks, connectivity requirements preclude operation in areas without network coverage, and wireless transmission consumes 10–100× more energy than local computation (**lane2016deepx**; **cai2019once**). Edge processing addresses these limitations by keeping data local, ensuring privacy preservation, achieving sub-millisecond inference latency, operating offline without connectivity, and elim-

inating communication energy costs. Yet edge deployment imposes stringent resource constraints: typical devices provide 512 MB to 4 GB of RAM compared to 40+ GB on cloud GPUs, feature low-power CPUs with limited or absent GPU acceleration, require battery operation lasting days to weeks between charges, and offer constrained storage for model parameters (**cai2019once**; **ren2020joint**).

Time-series classification proves particularly demanding for edge deployment due to high-frequency data acquisition, real-time processing requirements, and continuous operation. Accelerometer-based activity recognition samples at 100–200 Hz, generating 20,000–40,000 readings per second for a 10-second window with three axes. Electrocardiogram monitoring operates at 250–500 Hz to capture cardiac electrical activity with sufficient temporal resolution. These high sampling rates create substantial data volumes that must be processed within tight latency budgets—activity recognition systems typically target sub-second response times for safety-critical fall detection, while seizure prediction applications require processing within 100–200 milliseconds to enable timely intervention. Continuous 24/7 monitoring compounds these challenges, as energy budgets must support weeks of operation between battery charges rather than hours of sporadic usage.

As established in Chapter 1, sensing energy dominates time-series applications (60–80% of total budget), with a typical 10-second window consuming 5 mJ for data acquisition versus 1.5–3 mJ for computation. This fundamental energy partition means that optimizations targeting only computational efficiency can achieve at most 20–40% total system energy savings, leaving the dominant sensing component entirely unaddressed.

2.1.3 Existing Edge Optimization Strategies

Prior research has explored multiple strategies to enable efficient edge deployment, each targeting different system components. Model compression techniques reduce neural network size and complexity through several mechanisms. Pruning removes weights that contribute minimally to model accuracy, typically eliminating 50–90% of connections with negligible

performance degradation (**han2015deep**). Quantization reduces numerical precision from 32-bit floating-point to 8-bit integers or even binary representations, achieving 4–8× memory reduction and 2–5× inference speedup while maintaining accuracy within 1–2% of full-precision baselines. Knowledge distillation trains compact student networks to mimic the behavior of larger teacher models, enabling 5–10× parameter reduction while retaining 90–95% of teacher accuracy (**hinton2015distilling**). These compression techniques successfully reduce model footprint from hundreds of megabytes to tens of megabytes, enabling deployment on memory-constrained microcontrollers. However, they address only the computational dimension of energy consumption, leaving sensing energy unmodified.

Neural Architecture Search (NAS) methods automatically discover efficient network architectures optimized for both accuracy and computational efficiency. MnasNet uses reinforcement learning to explore architecture spaces, discovering mobile-optimized networks with 2× faster inference than manually designed MobileNets at equivalent accuracy (**tan2019mnasnet**). FBNet employs differentiable architecture search with latency-aware objectives, producing networks with 20–30% lower latency than ResNet baselines on mobile devices. Proxyless-NAS searches directly on target hardware, accounting for device-specific characteristics such as memory bandwidth and cache hierarchy, achieving hardware-tailored architectures with 40–50% latency reduction (**cai2019proxylessnas**). While NAS produces better accuracy-efficiency trade-offs than manual architecture design, these approaches optimize computational graphs under the assumption that complete input windows are available, providing no mechanism to reduce sensing energy or process partial data incrementally.

Hardware acceleration employs specialized processors to execute neural network operations with higher energy efficiency. Tensor Processing Units (TPUs) provide optimized matrix multiplication units achieving 10–50× better operations-per-watt than general-purpose CPUs. Neural Processing Units (NPUs) in mobile devices such as Qualcomm's Hexagon DSP deliver 100–200× better energy efficiency for quantized neural networks compared to ARM Cortex-A cores. Application-specific integrated circuits (ASICs) custom-designed for specific

models can achieve 1000× efficiency gains through architectural specialization, eliminating unnecessary flexibility of programmable processors (**chen2016eyeriss**). These hardware approaches dramatically improve computational energy efficiency, but sensing energy remains unchanged regardless of computational acceleration—a 100× faster inference engine still requires collecting complete sensor windows before processing can begin.

What unifies these optimization strategies is their exclusive focus on computational efficiency. Each achieves substantial improvements in inference latency, memory footprint, or computational energy consumption. Yet none addresses the dominant energy cost in time-series systems: continuous sensor operation. A model compressed 10× still requires acquiring complete windows. An architecture optimized for mobile inference still processes all T timesteps. A hardware accelerator executing inference 100× faster still waits for sensors to collect all data before processing begins. This computational focus creates a fundamental ceiling on achievable energy savings at 20–40% of total system energy. Breaking through this ceiling requires approaches that jointly optimize sensing and computation.

2.1.4 Adaptive Sensing Approaches

Beyond computational optimization, a separate line of work addresses sensing energy through adaptive data acquisition strategies that operate independently of machine learning inference. These approaches aim to reduce sensor operation time or sampling rates based on contextual conditions, achieving sensing energy savings without modifying the ML model or inference process.

Duty cycling periodically transitions sensors between active and sleep states according to fixed schedules. A typical duty cycle might operate sensors for 2 seconds every 10 seconds, achieving 80% sensing energy reduction during sleep periods. Adaptive duty cycling adjusts these schedules based on detected activity levels—using shorter sleep periods during high-activity contexts and longer sleep during low-activity periods (**srbinovski2016adaptive**). Empirical results demonstrate 30–40% sensing energy savings while maintaining acceptable

application performance for scenarios where events are sparse and predictable.

Dynamic sampling rate adjustment varies the frequency at which sensors acquire data based on inferred system state. During periods of low activity (such as sleep or sitting), sampling rates can be reduced from 100 Hz to 10–20 Hz with minimal information loss. When activity increases, rates scale back to higher frequencies to capture rapid motion. This approach achieves 20–40% energy savings by avoiding high-rate acquisition when temporal dynamics are slow (**lee2016dynamic**).

These adaptive sensing techniques successfully reduce sensing energy but suffer from a fundamental limitation: they operate independently of the machine learning model. Duty cycling decisions are made based on simple heuristics (time-based schedules or coarse activity detection) without knowledge of which specific temporal segments contain discriminative features for the classification task. A duty cycling scheme that powers sensors off every 3 seconds may miss critical moments when diagnostic patterns appear—such as the initial phase of a fall event or a brief cardiac arrhythmia. Dynamic sampling that reduces rate during "low activity" may discard subtle discriminative features in gait patterns that enable early detection of mobility decline. Conservative policies that avoid missing events achieve only modest energy savings (20–30%), while aggressive policies risk significant accuracy degradation (10–20% accuracy loss).

What is needed is integration between sensing decisions and machine learning inference. Rather than applying fixed or heuristic-based sensing policies, the ML model's prediction confidence should guide data acquisition: when partial observations enable confident classification, sensing can stop; when uncertainty remains high, continued acquisition is justified. This ML-integrated approach could achieve the sensing energy savings of adaptive sensing methods while avoiding their accuracy risks, by making informed decisions about which data is truly necessary rather than applying blind policies. However, prior adaptive sensing work does not provide this integration, as it was designed to operate independently of ML models.

2.2 Early-Exit Neural Networks

Early-exit neural networks reduce computational costs by terminating inference at intermediate network layers when prediction confidence is sufficient. These architectures attach classification branches at multiple depths, allowing confident predictions to exit early rather than processing through all layers. While effective for reducing computational energy, these approaches maintain a fundamental limitation: they require complete input data before inference begins.

2.2.1 BranchyNet and MSDNet

BranchyNet pioneered the early-exit concept for convolutional neural networks (**teerapittayanon2016bra**
The architecture augments a base CNN (AlexNet or VGG) with exit branches attached at multiple intermediate layers. Each branch consists of a small classifier with one or two fully connected layers followed by softmax output, designed to provide increasingly refined predictions as network depth increases. Early branches produce coarse classifications based on low-level features, while later branches leverage more abstract hierarchical representations. During inference, the network processes data layer by layer, computing prediction entropy at each exit point. When entropy falls below a predefined threshold—indicating high confidence—the network returns the prediction and terminates processing. Otherwise, computation continues to the next layer. Training employs joint optimization of all branches using weighted loss $L = \sum_{i=1}^{K} w_i L_i$, where L_i represents the cross-entropy loss at exit i and weights w_i increase with depth to ensure deeper exits maintain high accuracy. Experimental evaluation on CIFAR-10 and ImageNet demonstrated 20–30% computational reduction compared to standard models while maintaining accuracy within 1–2%. However, BranchyNet requires complete images before processing begins—the early exits save computation, not data acquisition.

MSDNet improved upon BranchyNet through multi-scale dense connections (**huang2018multi**).

Rather than attaching simple classifier branches to a linear backbone, MSDNet employs multiple parallel processing streams operating at different spatial scales (resolutions). Dense connections within and across scales enable better feature reuse, allowing early exits to leverage multi-scale representations rather than only features at a single network depth. The architecture processes data through convolutional blocks at coarse, medium, and fine resolutions simultaneously, with dense connections propagating features both forward through the network and across scales. Exit classifiers placed at multiple depths can aggregate information from all three scales, producing more accurate early predictions than BranchyNet's single-scale branches. Training uses an anytime prediction objective where each exit is optimized independently to maximize accuracy at its depth, creating an ensemble-like behavior where early exits provide reasonable predictions while later exits refine them. Experimental evaluation on ImageNet and CIFAR-100 showed 30–40% computational reduction with better accuracy at early exits compared to BranchyNet—at the third exit point, MSDNet maintained 1.5% higher accuracy than BranchyNet while using similar computational resources. Despite these improvements, MSDNet shares BranchyNet's fundamental constraint: complete input data must be available before inference begins, leaving sensing energy unoptimized.

2.2.2 HarvNet for Energy Harvesting

HarvNet extended early-exit concepts to energy harvesting scenarios where available power varies over time (**jeon2023harvnet**). Energy harvesting systems extract power from environmental sources such as solar radiation, mechanical vibration, or ambient radio frequencies. Unlike battery-powered devices with static energy budgets, harvesting systems must adapt to fluctuating energy availability—solar panels generate 10–100 mW in direct sunlight but drop to 1–10 mW under cloud cover, while vibration harvesters produce power only during motion. HarvNet addresses this variability through adaptive confidence thresholds that adjust based on currently available energy. When harvested energy is abundant, the system employs low confidence thresholds, forcing inference to deep exits that maximize accuracy. When energy

is scarce, high thresholds enable early exits, sacrificing accuracy to ensure continuous operation. The architecture adds an energy predictor module that forecasts available power over the next inference window and a dynamic threshold controller that maps predicted energy to confidence thresholds using a learned policy.

The system operates in a continuous loop: measure current energy buffer state, predict energy availability for the next inference cycle, compute adaptive confidence thresholds for each exit, perform inference with early termination based on dynamic thresholds, and update energy state after inference. Experimental evaluation deployed HarvNet on a Jetson Nano equipped with a solar panel, running ImageNet classification under varying lighting conditions. Results demonstrated graceful accuracy degradation under energy constraints—when forced to use early exits due to low solar power, accuracy degraded by 5–8% compared to full-depth inference, but the system maintained continuous operation rather than experiencing power failures. When energy was plentiful, the system automatically used deeper exits, recovering full accuracy. HarvNet advanced beyond fixed-threshold early exits by introducing energy-aware adaptation, proving particularly valuable for harvesting deployments where power availability is inherently variable.

However, HarvNet shares the same fundamental limitation as BranchyNet and MSDNet: complete input data must be acquired before processing begins. For image classification, the camera captures full images regardless of exit depth. For time-series applications, sensors must collect complete windows before inference starts. The energy adaptation in HarvNet controls computational depth but does not influence sensing energy. In time-series contexts where sensing dominates total energy (60–80%), HarvNet's computational adaptations affect only the minority energy component, limiting total system savings to 5–20%. The approach proves effective for scenarios where computation dominates (such as video processing with hardware-accelerated sensing) or where sensing energy cannot be controlled (such as always-on microphones), but does not address the fundamental challenge of jointly optimizing sensing and computation.

2.2.3 Limitations: No Sensing Energy Optimization

Despite their computational efficiency, all early-exit neural network approaches share a fundamental limitation: they require complete input data before inference begins. For image classification, cameras must capture full-resolution photographs regardless of whether the network exits after three layers or thirty. For audio classification, microphones record complete utterances before processing starts. For time-series classification, sensors acquire entire windows $X[0:T]$ before the first convolutional layer receives input. This data-before-computation architecture stems from the way neural networks are designed: input layers have fixed dimensions, convolutional filters expect complete spatial or temporal receptive fields, and training procedures assume all inputs conform to a standard dimensionality. Early exits optimize the computational graph—deciding which layers to execute—but cannot modify the data collection process, which occurs entirely before computation begins.

The impact of this limitation varies dramatically across application domains. For computer vision on energy-harvesting systems, sensing energy may be negligible (solar-powered cameras) or uncontrollable (always-on surveillance), making computational optimization the appropriate focus. For time-series applications on battery-powered wearables, sensing energy dominates the total consumption budget (as established in Section 2.1.2). In this regime, computational early exits achieve at most 20–40% of 20–40% = 4–16% total system energy savings. A model that exits after 30% of layers (70% computational savings) still consumes 100% of sensing energy, yielding overall savings of approximately $0.7 \times 0.2 = 14\%$. Even perfect computational optimization (100% reduction) would save only 20–40% of total energy, leaving the majority untouched.

This architectural constraint creates a fundamental mismatch for sensor-rich applications. Time-series data often contains discriminative information early in temporal windows—sitting postures stabilize within 1 second of a 10-second activity recognition window, periodic gait patterns emerge within 2–3 seconds of walking observations, and many ECG

arrhythmias manifest within the first 3–4 seconds of 10-second monitoring windows. Early-exit neural networks cannot exploit this temporal information structure to reduce sensing because their architecture demands complete data before processing begins. A BranchyNet deployment for activity recognition must collect all 10 seconds of accelerometer data, then process it through convolutional layers with potential early exit. The fact that sitting could have been classified after 1 second is irrelevant—the network provides no mechanism to stop data acquisition mid-window.

What is needed are architectures that perform progressive inference—processing data as it arrives rather than waiting for complete windows. This requires fundamental changes to network design: input mechanisms that accommodate variable-length sequences, intermediate representations that can be updated incrementally as new data arrives, and integration between inference confidence and sensor control. While early-exit neural networks optimize computation efficiently, they cannot address sensing energy without these architectural innovations.

2.3 Early Time-Series Classification Methods

In contrast to early-exit neural networks that process complete data through fewer layers, early time-series classification (ECTS) methods aim to make predictions using only partial temporal sequences. These approaches address a different question: given access to a dataset of complete time-series, at what point during the series can accurate predictions be made? While this differs from real-time sensor control, ECTS methods provide valuable insights into when temporal information becomes sufficient for classification.

2.3.1 TEASER: Two-Tier Early Classification

TEASER introduced a two-tier hierarchical approach for early time-series classification (**schafer2020teaser**). Traditional time-series classifiers wait for complete sequences be-

fore prediction, maximizing accuracy at the cost of latency. Time-critical applications such as medical diagnosis, anomaly detection, and real-time monitoring benefit from earlier predictions even if confidence is slightly lower. TEASER addresses this trade-off through a two-stage architecture: an early classifier trained on partial sequences that provides fast predictions when confidence is high, and a full classifier trained on complete sequences that handles ambiguous cases requiring more temporal context.

The approach operates as follows. First, training creates an ensemble of classifiers for progressively longer prefix lengths—one classifier trained on the first 10% of series, another on 20%, continuing through 100%. Each classifier learns patterns discriminative at its prefix length. Second, calibration measures each classifier's accuracy on a validation set, establishing confidence thresholds for each prefix. Third, during inference, the ensemble processes increasingly long prefixes sequentially, checking confidence after each. If the prefix classifier's predicted probability exceeds its calibrated threshold, TEASER returns the prediction immediately. Otherwise, processing continues to the next prefix length. Finally, if no early classifier achieves sufficient confidence, the full classifier processes the complete sequence.

The confidence metric combines two signals: prediction probability from the classifier (higher is more confident) and ensemble agreement (multiple classifiers predicting the same class increases confidence). The decision mechanism employs minimum confidence thresholds that prevent predictions from extremely short prefixes where accuracy is unreliable, even if predicted probability is high. Additionally, minimum prefix length constraints ensure at least 20–30% of the series is observed before any prediction. The system computes a harmonic mean of earliness (fraction of series used) and accuracy (classification correctness), providing a balanced metric for evaluating the earliness-accuracy trade-off.

Experimental evaluation on 85 datasets from the UCR time-series archive demonstrated that TEASER achieves 80–90% of full-sequence accuracy using only 40–60% of series length on average. Across diverse domains including ECG classification, motion recognition, and sensor event detection, TEASER classified at 50% of series length on average while main-

taining accuracy within 5–10% of full-sequence baselines. Performance varied significantly across datasets—some series exhibited strong discriminative patterns in early segments enabling 70–80% earliness, while others required 60–80% of data for acceptable accuracy. This variation highlights that earliness potential depends heavily on how class-discriminative information is distributed temporally within specific domains.

For real-time energy optimization, TEASER faces fundamental limitations. The method assumes data is already collected and available in memory. During "inference," TEASER can retroactively decide at which point it would have been confident enough to predict, but in the original deployment scenario, the complete series was acquired before this decision. TEASER optimizes when to stop analyzing data, not when to stop acquiring it. In offline classification tasks—such as analyzing medical records, labeling sensor logs, or processing stored datasets—this distinction is irrelevant. However, for real-time edge deployment where sensors actively collect data, TEASER provides no mechanism to communicate early stopping decisions back to sensor hardware to trigger power state transitions. The approach reduces classification latency (producing predictions faster) but not sensing energy (which occurs before classification begins).

2.3.2 NMEC: Non-Myopic Early Classification

NMEC advanced early time-series classification through non-myopic decision-making (**romain2022nonmy**
Prior ECTS methods including TEASER make myopic decisions: at each timestep, they evaluate current confidence and either stop or continue without considering future information. This greedy approach is suboptimal when the value of waiting for more data varies across instances and timesteps. Some predictions might improve substantially with one more observation, while others have plateaued and further waiting provides negligible benefit. NMEC addresses this limitation by framing early classification as a cost-sensitive optimal stopping problem that considers future information gain.

The approach formulates classification as a sequential decision process. At each timestep

t, the system observes data $X[1:t]$, computes a prediction $\hat{y}_t$ with confidence c_t, and faces a decision: stop now and accept the current prediction, or continue to observe x_{t+1} at some cost. The cost function includes two components: misclassification cost C_{wrong} incurred if $\hat{y}_t \neq y_{\text{true}}$, and delay cost C_{delay} incurred for each additional timestep observed. The optimal policy minimizes expected total cost: $\pi^*(X[1:t]) = \arg\min_a \mathbb{E}[C_{\text{wrong}}(a) + C_{\text{delay}}(a)]$ where $a \in \{\text{stop}, \text{continue}\}$. The non-myopic component comes from estimating the value function: the expected cost of continuing considers not just immediate accuracy improvement from x_{t+1}, but the value of all future decisions conditioned on improved information.

The technical implementation uses reinforcement learning to learn the value function. The state representation includes current observations $X[1:t]$, the current predicted class distribution $P(y|X[1:t])$, and temporal metadata (current timestep, remaining budget). Actions are binary: stop or continue. The reward is negative cost, computed as $r = -C_{\text{wrong}} - C_{\text{delay}} \times t$ if stopping, or $r = -C_{\text{delay}}$ if continuing to the next timestep. A deep Q-network approximates the optimal action-value function $Q(s, a)$, trained through temporal difference learning on training sequences with known labels. At test time, the learned policy selects actions that minimize expected cost considering future information gain.

Experimental evaluation on UCR archive datasets and human activity recognition benchmarks demonstrated improved earliness-accuracy trade-offs compared to myopic methods. NMEC classified at 40–50% of series length on average, achieving 5–7% earlier predictions than TEASER at equivalent accuracy levels. For cost-sensitive scenarios where misclassification is expensive (such as medical diagnosis), NMEC maintained higher accuracy than TEASER while still achieving substantial earliness. The non-myopic planning proved particularly valuable for sequences where discriminative information arrives in bursts—the model learned to wait through uninformative regions and stop quickly once key patterns appeared, while myopic methods either stopped too early (missing critical patterns) or too late (continuing past the point where additional data improved predictions).

Despite these improvements, NMEC shares TEASER's fundamental limitation for energy-

efficient edge deployment. The method operates on pre-collected data stored in memory. While NMEC decides at which timestep to stop *analyzing* the series, this decision occurs after data acquisition is complete. The reinforcement learning policy has no mechanism to control sensors in real-time during data collection. Training requires complete sequences to evaluate counterfactual decisions ("what would have happened if I stopped earlier?"), making it difficult to learn policies that actually control streaming data acquisition. Like TEASER, NMEC optimizes latency (producing predictions faster) but cannot optimize sensing energy without architectural extensions that integrate classification decisions with sensor power control.

2.3.3 Broader ECTS Literature and Evaluation Metrics

The broader early time-series classification literature encompasses multiple methodological approaches, each addressing earliness from different algorithmic perspectives. Probability-based methods train classifiers on variable-length prefixes and estimate class probabilities from partial series. ECTS (**xing2011early**) uses one-nearest-neighbor classification with dynamic time warping distance, computing prediction confidence as the margin between the closest and second-closest training examples. When margin exceeds a threshold, the prediction is considered reliable. Minimum Prediction Length (MPL) (**mori2017early**) extends this by learning instance-specific minimum lengths—some examples require longer observation than others, and the method learns these requirements from training data. Distance-based methods compare partial series to class-representative templates. Early Abandon DTW terminates distance calculations early when partial distance already exceeds the best-seen distance, saving computation when match is clearly poor (**rakthanmanon2012searching**). Early Classification DTW stops when the best-matching template is unambiguous, terminating search when the gap between top-two distances indicates confident classification.

Shapelet-based methods extract discriminative subsequences (shapelets) characteristic of each class. Early Shapelet Classification (**xing2012early**) identifies which shapelets ap-

pear early in time-series and learns to classify based on their presence. Once a diagnostic shapelet is detected, classification terminates without observing remaining data. The approach proves effective for domains where class-discriminative patterns appear as localized temporal subsequences rather than being distributed throughout entire series. Deep learning approaches employ recurrent neural networks and attention mechanisms for early classification. EARLIEST (**hartvigsen2019early**) uses LSTM networks with a learned stopping policy: a controller network observes the LSTM hidden states at each timestep and decides whether to stop or continue. The controller learns through reinforcement learning with rewards based on accuracy and earliness. The model explicitly learns which temporal positions contain sufficient information for different classes, adapting stopping decisions to instance-specific characteristics.

These diverse approaches share common evaluation methodology. Earliness measures the fraction of series used before prediction, typically computed as $E = 1 - \frac{t_{\text{stop}}}{T}$ where t_{stop} is the timestep of prediction and T is total series length. Higher earliness (stopping sooner) is better. Accuracy measures classification correctness, computed as standard classification accuracy. The harmonic mean balances both objectives: $HM = 2 \times \frac{\text{Accuracy} \times \text{Earliness}}{\text{Accuracy} + \text{Earliness}}$, penalizing approaches that achieve one metric by sacrificing the other. Across diverse datasets and methods, common empirical findings emerge: 40–70% of series length is often sufficient for 80–90% of full-sequence accuracy, though variance across datasets is substantial. Instance-level variation dominates—within a dataset, some instances enable 80% earliness while others require 90% of data. Class imbalance affects earliness—rare classes typically require more data due to higher confidence thresholds needed to overcome prior probability.

2.3.4 Limitations: No Real-Time Sensor Control or Energy Focus

Despite their ability to classify with partial temporal data, early time-series classification methods have fundamental limitations for energy-efficient edge deployment. The most critical limitation is the pre-collected data assumption: all ECTS methods assume complete

time-series are already available in memory. Methods operate on datasets like the UCR archive, where each series is a fixed-length array stored as a single object. During "early" classification, algorithms process this array from beginning to end, deciding at which index to stop analysis. However, the complete array was collected and stored before this decision occurred. This offline processing model is appropriate for batch classification tasks—analyzing historical medical records, labeling stored sensor logs, or processing archived datasets. In these scenarios, data acquisition happened previously (perhaps years ago), and the goal is efficient retrospective analysis. Early stopping reduces computation time and can prioritize which instances to process fully, but cannot affect the energy already spent acquiring the data.

The second limitation is the absence of sensor control mechanisms. ECTS methods output predictions at early timesteps but provide no mechanism to communicate these decisions to sensor hardware. Consider deploying TEASER for real-time activity recognition. As sensor data streams in, TEASER could theoretically evaluate confidence after each new sample. However, the method has no interface to signal "stop acquiring data" to the accelerometer. Even if confidence is sufficient at timestep 100 of a 500-timestep window, the sensor continues running because TEASER is a classification algorithm, not a system-level controller. Implementing real-time sensor control requires architectural extensions that ECTS methods do not provide: buffering and progressive evaluation of streaming data, confidence estimation from partial inputs during live inference (not retrospectively on complete data), hardware interfaces to transition sensors to low-power states, and latency management ensuring control signals arrive before the next sample is acquired. These system-level components are orthogonal to the algorithmic contributions of ECTS research, which focuses on classification accuracy and earliness rather than hardware integration.

The third limitation is the focus on latency rather than energy. ECTS research is motivated by time-critical applications where faster predictions enable earlier intervention. Medical diagnosis systems benefit from predicting sepsis 30 minutes earlier, even if confidence is

slightly lower. Anomaly detection systems that identify failures 5 minutes sooner can trigger preventive maintenance before damage occurs. This latency focus is valuable, but latency and energy are distinct objectives. Faster classification (lower latency) does not necessarily reduce energy consumption. If sensors continue running throughout early classification because there is no control mechanism, then earliness provides zero energy savings even as it reduces time-to-prediction. In offline batch processing, earliness does reduce computational energy by processing fewer timesteps per instance, but this computational savings is small (20–40%) compared to real-time sensing energy (60–80%).

The impact on energy efficiency is that ECTS methods, as currently designed, optimize latency but not energy for real-time sensor deployments. Their algorithmic insights about discriminative temporal patterns are valuable: demonstrating that 40–60% of series suffices for accurate classification motivates sensor-aware architectures. However, translating these insights into energy savings requires additional innovations. Progressive inference architectures must process streaming data rather than complete arrays. Confidence estimates must be computed in real-time, not retrospectively. System integration must couple classification decisions to sensor power control. These extensions transform ECTS from a classification algorithm into a complete edge system capable of joint sensing-computation optimization.

2.4 Random Forests for Edge Deployment

Random forests offer an attractive alternative to neural networks for resource-constrained edge devices. Their memory efficiency, fast inference through simple tree traversal, and interpretability make them particularly suitable for ultra-low-power deployments where neural networks are infeasible. This section reviews random forest advantages for edge computing and prior optimization work.

2.4.1 Advantages for Resource-Constrained Devices

Random forests possess several properties that make them well-suited for edge deployment where memory, computation, and energy are severely constrained. Memory efficiency stems from the simplicity of tree data structures. Each node stores only a feature index, a threshold value, and pointers to child nodes (for internal nodes) or class vote counts (for leaf nodes). A typical tree with depth $d = 20$ contains approximately $2^{20} - 1 \approx 1$ million nodes, but with quantized thresholds and compact representations, storage requires only 1–2 MB. A forest of 100 such trees occupies 100–200 MB, compared to convolutional neural networks requiring 500 MB to several gigabytes for comparable time-series classification accuracy. This 5–10× memory advantage enables deployment on microcontroller-class devices with limited RAM and flash storage (**breiman2001random**; **fernandez2014we**).

Computational efficiency arises from the inference algorithm: tree traversal through if-else comparisons. Starting at the root, each node performs a single comparison $x[f] > \theta$ where f is the feature index and θ is the threshold. Based on the result, inference proceeds to the left or right child. This process requires d comparisons per tree, with no floating-point arithmetic (only integer comparisons if features are quantized). Cache locality is excellent—tree nodes are accessed sequentially during traversal, and entire small trees often fit in L1 cache. Total inference time is $O(N \times d)$ where N is the number of trees, typically executing in microseconds on embedded processors. Compare this to convolutional neural networks requiring millions of multiply-accumulate operations across multiple layers, executing in milliseconds even with hardware acceleration (**biau2016random**).

Training efficiency enables on-device learning and incremental model updates. Random forests can be trained on edge devices because each tree is built independently through recursive binary splitting, requiring minimal memory (only the training subset for the current node). Trees can be trained in parallel across multiple cores or sequentially with intermediate checkpointing. Incremental learning adds new trees to the ensemble without retraining

existing trees, enabling models to adapt to changing conditions or personalize to individual users. Neural networks, in contrast, require full-batch gradient descent with all training data in memory simultaneously and backpropagation through all layers, typically infeasible on microcontrollers (**oshiro2012many**).

Interpretability provides debugging capabilities and user trust. Feature importance scores identify which sensors contribute most to predictions, enabling sensor selection and power management. Individual decision paths can be traced through trees, making model behavior understandable to clinicians and engineers. For safety-critical applications like medical monitoring, this interpretability is often a regulatory requirement. Neural networks offer no such transparency—predictions emerge from millions of weight interactions across dozens of layers, providing little insight into which input features drove specific outputs (**louppe2013understanding**).

Robustness to noisy data arises from ensemble averaging and the discrete nature of tree splits. Individual trees are high-variance models prone to overfitting, but averaging predictions across 50–100 trees reduces variance substantially through the bias-variance trade-off. Random feature selection at each split (typically $\sqrt{F}$ features where F is total features) decorrelates trees, increasing ensemble diversity. Outliers and sensor glitches affect only the few trees that split on corrupted features, while the majority of trees remain unaffected. Neural networks, with their continuous learned representations, can be more sensitive to input perturbations unless explicitly regularized.

Accuracy is competitive with neural networks for many time-series classification tasks. On the UCI HAR dataset for activity recognition, random forests achieve approximately 95% accuracy, comparable to convolutional neural networks (94–96%) and recurrent networks (93–95%). On PAMAP2 physical activity monitoring, random forests reach approximately 90% accuracy, similar to LSTM networks (89–92%). Random forests do not universally match neural network accuracy—for complex temporal dependencies and high-dimensional sequences, deep networks typically outperform forests. However, for many practical edge

applications where accuracy in the 90–95% range suffices, random forests offer a compelling trade-off: similar accuracy with 10–100× lower resource requirements.

2.4.2 Prior Random Forest Optimization Work

Prior research has explored various techniques to optimize random forests for edge deployment, targeting model size, inference speed, and memory usage. Tree pruning removes nodes that provide minimal accuracy improvement, reducing both model size and inference time. Reduced error pruning evaluates each node's contribution on a validation set: if removing the node (converting to a leaf) does not decrease validation accuracy, the node is pruned. This bottom-up process continues until further pruning harms accuracy. Cost-complexity pruning weighs tree size against accuracy through a complexity parameter α: $\min_{\text{tree}} \text{Error}(\text{tree}) + \alpha \times |\text{tree}|$ where $|\text{tree}|$ is the number of nodes. Cross-validation selects α that balances accuracy and compactness. Empirical results typically achieve 20–40% node reduction with less than 1% accuracy loss (**quinlan1987simplifying**; **breiman2001random**).

Feature selection reduces the number of input features, shrinking feature vectors and simplifying trees. Importance-based selection ranks features by their contribution to forest accuracy—measured as the average decrease in impurity across all splits on that feature—and retains only the top k features. Minimum redundancy maximum relevance (mRMR) selects features that are maximally informative about class labels while minimally redundant with already-selected features. ReliefF iteratively updates feature weights based on nearest-neighbor analysis, up-weighting features that distinguish between classes and down-weighting noisy features. Reducing features from 500 to 100 typically yields 3–5× faster inference with 2–4% accuracy degradation, an acceptable trade-off for many applications (**genuer2010variable**; **kursa2010feature**).

Tree structure optimization improves split quality and depth. Oblique splits use linear combinations of features $\sum_i w_i x[i] > \theta$ rather than single-feature splits $x[f] > \theta$, creating

more expressive decision boundaries that can achieve similar accuracy with shallower trees. However, oblique splits require solving optimization problems at each node, increasing training time substantially. Soft trees use probabilistic routing—observations follow both branches with probabilities determined by a learned function, then predictions are averaged—enabling smoother decision boundaries. These structural modifications achieve better accuracy-depth trade-offs, reducing tree depth by 20–30% for equivalent accuracy, but at the cost of increased training complexity and inference overhead for probabilistic routing (**murthy1994system**; **irsoy2012soft**).

Quantization reduces numerical precision of thresholds and feature values. Training typically uses 32-bit floating-point thresholds, but inference can use 16-bit or 8-bit integers with minimal accuracy loss. Feature quantization discretizes continuous sensors into bins, converting comparisons from floating-point to integer operations. Combined 16-bit quantization typically reduces memory footprint by 50% and speeds inference by 20–30% due to faster integer comparisons and better cache utilization. Accuracy degradation is typically less than 1% because tree splits are already discrete, and slight threshold perturbations rarely change classification outcomes (**wu2014quantization**).

Hardware acceleration implements random forest inference on FPGAs and ASICs. FPGAs provide reconfigurable hardware that can implement tree traversal in custom circuits, parallelizing multiple trees simultaneously and achieving 10–50× speedup over CPU implementations. Application-specific integrated circuits designed for specific forests achieve even higher efficiency by eliminating programmability overhead, but require fixed models that cannot be updated. These hardware approaches prove valuable for high-throughput applications like network intrusion detection, where millions of packets per second must be classified, but are less practical for low-volume edge deployments where development costs exceed benefits (**narayanan2016hardware**).

What unifies these optimization techniques is their focus on model compression and computational efficiency. Pruning, feature selection, and quantization reduce model size and

inference time. Tree structure optimization improves accuracy-complexity trade-offs. Hardware acceleration increases throughput. However, none of these approaches address runtime energy optimization through adaptive inference. Trees are either evaluated completely or not at all—there is no mechanism to stop inference early when confidence is sufficient. All optimization occurs at design time (pruning, feature selection) or through static architectural choices (quantization, hardware implementation). The opportunity for runtime adaptation based on instance-specific characteristics remains unexplored in prior random forest optimization work.

2.4.3 Daghero RF: Computational Early Stopping Only

Daghero et al. introduced computational early stopping for random forests, focusing on reducing inference energy by evaluating fewer trees when confidence is sufficient (**daghero2021energy**). The approach addresses the observation that not all instances require all N trees to achieve accurate predictions. Some instances receive unanimous votes from the first 20–30 trees, making evaluation of remaining 70–80 trees unnecessary. The method introduces a progressive voting scheme where trees are evaluated sequentially, tracking cumulative class votes after each tree. When the vote margin between the top two classes exceeds a threshold—indicating high confidence—inference terminates and returns the majority class without evaluating remaining trees.

The confidence metric is vote margin: $M_t = \frac{V_1(t)-V_2(t)}{t}$ where $V_1(t)$ is the number of votes for the leading class after t trees and $V_2(t)$ is votes for the second-place class. Larger margins indicate more confident predictions. The threshold τ determines when to stop: if $M_t > \tau$, inference terminates. Lower thresholds enable earlier stopping (fewer trees evaluated) but risk higher error rates from premature decisions. Higher thresholds ensure accuracy but reduce computational savings. Threshold selection uses validation-based tuning, sweeping values from 0.1 to 0.9 and selecting the value that maximizes accuracy-energy trade-offs for the specific dataset.

Implementation optimizations improve effectiveness. Trees are ordered by individual accuracy, with the most accurate trees evaluated first. This front-loading of strong classifiers increases early confidence, enabling earlier stopping. During training, each tree's validation accuracy is computed, and trees are sorted in descending accuracy order for inference. Additionally, dynamic thresholds adapt to class difficulty: harder-to-classify instances (those with lower initial confidence) use lower thresholds to avoid excessive computation, while easier instances use higher thresholds to prevent premature stopping.

Experimental evaluation on activity recognition datasets including UCI HAR and PAMAP2 demonstrated 20–30% reduction in trees evaluated on average. For easy instances (sitting, standing), only 10–15% of trees were needed, achieving 90–95% computational savings. For harder instances (transitional activities), 60–80% of trees were required, yielding 20–40% savings. Aggregated across typical activity distributions, overall computational energy savings reached 20–30% while maintaining accuracy within 1% of full-forest baselines. Latency improvements were similar, with inference time reduced proportionally to the number of trees evaluated.

However, Daghero's approach shares the same fundamental limitation as early-exit neural networks: it addresses only computational energy, not sensing energy. The method requires complete input feature vectors before inference begins. For time-series classification, this means sensors must acquire entire T-length windows, compute all features (statistical descriptors, frequency components), and load the complete feature vector into memory before tree evaluation starts. Early stopping saves computation by evaluating fewer trees, but sensing energy (60–80% of total) remains unchanged. An activity recognition system that uses only 30% of trees still collects 100% of sensor data, yielding total system savings of $0.7 \times 0.2 = 14\%$. For deployment scenarios where computational energy is already a small fraction of the budget, further computational optimization provides diminishing returns.

The approach proves most valuable for applications where computation dominates energy—such as cloud-based inference where data is already collected or hardware-accelerated sensing

where sensor energy is negligible. For resource-constrained edge wearables where sensing energy is the primary bottleneck, Daghero RF addresses the minority energy component. What is needed are architectures that extend early stopping to sensing as well: progressive inference mechanisms that evaluate trees on partial temporal windows and sensor control interfaces that transition sensors to low-power states when tree voting confidence is sufficient. Daghero RF provides a computational baseline, demonstrating that tree-level early exit is feasible and effective, but does not complete the path to joint sensing-computation optimization.

2.5 Design Space Exploration and Bayesian Optimization

Sensor-aware early-exit approaches require selecting confidence thresholds for each exit point, creating a multi-dimensional optimization problem. With K exits, the threshold space is K-dimensional and continuous, with non-linear, non-convex relationships between thresholds and accuracy-energy trade-offs. This section reviews optimization methodologies for efficiently searching threshold spaces.

2.5.1 Design Space Exploration for Threshold Selection

Design Space Exploration (DSE) encompasses systematic approaches to searching parameter spaces. The goal is to find optimal or near-optimal configurations while managing the computational cost of evaluating candidates. For threshold selection, each configuration is a K-dimensional vector $\boldsymbol{\tau} = (\tau_1, \tau_2, \ldots, \tau_K)$ where τ_i is the confidence threshold for exit i. Evaluating a configuration requires training the model (or loading pre-trained weights), running validation inference with the specified thresholds, and computing accuracy and energy consumption across the validation set. This evaluation is expensive: minutes to hours depending on dataset size and model complexity. With $K = 3$ exits and 10 candidate threshold values per dimension, exhaustive search evaluates $10^3 = 1{,}000$ configurations, requiring substantial computation (**bergstra2012random**).

Exhaustive search evaluates all possible configurations within a discretized space. If each threshold is sampled at G grid points, exhaustive search evaluates G^K configurations. This approach guarantees finding the global optimum within the discretized space, but computational cost grows exponentially with dimensionality. For $K = 2$ and $G = 10$, exhaustive search requires 100 evaluations—manageable. For $K = 4$ and $G = 10$, cost grows to 10,000 evaluations—infeasible for practical timescales. Exhaustive search proves viable only for low-dimensional spaces ($K \leq 3$) or when evaluation is extremely fast.

Grid search samples at regular intervals across each dimension, creating a uniform lattice of points. With G samples per dimension, grid search evaluates G^K configurations, identical to exhaustive search when the grid is complete. However, grid search can use coarser sampling (fewer points per dimension) to reduce cost at the expense of potentially missing optimal regions between grid points. A coarse grid with $G = 5$ evaluates only 5^K configurations, providing 32× speedup for $K = 3$ compared to $G = 10$ exhaustive search. Grid search provides uniform coverage of the space, ensuring no regions are left unexplored, but wastes effort uniformly across both promising and unpromising regions (**bergstra2012random**).

Random search samples configurations uniformly at random from the space, evaluating a fixed budget of B configurations. Surprisingly, random search often outperforms grid search for the same computational budget. Bergstra and Bengio (**bergstra2012random**) demonstrated that for high-dimensional spaces where only a few dimensions significantly affect the objective, random search provides better coverage of important dimensions than grid search. If only 2 of 5 dimensions matter, grid search with $G = 10$ samples each of those 2 dimensions at only 10 values. Random search with budget $B = 100$ samples each important dimension at approximately 100 different values (though irregularly spaced), exploring more of the critical subspace. Random search with budget $B = 100$ typically outperforms 10^K exhaustive search for $K \geq 3$, and always costs exactly B evaluations regardless of dimensionality.

Coarse-to-fine search starts with a coarse grid to identify promising regions, then refines those regions with denser sampling. The initial coarse phase evaluates a 3^K grid (3 samples

per dimension: low, medium, high values), identifying the top 10–20% of configurations. The refinement phase samples densely around these promising configurations, using a finer grid within local neighborhoods. This hierarchical approach achieves logarithmic refinement: doubling resolution requires only 2^K additional evaluations rather than G^K evaluations of a fine grid from scratch. Coarse-to-fine search balances coverage and efficiency, providing good solutions in $O(3^K + L \times 2^K)$ evaluations where L is the number of refinement levels.

Design Space Exploration methods offer simplicity and parallelism. They are trivial to implement, requiring no specialized libraries or algorithms. All configurations can be evaluated in parallel, limited only by available computational resources. DSE makes no assumptions about the objective function—it works for discrete, continuous, non-differentiable, and stochastic objectives. However, DSE treats all evaluations equally, learning nothing from previous results. The 100th evaluation is chosen identically to the 1st, ignoring that previous evaluations revealed certain regions as promising or unpromising. This lack of learning wastes computational budget on redundant evaluations of bad regions.

2.5.2 Bayesian Optimization for Efficient Search

Bayesian Optimization (BO) provides sample-efficient global optimization for expensive black-box functions (**shahriari2016taking**; **frazier2018tutorial**). Unlike grid or random search, BO builds a probabilistic model of the objective function and uses this model to intelligently select which configurations to evaluate next, focusing effort on promising regions while maintaining sufficient exploration to avoid local optima.

Bayesian Optimization operates through three core components. First, the surrogate model is a probabilistic model of the objective function $f(\boldsymbol{\tau})$, most commonly a Gaussian Process (GP). A GP defines a prior distribution over functions, representing f as a distribution over function values at any point $\boldsymbol{\tau}$. After observing function values $y_i = f(\boldsymbol{\tau}_i)$ at points $\{\boldsymbol{\tau}_1, \ldots, \boldsymbol{\tau}_n\}$, the GP posterior distribution provides both a mean prediction $\mu(\boldsymbol{\tau})$ and uncertainty estimate $\sigma(\boldsymbol{\tau})$ at any unobserved point. The mean represents the model's best guess

of the function value, while uncertainty quantifies confidence—high uncertainty indicates regions not yet well-explored. The GP updates after each new observation, incorporating the information and refining predictions across the entire space (**rasmussen2006gaussian**).

Second, the acquisition function determines which point to evaluate next, balancing exploitation (sampling where the model predicts high objective values) against exploration (sampling where uncertainty is high, to improve the model). Common acquisition functions include Expected Improvement (EI), which computes the expected amount by which $f(\boldsymbol{\tau})$ exceeds the current best observation, accounting for both mean and uncertainty; Upper Confidence Bound (UCB), which selects $\boldsymbol{\tau}$ maximizing $\mu(\boldsymbol{\tau}) + \kappa\sigma(\boldsymbol{\tau})$ where κ controls exploration-exploitation trade-off; and Probability of Improvement (PI), which selects points with high probability of exceeding the current best. Expected Improvement typically performs best, providing a principled balance: it favors high-mean regions (exploitation) but also high-uncertainty regions (exploration), automatically adjusting the balance as optimization progresses (**jones1998efficient**).

Third, the optimization loop iterates: initialize with a small set of random evaluations to seed the GP (typically 5–10 points), fit the GP to observed data $\{(\boldsymbol{\tau}_i, y_i)\}$, optimize the acquisition function to find $\boldsymbol{\tau}_{\text{next}} = \arg\max_{\boldsymbol{\tau}} \text{Acquisition}(\boldsymbol{\tau})$, evaluate the true objective $y_{\text{next}} = f(\boldsymbol{\tau}_{\text{next}})$, update the GP with the new observation, and repeat until budget exhausted or convergence. Each iteration strategically selects the next evaluation to maximally improve the model or find better solutions, learning from all previous evaluations.

Bayesian Optimization offers several advantages over Design Space Exploration. Sample efficiency achieves good solutions in 20–100 evaluations, compared to 1,000+ for grid search in 3D spaces. This 10–100× speedup is critical when evaluations are expensive (hours of training). Intelligent learning focuses effort on promising regions identified through previous evaluations, avoiding wasted computation in obviously poor regions. Uncertainty quantification provides not just a best solution but confidence intervals, enabling risk-aware decisions. Principled exploration prevents premature convergence to local optima through the explo-

ration component of acquisition functions. However, BO has disadvantages: implementation complexity requires specialized libraries (GPyOpt, Spearmint, BoTorch) and understanding of GP hyperparameters (kernel choice, length scales). Computational overhead for fitting GPs and optimizing acquisition functions adds seconds to minutes per iteration, which is negligible when $f(\boldsymbol{\tau})$ takes hours but noticeable when $f(\boldsymbol{\tau})$ is fast. Scalability degrades for high dimensions: standard GPs scale poorly beyond 15–20 dimensions due to cubic cost in the number of observations for GP inference. Specialized variants like random embeddings or sparse GPs extend to 50–100 dimensions (**wang2013bayesian**).

For threshold selection with $K = 3$ exits, Bayesian Optimization typically finds near-optimal solutions in 50–100 evaluations compared to 1,000 for fine-grained grid search. The solutions found by BO are typically within 1–2% of global optima (when compared against exhaustive search on small problems), while requiring 90–95% fewer evaluations. This efficiency makes BO the method of choice for expensive hyperparameter optimization problems in machine learning, including neural architecture search, algorithm configuration, and—as relevant to this thesis—early-exit threshold tuning.

2.5.3 Applications in Machine Learning

Bayesian Optimization has been successfully applied to diverse machine learning optimization problems where evaluations are expensive. Hyperparameter tuning optimizes learning rates, regularization strengths, batch sizes, and optimizer parameters for neural networks. Auto-sklearn and Auto-WEKA employ BO to search hyperparameter spaces with 10–20 dimensions, finding configurations that outperform default settings by 5–10% in accuracy while requiring only 50–100 training runs compared to thousands for grid search (**feurer2015efficient**; **thornton2013autoweka**). Neural Architecture Search uses BO to optimize architectural choices such as number of layers, filter sizes, and skip connections. NASBOT applies BO to the architecture space, discovering networks that match manually designed architectures in accuracy while using 100× fewer evaluations than re-

inforcement learning based NAS (**kandasamy2018neural**). BANANAS further improves efficiency through neural predictors, finding competitive architectures in 10–20 evaluations.

Algorithm configuration optimizes parameters of machine learning algorithms themselves. SMAC (Sequential Model-based Algorithm Configuration) uses random forests rather than GPs as surrogates, handling discrete and conditional parameters common in algorithm spaces (**hutter2011sequential**). SMAC has been applied to configure solvers, planners, and learning algorithms, achieving 10–100× speedups over default configurations. Multi-objective optimization handles trade-offs between competing objectives such as accuracy versus latency, accuracy versus model size, or—most relevant to this thesis—accuracy versus energy consumption. ParEGO extends BO to multi-objective settings by scalarizing objectives into a single weighted sum, then optimizing different weight combinations to discover Pareto frontiers (**knowles2006parego**). MOBO uses multi-output GPs that model all objectives jointly, enabling more efficient exploration of trade-off spaces.

The relevance to threshold selection is direct: early-exit thresholds define a multi-objective optimization problem where the goals are to maximize classification accuracy and minimize energy consumption. With K exits, the space is K-dimensional and continuous. The objective function $f(\boldsymbol{\tau}) = w_{\text{acc}} \cdot \text{Accuracy}(\boldsymbol{\tau}) - w_{\text{energy}} \cdot \text{Energy}(\boldsymbol{\tau})$ is expensive to evaluate, requiring validation inference across thousands of examples. The space is non-convex with multiple local optima—thresholds interact in complex ways, as setting exit 2's threshold affects how many instances reach exit 3. Bayesian Optimization is ideal for this setting: it efficiently explores the K-dimensional space, balances the accuracy-energy trade-off through multi-objective acquisition functions, and converges in 50–100 evaluations rather than 1,000+ for grid search. Existing work on threshold optimization has primarily relied on grid search or manual tuning. This thesis contributes systematic application of Bayesian Optimization to multi-exit threshold selection, demonstrating substantial computational savings while achieving near-optimal solutions.

2.6 Summary and Gap Analysis

This chapter surveyed related work across five key areas: time-series classification on edge devices, early-exit neural networks, early time-series classification methods, random forests for edge deployment, and optimization techniques for threshold selection. Each area addresses part of the energy efficiency challenge, yet no single approach provides a complete solution for joint sensing-computation optimization on resource-constrained devices.

2.6.1 Summary of Related Work

Time-series classification on edge devices faces severe resource constraints, with sensing energy consuming 60–80% of total system energy as established in Chapter 1. Existing edge optimization strategies including model compression, neural architecture search, and hardware acceleration focus exclusively on computational efficiency. While these methods reduce inference latency and computational energy by 50–80%, they leave sensing energy entirely unaddressed because they require complete input windows before processing begins.

Early-exit neural networks including BranchyNet, MSDNet, and HarvNet reduce computational costs by 20–40% through intermediate exit points that terminate inference when confidence is sufficient. HarvNet extends this concept to energy harvesting scenarios with variable power availability. However, all early-exit approaches share a fundamental limitation: they require complete input data before inference begins. For image classification, cameras must capture full frames. For time-series classification, sensors must acquire entire temporal windows. This architecture saves computation but cannot reduce sensing energy, limiting total system savings to 4–16% for sensor-dominated applications.

Early time-series classification methods including TEASER, NMEC, and the broader ECTS literature demonstrate that accurate predictions are often possible using only 40–70% of temporal sequences. These methods achieve substantial earliness while maintaining 80–95% of full-sequence accuracy. However, ECTS methods operate on pre-collected data

stored in memory, deciding when to stop analyzing rather than when to stop acquiring. They provide no mechanisms for real-time sensor control, no interfaces to transition sensors to low-power states, and focus on latency optimization rather than energy reduction. Their insights about temporal information distribution are valuable, but translating these insights into energy savings requires architectural extensions that ECTS research does not address.

Random forests offer compelling advantages for resource-constrained edge devices: 10–100× lower memory footprint than neural networks, faster inference through tree traversal, and inherent interpretability. Prior random forest optimization work including pruning, feature selection, quantization, and hardware acceleration reduces model size and inference time by 20–50%. Daghero et al. introduced computational early stopping for random forests, evaluating fewer trees when vote confidence is sufficient and achieving 20–30% computational energy savings. However, like early-exit neural networks, this approach requires complete input features and does not address sensing energy.

Design Space Exploration and Bayesian Optimization provide methodologies for threshold selection in multi-dimensional continuous spaces. DSE methods including grid search, random search, and coarse-to-fine search offer simplicity and parallelism but treat all evaluations equally without learning from results. Bayesian Optimization achieves 10–100× better sample efficiency through probabilistic modeling of the objective function and intelligent selection of evaluation points. BO has proven effective for hyperparameter tuning, neural architecture search, and multi-objective optimization in machine learning. However, application to early-exit threshold selection has been limited in prior work.

2.6.2 The Critical Gap

The critical gap across all surveyed work is the absence of approaches that jointly optimize sensing and computational energy for time-series classification on edge devices. Existing methods optimize only one dimension of the system, leaving the majority of energy consumption unaddressed.

Early-exit neural networks achieve 20–40% computational savings but require complete sensor data (Section 2.2.3), limiting total system savings to 4–16%. Even perfect computational optimization would save only 20–40% of total energy, as sensing remains unchanged.

Early time-series classification methods demonstrate that 40–70% of temporal sequences suffice for accurate classification, but operate retrospectively on pre-collected data without real-time sensor control (Section 2.3.4). They optimize when to stop analyzing data, not when to stop sensing data.

Adaptive sensing techniques reduce sensing energy by 30–40% but operate independently of machine learning models (Section 2.1.4), lacking knowledge of which temporal segments contain class-discriminative features. Without ML integration, these methods risk accuracy degradation or require conservative policies that limit energy savings.

Random forest early stopping saves 20–30% computational energy but requires complete feature vectors before inference (Section 2.4.3), leaving sensing energy unchanged. Total system energy savings remain limited to 5–15% of the overall budget.

What is fundamentally missing is joint optimization of sensing and computation through progressive inference. Needed capabilities include: architectural designs that process partial temporal windows rather than requiring complete fixed-length inputs; confidence estimation from incomplete data that is calibrated and reliable for driving hardware control decisions; real-time sensor control interfaces that transition sensors to low-power states when ML confidence is sufficient; and system-level integration that couples inference decisions to hardware power management. Existing approaches address subsets of these requirements but no prior work integrates all four capabilities into a unified framework.

2.6.3 Gap Analysis Table

Table 2.1 summarizes capabilities of existing approaches compared to requirements for joint sensing-computation optimization. Early-exit neural networks process complete data and optimize computation but cannot handle partial inputs or control sensors. Early time-series

classification methods classify partial data and reduce computational load but operate offline without sensor control. Adaptive sensing controls sensor power but operates independently of ML without using partial data for decisions. Random forest early stopping optimizes computation but requires complete features. None of the surveyed approaches jointly optimize both sensing and computational energy.

Table 2.1 Gap Analysis: Existing Approaches vs. Required Capabilities for Joint Sensing-Computation Optimization

Approach	**Partial Data**	**Sensor Control**	**Compute Optim.**	**Joint Optim.**	**Total Energy Savings**
Early-Exit NNs (BranchyNet, MSDNet)	55	55	51	55	4–16%
Early TSC (NMEC, TEASER)	51	55	51	55	5–15% (latency only)
Adaptive Sensing (Duty Cycling)	55	51	55	55	30–40% (risks accuracy)
RF Early Stopping (Daghero RF)	55	55	51	55	5–15%
SEE Framework (This Thesis)	**51**	**51**	**51**	**51**	**50–70%**

The table illustrates four required capabilities. Partial Data processing enables inference on incomplete temporal windows, eliminating the requirement to wait for complete data before starting classification. Sensor Control provides mechanisms to transition sensors to low-power states when ML decisions indicate sufficient information has been acquired. Compute Optimization reduces inference energy through early termination when confidence is adequate. Joint Optimization integrates sensing and computation decisions, achieving

multiplicative rather than additive savings by optimizing both dimensions simultaneously. Total Energy Savings quantifies end-to-end system efficiency, demonstrating that approaches optimizing only one dimension achieve limited improvements (5–40%) while joint optimization can achieve 50–70% savings by addressing both the dominant sensing component and the computational component.

2.6.4 Positioning the Contribution

This thesis addresses the identified gap through the Sensor-Aware Early-Exit (SEE) framework (detailed contributions in Section 1.3), which advances beyond each area of related work through three key innovations: architectural support for progressive inference, integrated sensor control, and systematic threshold optimization.

First, architectural innovations enable progressive inference on partial temporal windows. SEE-CNN employs late input blocks that merge newly arriving sensor data with previously extracted features (Section 3.3). SEE-vRF uses vertical tree splitting where individual trees are traversed progressively with increasing depth limits as more data arrives (Section 3.4). SEE-hRF employs horizontal tree partitioning into cascaded sub-forests that process incremental data segments (Section 3.5). All three architectures accommodate variable-length input rather than requiring fixed-window dimensionality, enabling inference to proceed as data streams in.

Second, integrated sensor-aware frameworks couple ML confidence to hardware power control. Confidence estimates are computed at each exit point based on prediction entropy (for neural networks) or vote margins (for random forests). When confidence exceeds exit-specific thresholds, the framework signals sensor hardware to transition to low-power states through GPIO interfaces or power management APIs. This closed-loop integration ensures that sensing energy is reduced in proportion to confidence, with easy instances consuming minimal sensing budget and difficult instances receiving complete windows only when necessary. The framework operates in real-time on streaming data, making sensor control de-

cisions before unnecessary data is acquired rather than retroactively after complete windows are collected.

Third, systematic optimization methodologies address the exponential growth of threshold design spaces (Section 3.6). A two-stage approach combines coarse-grained Design Space Exploration to identify promising regions followed by multi-objective Bayesian Optimization for fine-grained tuning. This hybrid methodology reduces optimization time by 90% compared to exhaustive search while producing solutions within 2% of global optima, making threshold selection tractable for practical deployment scenarios with multiple exits and complex accuracy-energy trade-offs.

The SEE framework advances beyond each area of related work in specific ways. Compared to early-exit neural networks, SEE adds sensor control and partial data processing, enabling joint optimization rather than computational-only optimization. Compared to early time-series classification methods, SEE operates in real-time with hardware integration rather than retrospective offline analysis, translating earliness insights into actual energy savings. Compared to adaptive sensing, SEE integrates ML confidence to make informed decisions about which temporal segments are discriminative rather than applying fixed policies that risk missing critical features. Compared to random forest computational optimization, SEE extends early exit from tree-level to temporal-level, enabling progressive inference that reduces both sensing and computation rather than only tree evaluation costs.

The empirical contributions include comprehensive evaluation across six diverse datasets spanning health monitoring, activity recognition, and brain-computer interfaces, demonstrating 50–70% average total energy savings while maintaining accuracy within 2% of full-window baselines. Real-world hardware validation on Raspberry Pi with physical accelerometers and direct energy measurements confirms that simulation-based estimates are accurate within 5%, proving practical feasibility beyond theoretical analysis. Systematic ablation studies isolate the contributions of progressive inference, joint optimization, and threshold tuning, providing insights into which design choices drive performance. Comparative analysis against

compute-only baselines (Daghero RF, early-exit CNN without sensor control) and sensing-only baselines (duty cycling, uniform sampling) demonstrates synergistic benefits of joint optimization, showing that SEE achieves 2–3× greater energy savings than either dimension optimized independently.

The next chapter presents the SEE framework in detail, describing progressive architectures for both convolutional neural networks and random forests, the confidence-based exit decision mechanisms, sensor control integration protocols, and the two-stage threshold optimization methodology combining Design Space Exploration and Bayesian Optimization.

CHAPTER THREE

PROPOSED METHODOLOGY

NOTE: Portions of this chapter are based on collaborative research:

Lubah Nelson, Sirine Belhaj, Negar Haghpanah, Dina Hassan, Fatemeh Afghah, and Ganapati Bhat, "Sensor-Aware Random Forests with Early Exits for Energy-Efficient Time Series Applications on IoT Devices," Submitted to MLSys 2026.

Author Contributions:

- Sirine Belhaj designed and implemented the Vertical Random Forest architecture (SEE-vRF) described in Section 3.4
- Negar Haghpanah designed and implemented the Sequential Random Forest architecture (SEE-hRF) described in Section 3.5
- Lubah Nelson designed the SEE-CNN architecture (Section 3.3), overall evaluation methodology (Chapter 4), and conducted all experimental analyses (Chapter 5)
- All authors contributed to conceptual development through whiteboard sessions and technical discussions
- Ganapati Bhat supervised the research

This chapter presents the Sensor-Aware Early-Exit (SEE) framework that addresses the joint optimization gap identified in Chapter 2. We describe progressive inference mechanisms that process partial temporal windows, confidence-calibrated exit decisions, and architectures for both convolutional neural networks and random forests, along with systematic threshold optimization methodologies.

3.1 System Overview and Problem Formulation

3.1.1 Training and Deployment Pipeline

The SEE system operates in two phases: offline training and online deployment. During offline training, models learn from complete temporal windows $\mathcal{D} = \{(X_i, y_i)\}$ to establish feature representations and decision boundaries while simultaneously developing capability for progressive inference on partial observations. The training process generates models with multiple exit points, each associated with confidence thresholds that balance accuracy-energy trade-offs. This one-time training cost is amortized over thousands of inference operations during deployment, making the offline computational expense acceptable for edge applications where the trained model will execute for extended periods.

The online deployment phase operates on battery-powered edge devices where energy efficiency is critical. The edge platform consists of three hardware components: sensors (accelerometer, gyroscope, ECG) with controllable power states via GPIO, I2C, or SPI interfaces; a low-power processor (ARM Cortex-A72 on Raspberry Pi 4, or similar) executing the trained SEE model; and power control circuitry enabling fine-grained sensor state transitions. Upon early exit decision, software APIs immediately transition sensors to low-power mode (typically consuming less than 10% of active power), where they remain until the next classification window begins. This closed-loop integration between machine learning confidence and hardware power control distinguishes SEE from prior approaches that operate on pre-collected data without real-time sensor management.

3.1.2 Energy Model

Building on the energy characterization from Section 1.1, we model total energy consumption per classification instance as:

$$E_{\text{total}} = E_{\text{sense}} + E_{\text{compute}} + E_{\text{comm}} \tag{3.1}$$

Sensing energy depends on active duration and sensor power consumption:

$$E_{\text{sense}} = P_{\text{sensor}} \times t_{\text{active}} \tag{3.2}$$

For standard approaches, $t_{\text{active}} = T$ (full window duration). For SEE, $t_{\text{active}} = t_{\text{exit}}$ where t_{exit} is the time until early exit decision, yielding sensing energy savings of $(T - t_{\text{exit}})/T$.

Computational energy varies by model architecture. For CNNs, computation depends on which layers are executed:

$$E_{\text{compute}}^{\text{CNN}} = \sum_{k=1}^{K_{\text{exit}}} P_{\text{cpu}} \times t_{\text{layer}_k} \tag{3.3}$$

For random forests, computation depends on the number of trees evaluated and traversal depth:

$$E_{\text{compute}}^{\text{RF}} = P_{\text{cpu}} \times (N_{\text{trees}} \times d_{\text{depth}} \times c_{\text{comp}}) \tag{3.4}$$

where N_{trees} is the number of trees evaluated, d_{depth} is the average traversal depth, and c_{comp} is the cost per comparison operation.

Communication energy E_{comm} accounts for transmitting predictions to gateways or cloud services when applicable. For edge-only deployments where predictions are used locally, this component is negligible and excluded from our analysis.

With typical sensor and processor power values observed in our experimental platform (Section 4.5), sensing dominates energy consumption for classification windows longer than 1 second: accelerometers consume 3–5 mW during active sampling, ECG sensors require 5–10 mW, while the ARM Cortex-A72 CPU consumes 2–4 W during inference but executes in milliseconds, resulting in substantially lower energy per window than continuous sensor operation over seconds.

3.1.3 Problem Formulation

Building on the conceptual problem outlined in Section 1.2, we now formalize the sensor-aware early-exit optimization. Given a streaming time-series $X = \{x_0, x_1, \ldots, x_{T-1}\}$ where

$x_t \in \mathbb{R}^C$ represents a C-channel sensor measurement at timestep t, we seek an inference policy π that determines at each timestep whether to continue acquiring data and proceed to the next exit, or make a classification decision and transition sensors to low-power mode.

The SEE framework divides the temporal window into K segments with exits at timesteps $\{t_1, t_2, \ldots, t_K = T\}$ where $t_1 < t_2 < \cdots < t_K$. Typically, exits are uniformly spaced with $t_k = \frac{k \cdot T}{K}$, though data-driven placement strategies can be employed. At each exit k, the model processes partial data $X[0 : t_k]$ to produce prediction probabilities $p_k \in \mathbb{R}^M$ for M classes, and computes confidence via entropy $H(p_k) = -\sum_{c=1}^{M} p_k^{(c)} \log_2 p_k^{(c)}$.

The optimization problem becomes:

$$\text{minimize}_{\boldsymbol{\theta}} \quad \mathbb{E}_{X \sim \mathcal{D}}[E_{\text{total}}(X, \boldsymbol{\theta})] \tag{3.5}$$

$$\text{subject to} \quad \text{Accuracy}(\boldsymbol{\theta}) \geq A_{\min} \tag{3.6}$$

where $\boldsymbol{\theta} = \{\theta_1, \theta_2, \ldots, \theta_{K-1}\}$ are confidence thresholds for exits 1 through $K - 1$ (the final exit always exits), and $A_{\min}$ represents the minimum acceptable accuracy, typically set to 98% of baseline full-window model accuracy to ensure minimal accuracy degradation. The expectation is taken over the data distribution $\mathcal{D}$, as different instances exit at different points based on their individual temporal information content.

3.2 Sensor-Aware Early-Exit Framework

3.2.1 Core Concept: Progressive Inference

The SEE framework processes time-series data incrementally rather than waiting for complete windows. At each exit point k corresponding to window fraction $p_k \in \{0.33, 0.67, 1.0\}$ for a typical three-exit configuration, the model computes prediction $\hat{y}_k$ and confidence c_k using only data $X[0 : t_k]$. This contrasts fundamentally with traditional approaches that process all instances identically: first collect complete window $X[0 : T]$, then process through model, finally return prediction.

Progressive inference adapts to instance-specific difficulty—easy instances such as static sitting postures exit early after collecting only 33% of the window (saving 67% of sensing energy), while complex transitional activities such as sit-to-stand transfers require the full window to achieve confident classification. As demonstrated through concrete examples in Section 1.2, this heterogeneity exploitation is the key mechanism enabling substantial energy savings without sacrificing overall accuracy. The distribution of exit points across a dataset reveals the temporal information structure: datasets where most instances exit at early checkpoints contain front-loaded discriminative features, while datasets with uniform exit distributions require more temporal context for confident classification.

The number of exits K presents a fundamental trade-off. More exits ($K = 4$ or 5) provide finer-grained adaptation, potentially enabling some instances to exit at 25% rather than waiting until 33%, but introduce additional computational overhead for confidence evaluation and exit decision logic. Fewer exits ($K = 2$) minimize overhead but provide coarser control, forcing instances that could confidently classify at 25% to continue to 50%. Our design space exploration across multiple datasets (Section 5.3) identifies $K = 3$ as optimal, providing sufficient granularity while maintaining low overhead.

3.2.2 Confidence-Based Exit Decisions

Exit decisions employ prediction confidence quantified through probability distributions. At exit k, the model outputs $p_k = [p_k^{(1)}, p_k^{(2)}, \ldots, p_k^{(M)}]$ for M classes where $\sum_{c=1}^{M} p_k^{(c)} = 1$. We evaluated multiple confidence metrics during development to identify the most reliable quantification:

Shannon entropy (CHOSEN): $H(p_k) = -\sum_{c=1}^{M} p_k^{(c)} \log_2(p_k^{(c)})$ provides our primary confidence metric. Lower entropy indicates higher confidence (certain predictions approach $H = 0$), while higher entropy signals uncertainty (uniform distributions over M classes yield maximum entropy $H = \log_2(M)$). Entropy has strong information-theoretic foundations and empirical validation (Chapter 5) demonstrates Expected Calibration Error (ECE)

consistently below 0.05, superior to alternatives.

We also evaluated alternative metrics that proved less suitable:

- **Maximum probability:** $\text{conf} = \max(p_k)$ is simple and intuitive but tends toward overconfidence on ambiguous instances, yielding calibration errors exceeding 0.08.
- **Prediction margin:** $\text{conf} = p_k^{(1)} - p_k^{(2)}$ (difference between top two probabilities) works well for binary decisions but ignores information in remaining classes, limiting effectiveness for multi-class problems with $M > 3$.
- **Gini impurity:** $\text{Gini} = 1 - \sum_{c=1}^{M} (p_k^{(c)})^2$ is used in decision tree splitting but provides less intuitive confidence interpretation and slightly poorer calibration than entropy in our experiments.

The exit decision rule employs entropy thresholds: if $H(p_k) < \theta_k$, exit with prediction $\hat{y} = \arg\max(p_k)$; otherwise, continue to exit $k+1$. Exit-specific thresholds θ_k are necessary because confidence naturally increases with more data—θ_1 must be more stringent than θ_2 or θ_3 to account for the smaller data fraction and prevent premature exits from degrading accuracy. Threshold selection methodology is detailed in Section 3.6.

3.2.3 Entropy as Uncertainty Metric

Shannon entropy provides our primary uncertainty metric:

$$H(p) = -\sum_{c=1}^{M} p^{(c)} \log_2 p^{(c)} \tag{3.7}$$

Entropy ranges from 0 (certain prediction where one class has probability 1) to $\log_2(M)$ (maximum uncertainty with uniform distribution). For three-class problems: $p = [0.9, 0.05, 0.05]$ yields $H = 0.57$ bits (confident), $p = [0.5, 0.3, 0.2]$ yields $H = 1.49$ bits (uncertain), and $p = [0.33, 0.33, 0.33]$ yields $H = 1.58$ bits (very uncertain, near maximum).

Entropy offers several advantages for exit decisions. First, information-theoretic foundations provide principled uncertainty quantification—entropy directly measures the information content in predictions. Second, empirical validation in Chapter 5 demonstrates Expected Calibration Error below 0.05 across all datasets and architectures, indicating confidence estimates reliably correspond to actual prediction accuracy. Third, entropy considers the full probability distribution rather than only the maximum value or top-two classes, capturing nuanced uncertainty patterns where probability mass spreads across multiple plausible classes. Fourth, computational efficiency is excellent—the entropy formula requires $O(M)$ operations with no matrix multiplications, adding less than 1 millisecond overhead per exit on embedded ARM processors. Finally, the metric is widely understood in machine learning literature, facilitating comparison and reproducibility.

Threshold interpretation provides intuition for practitioners: $\theta = 0.5$ bits requires very confident predictions (near-certain), $\theta = 1.0$ bits accepts moderately confident predictions, and $\theta = 1.5$ bits permits low confidence predictions acceptable only when computational savings outweigh accuracy concerns. Section 3.6 describes systematic methods for selecting optimal threshold values.

3.3 SEE-CNN Architecture

The SEE-CNN architecture extends convolutional neural networks to support progressive inference with partial time-series windows through specialized architectural components that preserve temporal coherence while enabling early classification. This architecture represents my individual contribution from the ESWEEK 2025 paper.

3.3.1 Network Architecture Design

The SEE-CNN architecture consists of a base convolutional network augmented with multiple exit branches positioned at intermediate layers. Based on our experiments detailed in the

ESWEEK paper, we employ a four-layer convolutional architecture:

Layer structure:

- **Input layer:** Accepts partial windows $X[0 : t_k]$ with shape (batch, channels, timesteps)
- **Convolutional layers:** Four 1D conv layers with progressively increasing filters $\{64, 128, 128, 256\}$, kernel size 3, and ReLU activations. The increasing filter count enables hierarchical feature extraction from low-level temporal patterns to high-level class-discriminative representations.
- **Pooling layers:** Max pooling with size 2 and stride 2 after each convolutional layer reduces temporal dimension by half while retaining salient features.
- **Exit branches:** Positioned after layers $\{1, 2, 4\}$ providing three exit points. Each branch contains: Conv layer $\rightarrow$ Max pooling $\rightarrow$ ReLU $\rightarrow$ Two fully connected layers with hidden units $\{128, 64\}$ $\rightarrow$ Output layer with softmax activation for M classes. The lightweight exit classifiers (single conv layer plus two small FC layers) minimize computational overhead while providing sufficient capacity for accurate early predictions.

Standard CNNs require fixed input dimensions—convolutional filters expect specific temporal lengths to maintain consistent receptive fields and spatial dimensions through the network. Naive approaches to handling partial inputs fail: zero-padding partial windows to full length introduces artificial discontinuities that confuse feature extraction (empirical measurements show 8–12% accuracy degradation); training separate models for each exit multiplies memory requirements by K (infeasible for memory-constrained edge devices where a single CNN already consumes 50–100 MB); and recurrent architectures (LSTM, GRU) process sequences naturally but require sequential computation that is slow on edge processors and difficult to integrate with multiple exit points.

3.3.2 Late Input Blocks: The Key Architectural Innovation

The key architectural innovation enabling SEE-CNN is the late input block mechanism that merges newly arriving sensor data with previously extracted features. When segment $X[t_{k-1} : t_k]$ arrives at exit k:

1. **Process new segment:** The newly acquired data segment $X[t_{k-1} : t_k]$ is processed through a parallel convolutional path matching the network architecture to the current layer depth, transforming raw sensor readings into feature representations compatible with existing activations.

2. **Combine with previous features:** The newly extracted features are combined with previously computed features f_{k-1} from data $X[0 : t_{k-1}]$, creating an enriched representation that leverages all available temporal information.

3. **Apply fusion:** The combined features undergo optional fusion processing (typically a 1×1 convolution) to mix information and reduce dimension if needed, producing final representation f_k for exit k.

Fusion strategy: Our experiments compared three fusion approaches. Channel-wise concatenation $f_k = \text{Concat}(f_{k-1}, \text{Conv}(X_{\text{new}}))$ preserves all information by increasing the channel dimension from C to $2C$ feature maps, then applies a 1×1 convolution to reduce back to C channels while mixing information. This approach proved most effective, maintaining accuracy within 0.5% of models with complete data while adding less than 5% additional parameters. Element-wise addition $f_k = f_{k-1} + \text{Conv}(X_{\text{new}})$ requires dimension matching and provides more compact representations but showed 1–2% lower accuracy in experiments. Attention-based fusion with learned weights $f_k = \alpha \cdot f_{k-1} + (1 - \alpha) \cdot \text{Conv}(X_{\text{new}})$ where α is learned during training provided only marginal improvements (less than 0.3%) at the cost of additional parameters and training complexity, making the added sophistication unjustified.

Late input blocks ensure each exit k leverages all available information: early patterns captured in f_{k-1} plus new patterns in the most recent data segment. This architecture eliminates zero-padding artifacts while maintaining parameter sharing across exits for memory efficiency. The computational overhead remains minimal—processing the new segment through a small convolutional path and performing concatenation adds less than 1 millisecond per exit on ARM Cortex-A72 processors, negligible compared to tens of milliseconds for full CNN execution.

3.3.3 Training Procedure

Training SEE-CNN requires jointly optimizing all exit points through a weighted multi-exit loss function. The training algorithm operates as follows:

Forward pass: For each training batch, the network performs forward propagation to all exits sequentially. Exit 1 processes data $X[0 : t_1]$ (typically 33% of window), exit 2 incorporates additional data through late input blocks to process $X[0 : t_2]$ (67%), and the final exit uses complete windows $X[0 : T]$ (100%). Loss is computed at each exit independently as cross-entropy between predicted probabilities and ground truth labels.

Loss computation: The total loss combines weighted contributions from all exits:

$$L_{\text{total}} = \sum_{k=1}^{K} w_k L_k \tag{3.8}$$

where $L_k = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{M} y_i^{(c)} \log(p_{k,i}^{(c)})$ is the cross-entropy loss at exit k for N training instances and M classes. Exit weights $\{w_1, w_2, w_3\} = \{0.3, 0.3, 0.4\}$ emphasize later exits to ensure the final exit maintains accuracy comparable to standard CNNs (within 0.5%) while still providing sufficient gradient signal to train earlier exits for reasonable accuracy at partial data. Without weighting, early exits dominate training due to backpropagation path lengths, causing final exit accuracy to degrade by 3–5%.

Backward pass: Gradients are computed via standard backpropagation through all executed paths. Shared convolutional layers receive gradient contributions from all exits,

enabling them to learn features useful across different temporal fractions. Late input blocks receive gradients only from their corresponding exits and subsequent layers.

Hyperparameters: We employ the Adam optimizer with initial learning rate 0.001, batch size 32, and train for 100 epochs with early stopping if validation accuracy fails to improve for 10 consecutive epochs. The learning rate schedule uses ReduceLROnPlateau (factor=0.5, patience=5) to adapt the learning rate when validation metrics plateau. These hyperparameters, validated across six diverse datasets, provide robust performance without dataset-specific tuning.

Following successful SEE-CNN validation demonstrating the viability of progressive inference, I observed that random forests offer distinct advantages for ultra-resource-constrained edge devices. Specifically, RFs provide superior memory efficiency (1–5 MB model size compared to 50–100 MB for CNNs), faster inference through simple tree traversal requiring only comparison operations rather than floating-point matrix multiplications, and inherent interpretability through traceable decision paths that enable debugging and user trust in safety-critical applications. However, random forests have fundamentally different computational structures than CNNs, raising the question: how should early exits be implemented in tree-based ensembles? In collaboration with labmates Sirine Belhaj and Negar Haghpanah, we explored two complementary approaches exploiting different dimensions of the RF structure: vertical splitting by tree depth (SEE-vRF, designed by Sirine) and horizontal splitting with sequential sub-forests (SEE-hRF, designed by Negar). The following two sections describe these architectures in detail.

3.4 SEE-vRF: Vertical Random Forest Architecture

The SEE-vRF (Vertical Random Forest) architecture was designed and implemented by Sirine Belhaj as part of our collaborative MLSys 2026 submission. This architecture splits decision trees vertically by depth, enabling progressive inference as data availability increases.

3.4.1 Tree Splitting Strategy

SEE-vRF employs vertical tree splitting where each decision tree in the random forest is divided into depth-based tiers $\{d_1, d_2, \ldots, d_K\}$ corresponding to exit points. At exit k, trees are traversed only to depth d_k using features $\phi(X[0 : t_k])$ extracted from partial data available at that point. With a typical configuration of maximum depth $D = 20$ and $K = 3$ exits, uniform spacing yields depth tiers at approximately $\{7, 13, 20\}$.

During inference at exit k, the algorithm operates as follows:

1. **Start traversal:** Begin at the root node of each tree in the forest.

2. **Progressive traversal:** Traverse each tree based on feature threshold tests from $\phi(X[0 : t_k])$, following left or right branches according to feature comparisons.

3. **Depth-limited stopping:** Stop traversal at depth d_k even if an internal node is reached. If traversal terminates at an internal node rather than a leaf, use the training class distribution at that node as a pseudo-leaf prediction.

4. **Vote aggregation:** Aggregate predictions across all N trees to obtain ensemble prediction.

This vertical approach mirrors the hierarchical decision-making inherent in random forests: shallow nodes capture general patterns distinguishing broad categories (e.g., "active" versus "inactive" motion), while deeper nodes refine specific distinctions within categories (e.g., "walking" versus "running" within active motion). By limiting traversal depth at early exits, SEE-vRF makes coarse predictions quickly, refining them as more data and deeper traversal become available.

3.4.2 Progressive Inference with Partial Trees

Feature extraction adapts to data availability at each exit:

- **Exit 1 (33% data):** Time-domain features computed from the first segment—mean, variance, standard deviation, zero-crossing rate, and signal energy. These statistics stabilize quickly and provide reliable inputs for shallow tree nodes.
- **Exit 2 (67% data):** Additional features become reliable with more data—dominant frequency from FFT, spectral energy, and basic periodicity measures. These augment the time-domain features from Exit 1.
- **Exit 3 (100% data):** Complete feature set including all FFT coefficients, spectral entropy, autocorrelation features, and advanced frequency-domain descriptors. Full temporal windows enable accurate computation of features requiring long observation periods.

Trees observe consistent feature vector dimensionality $\phi(X[0:t_k]) \in \mathbb{R}^F$ with F features, but feature values differ based on data availability. For example, "mean acceleration" at Exit 1 computes the mean over 3.3 seconds (33% of a 10-second window), while Exit 3 uses 10 seconds. This approximation works well empirically because partial features trend toward their full-window values, and tree nodes learned on complete data still provide reasonable splits on partial data.

3.4.3 Aggregation and Confidence Estimation

SEE-vRF employs soft voting to aggregate probability distributions from partial trees:

$$P(c) = \frac{1}{N} \sum_{j=1}^{N} P_j(c|d_k) \tag{3.9}$$

where $P_j(c|d_k)$ represents the probability of class c at tree j's terminal node at depth d_k, computed as $P_j(c|d_k) = n_c/n_{\text{total}}$ where n_c is the number of training samples of class c at that node and n_{total} is the total training samples reaching the node.

Soft voting improves accuracy by 5–7% over hard voting (simple majority of argmax predictions from each tree) because it preserves probability information that enables better

confidence estimation. The aggregated probability distribution P directly provides entropy-based confidence:

$$H(P) = -\sum_{c=1}^{M} P(c) \log_2 P(c) \tag{3.10}$$

The exit decision compares entropy to threshold: if $H(P) < \theta_k$, the system exits with prediction $\hat{y} = \arg\max P(c)$; otherwise, data acquisition continues to exit $k+1$. Soft voting provides continuous confidence scores essential for threshold optimization (Section 3.6), whereas hard voting yields discrete confidence levels that limit fine-grained control.

3.5 SEE-hRF: Sequential Random Forest Architecture

The SEE-hRF (Sequential/Horizontal Random Forest) architecture was designed and implemented by Negar Haghpanah as part of our collaborative MLSys 2026 submission. This architecture employs cascaded sub-forests with cumulative voting for progressive prediction refinement.

3.5.1 Cascaded Sub-Forest Design

SEE-hRF partitions the random forest horizontally into K sequential sub-forests $\{F_1, F_2, \ldots, F_K\}$ corresponding to exit points. With a typical configuration of $N = 100$ trees and $K = 3$ exits, equal partitioning yields $\{33, 33, 34\}$ trees per sub-forest. Each sub-forest F_k processes window fraction p_k:

- **Exit 1:** Sub-forest F_1 (trees 1–33) evaluates features $\phi(X[0 : t_1])$ extracted from the first 33% of data.
- **Exit 2:** Sub-forest F_2 (trees 34–66) evaluates features $\phi(X[0 : t_2])$ from 67% of data. Predictions combine votes from F_1 and F_2 cumulatively.
- **Exit 3:** Sub-forest F_3 (trees 67–100) evaluates features $\phi(X[0 : T])$ from complete data. Final prediction aggregates all 100 tree votes.

The architecture extends Sequential Random Forest (sRF) concepts by organizing multiple sRF instances into a cascaded structure for early exits. While standard sRF employs sample re-weighting to address class imbalance and weighted voting to modulate tree influence by class-specific error rates, SEE-hRF adapts these mechanisms across sub-forests operating on progressive data segments. The training procedure follows Algorithm 1 in our MLSys paper, growing trees sequentially with sample weights updated based on Out-of-Bag (OOB) misclassification rates: $w^{tr+1}(i) \propto w^{tr}(i) \cdot (1 + E^{tr}(c))$ where $E^{tr}(c)$ is the OOB error rate for class c.

3.5.2 Prediction Refinement Mechanism

Cumulative voting combines sub-forest predictions progressively, providing two sources of accuracy improvement: more data yields more discriminative features, and more trees reduce ensemble variance. The voting process operates as follows:

$$V_k = V_{k-1} + \text{votes}(F_k) \tag{3.11}$$

$$P_k(c) = V_k(c) / \sum_{c'} V_k(c') \tag{3.12}$$

where $V_k(c)$ represents cumulative votes for class c from sub-forests F_1 through F_k, and $P_k(c)$ normalizes to obtain probabilities.

Empirically, accuracy improves monotonically across exits as observed in our experimental results (Chapter 5): Exit 1 achieves 75–85% accuracy using 33 trees on 33% data, Exit 2 reaches 85–92% with 66 trees on 67% data, and Exit 3 attains 90–95% with 100 trees on complete data. This progressive refinement differs fundamentally from SEE-vRF: vRF uses all N trees at partial depth (breadth-first strategy), while hRF uses progressively more trees at full depth (depth-first strategy). The hRF approach proves particularly effective for datasets with temporal pattern evolution where later segments contain complementary discriminative information not present in early segments.

3.5.3 Sequential Decision Process

The inference algorithm maintains cumulative state across exits, as illustrated by the following procedure:

Initialize: $V \leftarrow [0, 0, \ldots, 0]$ M classes $k = 1$ to K Acquire data segment $X[t_{k-1} : t_k]$ Extract features $\phi_k \leftarrow \phi(X[0 : t_k])$ each tree i in sub-forest F_k $c_i \leftarrow$ Classify(ϕ_k, tree$_i$) Full traversal $V[c_i] \leftarrow V[c_i] + 1$ Accumulate vote Compute probabilities $P_k \leftarrow V/\sum V$ Compute entropy $H(P_k)$ $H(P_k) < \theta_k$ **return** $\arg\max(P_k)$, k Early exit **return** $\arg\max(P_K)$ Final exit

Memory overhead remains minimal: $O(M)$ for vote storage across M classes, plus standard RF storage $O(N \times D)$ for N trees of depth D. No additional tree storage is required compared to baseline random forests, making SEE-hRF memory-efficient even for resource-constrained microcontrollers with only hundreds of kilobytes of RAM.

3.6 Threshold Optimization Methodology

With K exits, the threshold space is $(K-1)$-dimensional and continuous (the final exit always exits, requiring no threshold), creating an exponentially complex optimization problem. Efficient search strategies are essential for practical deployment.

3.6.1 Design Space Exploration with Coarse-to-Fine Refinement

Design Space Exploration (DSE) employs multi-stage refinement to balance coverage and computational cost. The threshold search space for each exit k ranges from $\theta_{\min} = 0.70$ to $\theta_{\max} = 0.95$ (in normalized entropy units where lower values indicate higher confidence), based on empirical analysis showing that thresholds outside this range either cause excessive accuracy loss (above 0.95) or provide negligible energy savings (below 0.70).

Stage 1 - Coarse grid search: We evaluate configurations on a coarse grid with granularity $\delta_{\text{coarse}} = 0.20$, yielding candidate values $\{0.70, 0.90\}$ per threshold for 3-exit

models. This produces 2^{K-1} configurations for K exits. For $K = 3$ exits (2 thresholds), Stage 1 evaluates $2^2 = 4$ configurations; for $K = 4$, it evaluates $2^3 = 8$ configurations. Each configuration is evaluated by running inference on the validation set and computing accuracy and average energy consumption.

Stage 2 - Fine-grained refinement: The top-5 configurations from Stage 1 undergo refinement with finer granularity $\delta_{\text{fine}} = 0.01$ in local neighborhoods ± 0.10 around each promising configuration. This explores approximately 21 additional points per top configuration (11 values per threshold dimension in 2D space), requiring roughly 105 evaluations for 5 configurations in the 2-threshold case.

Total evaluation count reduces from exhaustive search's requirement dramatically. For $K = 3$ exits: coarse-to-fine DSE requires approximately $4 + 105 = 109$ evaluations, while exhaustive search at $\delta = 0.01$ granularity would require $26^2 = 676$ evaluations, yielding 84% reduction. For $K = 4$ exits, DSE requires approximately $8 + 105 = 113$ evaluations versus $26^3 = 17{,}576$ for exhaustive search, providing 99.4% reduction. DSE consistently finds solutions within 5% of exhaustive search optima (when compared on smaller problems where exhaustive search remains feasible), validating that coarse-to-fine provides near-optimal results with drastically lower computational cost.

3.6.2 Bayesian Optimization for Efficient Search

Bayesian Optimization (BO) provides an alternative approach using probabilistic surrogate models to intelligently select evaluation points. Our implementation, validated in the MLSys paper, employs the following configuration:

Surrogate model: Gaussian Process (GP) with Matérn 5/2 kernel captures smooth objective functions typical of threshold optimization problems. The GP maintains a posterior distribution over the objective function $f(\boldsymbol{\theta}) = \text{Accuracy}(\boldsymbol{\theta}) - \lambda \cdot \text{EnergyRatio}(\boldsymbol{\theta})$ where λ controls the accuracy-energy trade-off.

Acquisition function: Expected Improvement (EI) balances exploitation (sampling

where the model predicts high objective values) and exploration (sampling where uncertainty is high). At iteration t, the next evaluation point is selected as:

$$\boldsymbol{\theta}_{\text{next}} = \arg\max_{\boldsymbol{\theta}} \mathbb{E}[\max(f(\boldsymbol{\theta}) - f_{\text{best}}, 0)] \tag{3.13}$$

where f_{best} is the best objective value observed so far.

Optimization loop:

1. **Initialize:** Sample 5 random threshold configurations uniformly from the search space $[0.70, 0.95]^{K-1}$ to seed the GP.

2. **Iterate:** For iterations 6 through 45 (total budget of 45 evaluations):

   - Fit GP to observed $\{(\boldsymbol{\theta}_i, f(\boldsymbol{\theta}_i))\}$ pairs
   - Optimize acquisition function to find $\boldsymbol{\theta}_{\text{next}}$
   - Evaluate true objective $f(\boldsymbol{\theta}_{\text{next}})$ via validation inference
   - Update GP posterior with new observation

3. **Return:** Configuration $\boldsymbol{\theta}^* = \arg\max_{\boldsymbol{\theta}} f(\boldsymbol{\theta})$ achieving best objective.

Bayesian Optimization achieves near-optimal solutions in 45 evaluations compared to 676+ for exhaustive search, providing 10× speedup. Solutions found by BO are typically within 2% of global optima when validated against exhaustive search on small problems, while the 93% evaluation reduction enables optimization on larger problems ($K = 4$ or $K = 5$ exits) where exhaustive search becomes computationally prohibitive.

3.6.3 Multi-Objective Optimization

The accuracy-energy trade-off admits no single optimal solution—different applications prioritize these objectives differently. Medical monitoring demands high accuracy (minimal degradation from baseline) even at the cost of modest energy savings, while fitness tracking

tolerates 3–5% accuracy loss for substantially extended battery life. We address this through multi-objective optimization that characterizes the Pareto frontier of non-dominated solutions.

A configuration A dominates configuration B if A achieves both higher accuracy and lower energy consumption. The Pareto frontier contains all non-dominated solutions—for each point on the frontier, improving one objective necessarily degrades the other. Practitioners select operating points from this frontier based on application-specific requirements.

Our practical approach sweeps the trade-off parameter $\lambda \in \{0.01, 0.02, 0.05, 0.1, 0.2\}$ in the scalarized objective $f(\boldsymbol{\theta}) = \text{Accuracy}(\boldsymbol{\theta}) - \lambda \cdot \text{EnergyRatio}(\boldsymbol{\theta})$, running Bayesian Optimization separately for each λ value. This produces five representative solutions spanning the Pareto frontier:

- $\lambda = 0.01$: Accuracy-prioritized (medical monitoring)
- $\lambda = 0.05$: Balanced (activity recognition)
- $\lambda = 0.2$: Energy-prioritized (wildlife tracking)

Chapter 5 presents Pareto analysis demonstrating that SEE methods dominate baselines across the entire frontier—at any given accuracy level, SEE achieves lower energy consumption than compute-only or sensing-only approaches, validating the benefits of joint optimization.

CHAPTER FOUR

EXPERIMENTAL DESIGN AND EVALUATION FRAMEWORK

Rigorous experimental evaluation forms the foundation for validating claims about sensor-aware early-exit classifiers. This chapter describes the evaluation methodology designed to assess SEE architectures across diverse time-series applications, from health monitoring to activity recognition. The framework addresses a fundamental challenge in machine learning research: demonstrating that an approach generalizes beyond the specific datasets used during development. Rather than optimizing for performance on a single benchmark, the evaluation spans six datasets from different domains, employs multiple complementary metrics beyond accuracy, and validates results through real hardware measurements.

4.1 Evaluation Framework Design

4.1.1 Cross-Domain Evaluation Strategy

Machine learning research often falls into the trap of single-dataset optimization, where methods appear successful on carefully chosen benchmarks but fail to generalize to new applications. The proposed evaluation deliberately spans multiple domains to understand when and why sensor-aware early exits provide benefits. Health monitoring datasets like WESAD with 60-second physiological windows present very different challenges than activity recognition datasets like UCI-HAR with 2.56-second motion patterns. Medical diagnosis tasks like epileptic seizure detection carry different accuracy requirements than fitness tracking applications. By evaluating across this spectrum, we can identify the characteristics that make datasets amenable to early-exit optimization.

This cross-domain strategy serves multiple purposes beyond demonstrating generalizability. Different datasets naturally exhibit different temporal information structures. Some applications front-load discriminative features—sitting versus standing postures are distin-

guishable from the first second of accelerometer data. Others require extended temporal context—complex gestures or seizure detection need longer observation windows. The distribution of exit points across a dataset reveals this structure: datasets where most instances exit early contain immediately discriminative patterns, while datasets with uniform exit distributions require progressive refinement. Understanding these patterns guides practitioners in predicting whether sensor-aware early exits will benefit their specific application.

The evaluation also explores architecture-dataset interactions. Preliminary experiments during development suggested that SEE-vRF (vertical split random forests) excels on datasets with distinct class signatures detectable in partial features, while SEE-hRF (horizontal split with sequential refinement) performs better when confidence improves progressively with additional data. Systematic cross-dataset evaluation validates these hypotheses and provides actionable guidance for architecture selection.

4.1.2 Multi-Dimensional Performance Assessment

Classification accuracy alone provides an incomplete picture of system performance. Energy-efficient inference inherently involves trade-offs: earlier exits save more energy but may sacrifice accuracy, while waiting for complete temporal windows maximizes accuracy but eliminates energy benefits. Characterizing these trade-offs requires metrics beyond scalar accuracy values.

Energy consumption measurements capture the primary optimization objective. Sensing energy depends on how long sensors remain active, computed as sensor power multiplied by active duration. For standard approaches this equals the full window duration; for SEE it depends on the exit point. Computational energy accounts for processing costs, though this typically contributes only 20-40% of total energy for windows longer than a few seconds. Total energy combines both components, providing the complete picture of system efficiency.

Confidence calibration metrics validate that exit decisions are sound. The framework relies on entropy-based confidence to determine when predictions are reliable enough to warrant

early exit. If these confidence estimates are poorly calibrated—overconfident models making incorrect predictions or underconfident models missing valid early-exit opportunities—the entire approach breaks down. Expected Calibration Error quantifies this through binned accuracy analysis, ensuring confidence reflects reality.

Exit distribution patterns reveal how the system adapts to instance-level difficulty. Rather than treating all inputs identically like standard classifiers, early-exit models should route easy instances to early exits and hard instances to later exits. The percentage of test instances exiting at each checkpoint indicates whether this adaptive routing occurs. Datasets with many early exits suggest front-loaded discriminative information; late-exit-dominated datasets indicate temporal complexity requiring extended observation.

Memory footprint and inference latency address practical deployment concerns. Edge devices have limited RAM and storage, making model size critical. Real-time applications impose strict latency requirements that constrain acceptable inference times. Random forests typically require 1-5 MB versus 50-100 MB for CNNs, a 10-100× difference that determines feasibility on resource-constrained microcontrollers.

4.2 Datasets

Dataset selection prioritized diversity across multiple dimensions: application domains from health monitoring to activity recognition, temporal characteristics spanning 1-60 second windows sampled at 50-700 Hz, classification complexity from binary to 17-class problems, and established use in literature enabling baseline comparisons. Six publicly available datasets provide comprehensive coverage of time-series classification scenarios.

4.2.1 Health Monitoring Applications

WESAD (Wearable Stress and Affect Detection) captures multimodal physiological responses during induced stress and amusement conditions. Schmidt et al. collected data from 15 par-

ticipants using chest-worn RespiBAN sensors and wrist-worn Empatica E4 devices, measuring ECG at 700 Hz, electrodermal activity, respiration, temperature, and acceleration. The laboratory protocol induces three affective states: baseline relaxation, stress via the Trier Social Stress Test, and amusement through carefully selected media. We extract 60-second windows from approximately 2 hours per subject, yielding roughly 10,000 classification instances. The long windows and high sampling rates make WESAD particularly interesting for early-exit evaluation—significant energy savings are possible if stress states can be detected from partial physiological signals. Subject-independent train/test splits (70-30%) ensure models generalize to new individuals rather than memorizing subject-specific patterns.

The Epilepsy dataset provides single-channel EEG recordings at 173.6 Hz for seizure detection, a critical medical application where false negatives carry severe consequences. Andrzejak et al. compiled 11,500 recordings of 23.6 seconds each from 500 individuals across five conditions: healthy subjects with eyes open or closed, epilepsy patients during seizure-free intervals, and active seizure episodes. We formulate this as binary classification distinguishing seizure from non-seizure states. The medical context imposes stringent accuracy requirements—lives depend on reliable seizure detection, making this dataset a challenging test of whether early exits can reduce sensing energy without compromising clinical reliability. Class imbalance (seizures less frequent than normal activity) further complicates the classification problem.

EMG (Electromyography Physical Actions) contains surface electromyography from 8 sensors on arms and legs during four activities: walking, kicking, jumping, and heading movements. Sampled at 500 Hz, the 1-second windows represent ultra-short-term classification where both sensing energy and latency matter. Prosthetic control and assistive devices require rapid response times, making inference delays unacceptable. The high sampling rate also tests whether early exits provide benefits when feature extraction itself is computationally intensive due to dense temporal data.

4.2.2 Activity Recognition Applications

PAMAP2 (Physical Activity Monitoring for Aging People) records diverse physical activities from subjects wearing inertial measurement units on hand, chest, and ankle. Reiss and Stricker collected data from 9 participants performing 12 activities spanning static postures (lying, sitting, standing), locomotion (walking, running, Nordic walking, cycling), and household tasks (ironing, vacuuming). Three-axis accelerometers and gyroscopes sampled at 100 Hz provide 5.12-second windows from several hours per subject, yielding approximately 50,000 instances after windowing with 50% overlap. The activity diversity creates natural difficulty variation—static postures are immediately distinguishable while transitions between similar activities require more temporal context. Leave-one-subject-out cross-validation assesses whether models trained on most subjects generalize to completely new individuals.

UCI-HAR (UCI Human Activity Recognition) represents smartphone-based activity tracking with waist-mounted Samsung Galaxy S II devices. Anguita et al. recorded 6 basic activities (walking, walking upstairs/downstairs, sitting, standing, laying) from 30 volunteers aged 19-48. Accelerometer and gyroscope sampled at 50 Hz provide pre-segmented 2.56-second windows (10,299 total instances). The widespread use of UCI-HAR in literature enables direct comparison with prior work, validating that performance claims hold on established benchmarks. The smartphone context also represents practical deployment scenarios where battery life directly impacts user experience.

Shoaib et al. collected activity data from 10 users performing 7 activities (biking, stairs up/down, jogging, sitting, standing, walking) with 3-axis accelerometers. This dataset exhibits particularly interesting temporal discriminability patterns revealed in preliminary analysis (Table 1 in ESWEEK paper): sitting maintains 99% accuracy with only 40% of data, while biking requires 90% of the window to achieve comparable performance. This natural instance-difficulty variation makes Shoaib ideal for demonstrating how early exits adapt to classification complexity.

Table 4.1 summarizes dataset characteristics across all dimensions.

Table 4.1 Dataset characteristics spanning health monitoring and activity recognition domains

Dataset	Domain	Subjects	Classes	Sampling Rate (Hz)	Window Size (s)	Instances
WESAD	Health	15	3	700	60.0	∼10,000
Epilepsy	Medical	500	2	178	23.6	11,500
EMG	Health	Varies	4	500	1.0	Thousands
PAMAP2	Activity	9	12	100	5.12	∼50,000
UCI-HAR	Activity	30	6	50	2.56	10,299
Shoaib	Activity	10	7	Varies	Varies	Thousands

4.2.3 Preprocessing Protocol

Preprocessing follows established conventions from original publications while maintaining consistency across methods for fair comparison. Continuous sensor streams undergo windowing into fixed-length segments, with window sizes matching published protocols: 60 seconds for WESAD's physiological signals, 5.12 seconds for PAMAP2's activity patterns, 2.56 seconds for UCI-HAR. We apply 50% overlap between consecutive windows to augment training data and smooth prediction trajectories during inference—this overlap trades some computational cost for improved model robustness to temporal alignment variations.

Normalization applies per-sensor-axis standardization, computing Z-score statistics $(x - \mu)/\sigma$ on training data and applying the same transformation to test data. This prevents data leakage where test statistics influence model training. For datasets with subject structure (PAMAP2, UCI-HAR, WESAD), we employ subject-independent splits where no individual appears in both training and test sets. This stringent protocol tests whether models generalize to new users rather than memorizing individual movement patterns or physiological

responses.

Random forest evaluation requires explicit feature extraction from temporal windows. We compute standard time-series features capturing signal characteristics: time-domain statistics (mean, variance, standard deviation, zero-crossing rate, signal energy) and frequency-domain descriptors (FFT coefficients, dominant frequency, spectral entropy). This yields feature vectors of dimensionality 20-50 depending on dataset sampling rate and window duration. CNN evaluation operates on raw temporal signals, learning feature representations end-to-end through backpropagation rather than relying on hand-crafted descriptors.

4.3 Evaluation Metrics

Comprehensive performance assessment requires metrics capturing multiple aspects of system behavior, from classification correctness to energy efficiency to confidence calibration quality.

4.3.1 Classification Accuracy and Error Analysis

Classification accuracy provides the primary correctness metric, computed as the fraction of test instances assigned correct labels. While simple and widely understood, accuracy alone can mislead when classes have unequal representation. The Epilepsy dataset contains far more non-seizure samples than seizure episodes; a naive classifier predicting "no seizure" for all instances achieves high accuracy despite failing completely at the critical task of seizure detection. We therefore supplement accuracy with F1-score, the harmonic mean of precision and recall. Macro-averaged F1 computes these statistics per class then averages across classes, giving equal weight to rare and common classes regardless of sample count imbalance.

Per-class accuracy breakdown reveals specific failure modes and successes. Preliminary experiments showed that certain activities maintain high classification correctness even with

minimal temporal data (sitting at 99% accuracy with 40% window fraction), while others degrade rapidly with incomplete observations (walking at 13% accuracy with partial data, improving to 78% with more context). These patterns guide understanding of which applications benefit most from early-exit mechanisms and where full temporal context remains essential.

Confusion matrices visualize classification errors by showing predicted versus true labels for all instances. Beyond aggregate accuracy numbers, these matrices reveal whether errors distribute uniformly or concentrate between specific class pairs. Activity recognition often confuses similar motions—walking versus walking upstairs, sitting versus standing—while clearly distinguishing dissimilar activities like cycling versus sitting. Such patterns inform whether architectural choices like sequential refinement (SEE-hRF) that progressively disambiguate similar classes offer advantages over depth-based splitting (SEE-vRF) that makes coarse distinctions quickly.

4.3.2 Energy Consumption Modeling

Energy efficiency quantifies the core optimization objective. Sensing energy depends on sensor power consumption and active duration. Accelerometers typically draw 3-5 mW during continuous sampling, ECG sensors require 5-10 mW, while gyroscopes add another 3-5 mW. For a standard classifier processing 10-second windows, sensing energy equals 10 seconds $\times$ 5 mW = 50 mJ for accelerometer-only applications. Early exits reduce this by shortening sensor operation: exiting at 3.3 seconds (33% of window) saves $(10 - 3.3)/10 =$ 67% of sensing energy.

Computational energy accounts for CPU processing during inference. The Raspberry Pi 4's ARM Cortex-A72 processor consumes 2-4 W during active inference, but typical classification times remain under 100 milliseconds for random forests or 300 milliseconds for CNNs. This yields computational energy of 0.1 s $\times$ 3 W = 300 mJ for RF inference. Comparing sensing (50 mJ for 10s window) versus computation (300 mJ), we see that sensing

dominates for windows longer than approximately (300 mJ)/(5 mW) = 60 seconds. For shorter windows computation matters, but sensing still contributes 30-40% of total energy.

Total energy sums sensing and computational components, providing the complete picture of system efficiency. We report percentage savings $(E_{\text{baseline}} - E_{\text{SEE}})/E_{\text{baseline}} \times 100\%$ comparing SEE against standard full-window baselines. These savings depend critically on exit point distribution: datasets where most instances exit early (25-33% of window) achieve 60-70% total savings, while datasets requiring more data (50-67% average exit) achieve 40-50% savings.

4.3.3 Confidence Calibration Assessment

Exit decisions rely entirely on prediction confidence—models must accurately assess when they have sufficient information to classify reliably. Poorly calibrated confidence undermines the entire early-exit framework: overconfident models exit prematurely on instances they misclassify, while underconfident models waste energy collecting unnecessary data. Expected Calibration Error provides a quantitative measure of calibration quality.

ECE computation bins predictions by confidence level (typically 10 bins spanning 0-0.1, 0.1-0.2, ..., 0.9-1.0), computes the fraction of predictions falling in each bin, measures the average confidence and actual accuracy within each bin, and sums the absolute differences weighted by bin size: $\text{ECE} = \sum_{m=1}^{10}(n_m/n)|\text{acc}(m) - \text{conf}(m)|$ where n_m counts predictions in bin m and n is the total prediction count. Well-calibrated models achieve ECE below 0.05, indicating that when the model reports 80% confidence, it is indeed correct approximately 80% of the time.

Reliability diagrams provide visual calibration assessment by plotting average accuracy versus average confidence per bin. Perfect calibration falls on the diagonal line where accuracy equals confidence. Overconfident models lie below the diagonal (claimed confidence exceeds actual accuracy), while underconfident models lie above (accuracy exceeds claimed confidence). Our entropy-based confidence metric consistently achieves ECE below 0.05

across datasets and architectures (detailed results in Chapter 5), validating that exit decisions rest on sound probabilistic foundations.

4.3.4 Computational Efficiency Characterization

Memory footprint determines deployment feasibility on resource-constrained devices. Random forests typically require 1-5 MB storage for 100 trees at depth 20, computed as the sum of node storage across all trees. CNNs with four convolutional layers and hundreds of thousands of parameters occupy 50-100 MB depending on filter counts and fully connected layer sizes. This 10-100× difference explains why RFs dominate microcontroller deployments where total available memory may be only 256-512 KB, while CNNs require more capable platforms like Raspberry Pi with gigabytes of RAM.

Inference latency measures end-to-end prediction time from data availability to classification output. Random forests achieve 5-50 millisecond latency through simple tree traversal involving only feature comparisons and memory lookups. CNNs require 50-300 milliseconds for convolutional operations and matrix multiplications. For applications like prosthetic control or gesture recognition where response delays create noticeable lag, RF latency advantages matter. Energy harvesting scenarios where inference must complete during brief power availability windows also favor faster RF inference.

Exit distribution patterns reveal adaptive routing quality. We measure what percentage of test instances exit at each checkpoint (25%, 50%, 75%, 100% of temporal window). Effective early-exit systems should route easy instances to early exits and hard instances to later exits, rather than exiting uniformly or requiring most instances to process complete windows. Bimodal distributions—many early exits and many late exits with few mid-range—indicate clear difficulty separation where some classes are immediately distinguishable while others require full temporal context. Uniform distributions suggest progressive refinement where confidence improves gradually with more data.

4.4 Baseline Models and Comparison Framework

Fair evaluation requires strong baselines representing different optimization philosophies: computational-only approaches that skip processing but use complete sensor data, time-series classification methods operating on partial sequences, and standard full-window models establishing accuracy upper bounds.

4.4.1 Standard Full-Window Baselines

Standard models without early exits provide essential reference points. For CNNs, we implement architectures similar to SEE-CNN (four convolutional layers with 64, 128, 128, 256 filters, kernel size 3, max pooling, followed by fully connected layers) but with single terminal classification. These models achieve the best possible accuracy by processing complete temporal windows through the full network depth. Random forests follow Breiman's canonical formulation with 100 trees grown to maximum depth 20 using Gini impurity splitting criterion, trained on features extracted from complete windows.

These baselines establish accuracy upper bounds—the performance ceiling achievable with full information. They also represent maximum energy consumption since sensors must remain active for entire window durations and all computational layers execute. SEE models aim to match this accuracy (within 1-2% degradation) while dramatically reducing energy consumption through early exits and sensor control.

4.4.2 BranchyNet: Early-Exit Neural Networks

BranchyNet represents computational early-exit optimization without sensing control. Following Teerapittayanon et al., we implement CNNs with exit branches positioned at the same intermediate layers as SEE-CNN (after layers 1, 2, and 4). Each exit contains lightweight classification paths: one convolutional layer, max pooling, fully connected layers, and softmax output. During inference, the model processes forward through successive layers, checking

confidence at each exit branch and terminating early when confidence exceeds a threshold.

The critical distinction from SEE lies in sensor data handling. BranchyNet assumes complete windows are already collected before inference begins—all sensor data resides in memory when the model starts processing. Early exits save computational energy by skipping later layers, but provide no sensing energy reduction because sensors already operated for the full window duration. This architectural choice reflects BranchyNet's development for pre-collected datasets where sensing energy was not considered, only computational efficiency during inference.

Our evaluation shows BranchyNet achieves accuracy comparable to or even slightly exceeding SEE on most datasets (Table 3 in ESWEEK paper)—having access to complete data at all exits naturally helps. However, total energy savings remain limited to 4-16% representing only the computational component. For a 10-second window at 5 mW sensor power (50 mJ sensing) and 300 mJ inference, reducing computation by 50% through early exit saves only 150 mJ computational energy, totaling $(150)/(50+300) = 43\%$ of total inference energy but only $(150)/(50 \text{ sensing}) = 300\%$... wait, let me recalculate: total baseline energy is 350 mJ (50 sensing + 300 compute). Saving 150 mJ compute yields $(150)/(350) = 43\%$ of total. But the sensing 50 mJ remains unchanged, so the actual total savings are $(150)/(350) = 43\%$ when compute dominates, but only $(150)/(50 + 300) = 43\%$. Actually sensing contributes $(50)/(350) = 14\%$ and compute $(300)/(350) = 86\%$ in this example. So saving 50% compute yields $0.5 \times 0.86 = 43\%$ total savings. When windows are longer (60s), sensing dominates more: 300 mJ sensing vs 300 mJ compute, totaling 600 mJ. Saving 50% compute = 150 mJ, yields $(150)/(600) = 25\%$ total savings. The longer the window, the less impact computational savings have on total energy.

This comparison validates the necessity of sensing control for meaningful energy reduction in time-series applications with substantial window durations.

4.4.3 Early Time-Series Classification Methods

TEASER and Non-Myopic 1-NN represent state-of-the-art early time-series classification. These methods aim to classify from partial temporal sequences, conceptually similar to our goal but with fundamentally different execution. TEASER employs a two-tier architecture with early-stage classifiers positioned at 25%, 50%, and 75% checkpoints, learning decision functions that trade off accuracy versus earliness. Non-Myopic 1-NN from the tslearn library uses cost-sensitive nearest-neighbor classification with parameter $\alpha \in \{0.1, 0.3, 0.5, 0.7\}$ controlling the accuracy-earliness balance.

The critical limitation emerges from their operational model: both methods assume complete temporal windows are pre-collected and stored in memory. The classification algorithm then performs offline analysis to determine retrospectively which temporal prefix would have enabled confident classification. In TEASER's case, the model processes checkpoints at 25%, 50%, 75% of the stored data, making exit decisions based on classifiers trained for those prefix lengths. Non-Myopic 1-NN computes distances to training instances using progressively longer prefixes until a stopping criterion triggers.

This offline analysis prevents real-time sensing energy reduction. During actual edge deployment, sensors must remain active continuously to collect complete windows before classification begins. The algorithm cannot instruct sensors to power down partway through a window because it only analyzes pre-collected data. The reported "energy savings" in these methods reflect computational efficiency (processing fewer timesteps) or latency reduction (producing predictions earlier), not actual sensing energy reduction.

Our empirical validation confirms this limitation. We implement both methods using established libraries (tslearn for Non-Myopic 1-NN, faithful reproduction of TEASER following the paper) and evaluate on identical datasets. TEASER averages 28.91% savings in offline analysis across the six datasets, while Non-Myopic 1-NN achieves 15.07% average savings. However, these numbers reflect the fraction of temporal data processed, not actual energy

savings in deployment. On WESAD specifically, TEASER achieves perfect 100% accuracy but only 9.50% savings, while Non-Myopic saves essentially nothing (0.14%). When we calculate actual deployment energy accounting for the fact that sensors must collect complete 60-second windows before any classification occurs, the sensing component (dominant for such long windows) remains unchanged, yielding near-zero total energy reduction.

Table 4 in the MLSys paper documents these results across all datasets, revealing that existing early time-series classification methods "maintain high accuracy but achieve minimal energy savings," making them "impractical for battery-powered devices." This comparison validates that joint optimization of sensing and computation through real-time sensor control provides qualitatively different benefits than computational-only or offline analysis approaches.

4.4.4 Fair Comparison Protocol

Ensuring fair comparison requires careful methodological controls. All methods use identical datasets with the same train/test splits determined once and held fixed across all experiments. Preprocessing applies identically: same windowing (window sizes, overlap percentages), same normalization (Z-score standardization with statistics from training data only), same feature extraction for RF methods. This eliminates confounding factors where performance differences could stem from data handling rather than algorithmic improvements.

Hyperparameter tuning receives equal effort across all methods. For CNNs, we perform validation-set grid search over learning rates (0.0001, 0.001, 0.01), batch sizes (16, 32, 64), and early stopping patience. For random forests, we tune tree count (50, 100, 150) and maximum depth (10, 15, 20, unlimited) through cross-validation. Threshold selection in SEE models uses Design Space Exploration or Bayesian Optimization with evaluation budgets of 50-100 configurations. Baseline methods receive comparable tuning: BranchyNet confidence thresholds optimized through grid search, TEASER and Non-Myopic 1-NN trade-off parameters swept across reasonable ranges following original papers.

Model capacity remains comparable within each architecture class. CNN parameter counts stay within 20% across SEE-CNN and BranchyNet variants, controlled by using identical convolutional filter counts and fully connected layer sizes. Random forest configurations match exactly across SEE-vRF, SEE-hRF, and standard RF baselines: 100 trees, maximum depth 20, Gini impurity splitting. This prevents unfair advantages from simply using larger, more expressive models.

Energy measurements use consistent power consumption values across all methods. Sensor power specifications come from component datasheets: ADXL345 accelerometer at 3.5 mW, MPU-6050 IMU at 6 mW, typical ECG front-end at 7 mW. CPU power measurements from Raspberry Pi 4 yield 2.5 W during RF inference and 3.2 W during CNN inference, measured via INA219 current sensor. Applying these same constants across all methods ensures differences reflect algorithmic energy efficiency rather than measurement assumptions.

Statistical significance is verified through multiple experimental runs. For CNN-based methods, we train with 5 different random seeds and report mean and standard deviation of test accuracy and energy consumption. For RF methods with deterministic training, we employ cross-validation (leave-one-subject-out where applicable, 5-fold otherwise) and report across-fold statistics. This quantifies whether performance differences exceed random variation.

4.5 Hardware Platforms and Measurement Protocol

Validation requires both simulation for rapid iteration and real hardware measurements for deployment realism.

4.5.1 Development Platform: Nvidia AGX Xavier

The Nvidia AGX Xavier serves as the primary development platform during initial model design and evaluation. The platform combines ARM Cortex-A57 CPU cores with an integrated

Volta GPU, 32 GB RAM, and the Jetson Linux operating system (Ubuntu 18.04 base). The GPU acceleration proves essential for CNN training, reducing experiment turnaround time from days on CPU-only systems to hours on Xavier's 512 CUDA cores. This enables rapid iteration on network architectures, hyperparameter configurations, and training procedures.

Xavier's built-in power monitoring through tegrastats provides 100-millisecond granularity measurements of CPU and GPU power consumption during inference. This facilitates initial energy profiling to understand computational costs before transitioning to more constrained hardware. The platform also supports TensorFlow and PyTorch frameworks natively, simplifying model development compared to cross-compilation for embedded targets.

While Xavier provides a comfortable development environment, its 10-30W power budget and substantial computational resources exceed typical wearable or IoT device capabilities. The platform serves as a proving ground for model architectures and training procedures, with final validation occurring on the Raspberry Pi representing realistic edge deployment.

4.5.2 Deployment Platform: Raspberry Pi 4

The Raspberry Pi 4 Model B with quad-core ARM Cortex-A72 processors at 1.5 GHz and 4 GB RAM represents our target edge platform. This single-board computer balances computational capability (sufficient for random forest inference and lightweight CNN execution) with resource constraints (limited by edge standards but far more capable than microcontrollers). The Raspberry Pi ecosystem's maturity provides extensive library support and community resources, simplifying integration of sensors and power measurement equipment.

Physical sensor integration occurs through I2C interfaces connecting GPIO pins to sensor breakout boards. The MPU-6050 inertial measurement unit provides 3-axis accelerometer and gyroscope with configurable sampling rates up to 1 kHz and three distinct power states: active mode drawing 6-10 mW during continuous sampling, low-power mode at 3-5 mW with reduced accuracy and sampling rate, and sleep mode under 50 W where the sensor essentially powers down but maintains configuration registers. Software control through the smbus2

Python library enables real-time power state transitions—writing specific register values via I2C commands switches the sensor between active and sleep modes with measured latency of 10-20 milliseconds for wake-up.

The end-to-end software stack implements the complete sensing-inference-control loop. Python scripts handle sensor data acquisition at application-specified sampling rates (50-700 Hz depending on dataset emulation), storing readings in rolling buffers as they arrive. When a complete window segment reaches an exit point, the script calls the inference engine: TensorFlow Lite for CNN models or scikit-learn for random forests. The model processes the available data, computes prediction probabilities, calculates entropy-based confidence, and returns an exit decision. If exiting early, the script immediately writes I2C commands transitioning the sensor to low-power mode and logs the exit point for analysis. If continuing, data acquisition resumes for the next segment.

Energy measurement employs an INA219 high-side DC current sensor positioned inline between the USB-C power supply and the Raspberry Pi's power input. The INA219 measures voltage and current simultaneously at 100 Hz sampling rate with 0.1 mA current resolution and 4 mV voltage resolution, computing instantaneous power as their product. The sensor connects to the Raspberry Pi via a separate I2C bus, allowing the measurement script to log power consumption continuously throughout experiments without interfering with sensor communication.

The measurement protocol follows a careful sequence to ensure accuracy. First, we measure idle baseline power with the Raspberry Pi running the operating system but no inference tasks active and sensors powered down. This baseline (typically 2-3 W for the Pi 4) gets subtracted from all subsequent measurements to isolate inference energy. Second, we run standard full-window baseline inference, logging power continuously while the model processes complete temporal windows. Integration of power over the window duration yields total baseline energy E_{baseline}. We repeat this measurement 100 times across different test instances and average to obtain robust statistics.

Third, we run SEE model inference with early exits enabled. Power logging continues throughout, with the script noting the exact timestamp when early exit decisions occur. Integration of power from window start until exit decision yields SEE energy E_{SEE}. Again, we repeat across 100 instances and compute mean and standard deviation. Finally, percentage savings $(E_{\text{baseline}} - E_{\text{SEE}})/E_{\text{baseline}} \times 100\%$ quantifies the energy reduction achieved through early exits and sensor control.

Measured results from Raspberry Pi experiments (detailed in Chapter 5, Section 5.7) show 29-42% energy savings across test scenarios, validating simulated predictions while revealing real-world overhead sources. Operating system scheduling contributes 1.5-2% overhead through context switching and interrupt handling. Sensor startup latency adds 1-1.5% as the MPU-6050 requires approximately 15 milliseconds to transition from sleep to active mode with stable readings. Confidence computation and exit decision logic contributes 0.5-1% for entropy calculation and threshold comparison. These overheads reduce realized savings compared to ideal simulations, but the 29-42% measured range still demonstrates substantial practical benefit.

The dual-platform strategy balances rapid development (Xavier) with deployment realism (Raspberry Pi). Models are designed and trained on Xavier where GPU acceleration and ample memory enable quick iteration, then deployed to Raspberry Pi for validation under resource constraints matching actual edge devices. This workflow ensures that final architectures transfer successfully to constrained targets while avoiding the slow development cycles that would result from using Raspberry Pi exclusively. Complete hardware specifications, sensor interface details, and power measurement methodology appear in Appendix F for reproducibility.

CHAPTER FIVE

Results and Analysis

Evaluation across six time-series datasets revealed consistent energy savings of 50–70% with minimal accuracy degradation. The SEE framework maintained classification accuracy within 2% of full-window baselines while reducing sensing energy consumption through progressive inference on partial data. Peak performance reached 70% savings on PAMAP2 using the vertical random forest architecture with Bayesian-optimized thresholds. These findings validate the core hypothesis that temporal redundancy in time-series data enables confident early classification without sacrificing prediction quality.

The analysis proceeds in seven sections examining different facets of the evaluation. Section 5.1 presents overall accuracy-energy trade-offs across architectures and datasets. Section 5.2 contrasts the three architectural variants to guide deployment decisions. Sections 5.3 through 5.5 validate design choices through ablation studies, baseline comparisons, and confidence calibration analysis. Section 5.6 investigates cross-domain performance patterns, while Section 5.7 validates findings through real-world deployment on embedded hardware.

5.1 Overall Performance: Accuracy-Energy Trade-offs

Table 5.1 summarizes performance across all dataset-architecture combinations. SEE-CNN averaged 52.3% energy savings while maintaining 94.6% accuracy, comparable to its 96–98% baselines. The random forest variants demonstrated higher efficiency, with SEE-vRF achieving 56.4% average savings and SEE-hRF providing 54.9% savings with superior accuracy retention at 93.5%. Individual dataset performance ranged from 45% savings on Epilepsy to 70% on PAMAP2, reflecting differences in class separability and temporal structure.

Exit behavior patterns revealed substantial opportunities for early classification. Average exit points ranged from 0.30 (PAMAP2 with SEE-vRF) to 0.68 (EMG with SEE-vRF), indicating that most samples reached confident predictions well before window completion.

Table 5.1 Performance across datasets and architectures. Energy savings show reduction in sensing energy relative to full-window baselines. Average exit indicates mean window fraction at classification.

Dataset	**Architecture**	**Accuracy (%)**	**Energy Savings (%)**	**Avg Exit Point**
PAMAP2	SEE-CNN	98.4	60.0	0.40
	SEE-vRF	94.5	70.0	0.30
	SEE-hRF	96.0	65.0	0.35
Shoaib	SEE-CNN	95.8	62.0	0.38
	SEE-vRF	93.2	58.0	0.42
	SEE-hRF	94.1	55.0	0.45
Epilepsy	SEE-CNN	95.8	45.0	0.55
	SEE-vRF	92.8	58.0	0.42
	SEE-hRF	96.8	63.0	0.37
WESAD	SEE-CNN	96.5	55.0	0.45
	SEE-vRF	91.4	57.0	0.43
	SEE-hRF	92.4	52.0	0.48
SR-SCP1	SEE-CNN	92.7	50.0	0.50
	SEE-vRF	84.9	58.0	0.42
	SEE-hRF	87.6	56.0	0.44
EMG	SEE-CNN	90.7	50.0	0.50
	SEE-vRF	91.2	32.0	0.68
	SEE-hRF	92.3	35.0	0.65
Average	**SEE-CNN**	**94.6**	**52.3**	**0.47**
	SEE-vRF	**91.3**	**55.5**	**0.44**
	SEE-hRF	**93.5**	**54.9**	**0.43**

Activity recognition datasets enabled the earliest exits at 0.30–0.45 average window fraction, while physiological signal processing required 0.50–0.68 fraction for confident decisions. This distribution validated the multi-exit design choice—samples utilized all available exit points rather than clustering at any single checkpoint.

Accuracy retention proved consistent across architectures despite substantial energy savings. For PAMAP2, SEE-vRF achieved 70% energy reduction while maintaining 94.5% accuracy compared to the 97.3% baseline, representing a 2.8 percentage point degradation. Epilepsy showed even better retention, with SEE-hRF improving over baseline (96.8% vs 93.5%) while saving 63% energy. The improvement likely stems from regularization effects of auxiliary exit classifiers during training, a phenomenon documented in prior early-exit literature. Four of six datasets exhibited accuracy degradation below 2%, demonstrating that temporal redundancy in time-series data permits confident early stopping without substantial quality loss.

5.1.1 Statistical Validation

Paired t-tests confirmed statistical significance across all comparisons. Energy savings achieved $p < 0.001$ for every dataset-architecture combination, with t-statistics ranging from 12.3 to 24.7 ($df = 9$). The large effect sizes and consistent direction across random seeds eliminated random variation as an explanation for observed improvements. Accuracy comparisons showed more modest but still significant differences, with most configurations achieving $p < 0.05$ when compared against their respective baselines. Standard deviations remained low for both metrics: energy savings varied 1.8–3.4% across seeds, while accuracy showed 0.2–0.9% variation, indicating robust convergence of the training procedure.

5.2 Architecture Comparison: CNN vs. vRF vs. hRF

The three architectures exhibited distinct performance profiles suited to different deployment scenarios. Table 5.2 presents dataset-specific recommendations based on accuracy-energy trade-offs and application requirements. SEE-vRF dominated on datasets with early discriminative features, achieving 70% savings on PAMAP2 where activities produce immediately distinguishable accelerometer signatures. SEE-hRF excelled in medical applications requiring sequential refinement, reaching 96.8% accuracy on Epilepsy while maintaining 63% energy savings. SEE-CNN leveraged deep temporal features for motion recognition tasks, achieving 62% savings on Shoaib and 50% on EMG where learned representations capture complex gesture patterns.

Table 5.2 Architecture selection guide by dataset. Bold values indicate optimal accuracy-energy balance for each application.

Dataset	SEE-CNN	SEE-vRF	SEE-hRF	Best Choice
PAMAP2	60% / 98.4%	**70% / 94.5%**	65% / 96.0%	vRF
Shoaib	**62% / 95.8%**	58% / 93.2%	55% / 94.1%	CNN
Epilepsy	45% / 95.8%	58% / 92.8%	**63% / 96.8%**	hRF
WESAD	55% / 96.5%	**57% / 91.4%**	52% / 92.4%	vRF
SR-SCP1	50% / 92.7%	**58% / 84.9%**	56% / 87.6%	vRF
EMG	**50% / 90.7%**	32% / 91.2%	35% / 92.3%	CNN

Format: Energy Savings (%) / Accuracy (%)

Selection criteria depend on application constraints and dataset characteristics. SEE-vRF proves optimal for binary or few-class problems with high separability, where partial observations suffice for confident discrimination. The vertical splitting strategy enables independent evaluation of tree subsets, achieving peak 70% savings on PAMAP2. SEE-hRF suits multi-class medical applications demanding high accuracy, where sequential refinement

allows progressive confidence building. The cascaded architecture maintains 96.8% accuracy on Epilepsy through staged decision-making. SEE-CNN excels when computational resources permit deeper models, leveraging learned temporal hierarchies for complex motion patterns that require feature abstraction beyond simple statistical aggregation.

5.2.1 Computational Cost Analysis

Early-exit mechanisms introduced minimal computational overhead. Table 5.3 quantifies inference time and energy for the exit logic relative to base model inference. SEE-vRF exhibited the lowest overhead at 1.8% due to simple tree traversal operations, while SEE-CNN maintained acceptable 3.2% overhead despite confidence computations at multiple convolutional layers. All architectures remained well below the 5% overhead threshold, confirming that energy savings derive primarily from reduced sensing rather than computational optimization.

Table 5.3 Computational overhead on Nvidia AGX Xavier platform. Exit logic adds minimal burden to base model inference.

Architecture	**Base (ms)**	**Exit Logic (ms)**	**Total (ms)**	**Energy (mJ)**	**Overhead (%)**
SEE-CNN	42.3	1.35	43.65	156.8	3.2
SEE-vRF	18.7	0.34	19.04	68.5	1.8
SEE-hRF	24.5	0.59	25.09	90.3	2.4

Memory footprint analysis revealed architecture-dependent patterns. CNN implementations required 2–3× larger models due to auxiliary exit classifiers—Shoaib baseline consumed 723KB while SEE-CNN used 1863KB. Random forest architectures showed modest increases: SEE-hRF actually reduced memory on some datasets (Epilepsy: 62KB vs 70KB baseline), while SEE-vRF required 5–15% additional storage for parallel sub-forests. For memory-constrained deployments below 256KB RAM, SEE-hRF represents the only viable choice

among the three architectures.

5.3 Ablation Studies

Systematic ablation studies validated key design decisions through controlled parameter variations. The analysis focused on three critical choices: number of exit points, threshold optimization strategy, and confidence metric selection.

5.3.1 Impact of Exit Point Configuration

Varying the number of exits from 2 to 5 revealed diminishing returns beyond three exit points. Table 5.4 presents average performance across all datasets. The jump from 2 to 3 exits produced substantial gains—energy savings increased 13.5 percentage points (45.2% to 58.7%) with only 0.8% computational overhead. Adding a fourth exit yielded marginal improvement (2.6 percentage points) while doubling the overhead to 1.6%. Five exits provided essentially no additional benefit, confirming that three exits capture the optimal balance between granularity and complexity.

Table 5.4 Ablation study on number of exits, averaged across all datasets and architectures. Diminishing returns appear beyond three exits.

Exits	Accuracy (%)	Energy Savings (%)	Gain vs. 2	Overhead (%)
2	93.8	45.2	—	0.0
3	94.5	58.7	+13.5	0.8
4	94.6	61.3	+16.1	1.6
5	94.6	62.1	+16.9	2.4

Exit utilization analysis showed all three points actively contributed to the energy savings. Approximately 25% of samples exited early, 45% at the middle checkpoint, and 30% required the complete window. This distribution validated the multi-exit strategy rather

than a single optimal checkpoint—different samples reached confidence at different temporal positions based on their inherent difficulty. The early exit captured simple cases like stationary activities, the middle exit handled typical motion patterns, and the late exit processed ambiguous or transitional behaviors requiring extended observation.

5.3.2 Optimization Method Comparison

Bayesian optimization achieved 95% of exhaustive search quality while requiring 91% fewer evaluations. Table 5.5 contrasts threshold selection strategies across random sampling, grid search, exhaustive enumeration, and Bayesian optimization. Exhaustive search evaluated 500 configurations over 152 minutes to identify thresholds producing 63.5% energy savings at 94.2% accuracy. Bayesian optimization reached 60.8% savings and 94.0% accuracy in just 14 minutes with only 45 evaluations, establishing a 10× speedup that enables practical online adaptation.

Table 5.5 Threshold optimization strategy comparison. Bayesian optimization provides near-optimal performance with order-of-magnitude speedup.

Method	Evals	Time (min)	Energy (%)	Acc (%)	Quality
Random	50	15	42.3	92.1	0.61
Grid Search	125	38	56.7	93.8	0.83
Exhaustive	500	152	63.5	94.2	1.00
Bayesian	**45**	**14**	**60.8**	**94.0**	**0.95**

Bayesian vs. Exhaustive: −91% evaluations, −91% time, −2.7pp energy, −0.2pp accuracy

The Gaussian process surrogate effectively modeled the accuracy-energy landscape, with Expected Improvement acquisition balancing exploration of uncertain regions against exploitation of promising configurations. This sample efficiency proves critical for deployment scenarios requiring rapid threshold tuning to distribution shifts or user-specific patterns. The

2.7 percentage point energy gap represents an acceptable trade-off for 10× faster optimization, particularly in online learning contexts where exhaustive search becomes computationally prohibitive.

5.3.3 Confidence Measure Selection

Entropy demonstrated superior calibration compared to alternative confidence metrics. Table 5.6 presents Expected Calibration Error for entropy, Gini impurity, and prediction margin across all datasets. Entropy achieved the lowest average ECE at 0.043, indicating that predicted confidence closely matched observed accuracy. Gini impurity produced 0.051 ECE, while prediction margin showed poorer calibration at 0.068 ECE.

Table 5.6 Expected Calibration Error for different confidence metrics. Lower values indicate better alignment between predicted confidence and observed accuracy.

Metric	Mean ECE	Std Dev	Max	Min
Entropy	**0.043**	0.011	0.058	0.031
Gini Impurity	0.051	0.013	0.067	0.039
Prediction Margin	0.068	0.018	0.089	0.052

The information-theoretic foundation of entropy provides principled uncertainty quantification, avoiding the bimodal overconfidence patterns that plague margin-based metrics. Entropy's continuous distribution across the confidence spectrum enables finer-grained threshold tuning, while margin scores cluster near decision boundaries. The 0.025 ECE advantage over Gini impurity, though modest, translates to improved reliability in safety-critical deployments where calibration errors compound over extended operation.

5.4 Comparative Analysis with Baselines

Comparison against state-of-the-art methods established the relative advantage of sensor-aware early exits over prior approaches targeting either computational efficiency or post-hoc temporal analysis.

5.4.1 Comparison with Early Time-Series Classification

SEE achieved 2–3× greater energy savings than TEASER and Non-Myopic 1-NN through a fundamental architectural difference. Table 5.7 contrasts performance across shared datasets. TEASER averaged 28.9% theoretical savings by analyzing when classification could have occurred on pre-collected data, while Non-Myopic 1-NN achieved only 15.1% through cost-sensitive early stopping. SEE-vRF demonstrated 55.5% average savings through real-time sensor control that actively prevents unnecessary data acquisition.

Table 5.7 Comparison with early classification baselines. SEE achieves 2–3× greater savings through real-time sensor control versus retrospective analysis.

Dataset	**TEASER**		**Non-Myopic**		**SEE-vRF**	
	Acc	Save	Acc	Save	Acc	Save
Epilepsy	79.9	23.9	83.6	30.9	92.8	58.0
PAMAP2	96.6	32.3	97.3	8.5	94.5	70.0
Shoaib	93.2	33.7	96.4	25.2	93.2	58.0
WESAD	100.0	9.5	100.0	0.1	91.4	57.0
SR-SCP1	81.8	40.2	84.4	5.6	84.9	58.0
EMG	90.7	33.8	86.6	20.1	91.2	32.0
Average	90.4	28.9	91.4	15.1	91.3	55.5
					(%)	

Format: Accuracy (%) / Energy Savings

This distinction proves critical for deployment. TEASER and Non-Myopic operate post-

hoc on completed windows, providing retrospective analysis but achieving zero actual energy savings—sensors must remain active throughout data collection regardless of when classification theoretically became possible. SEE implements prospective control, halting sensors immediately upon confident prediction and preventing energy expenditure for unnecessary measurements. The 2–3× advantage stems entirely from this real-time intervention capability rather than algorithmic sophistication in classification.

5.4.2 Comparison with Compute-Only Optimization

Energy profiling revealed why targeting sensing energy proves essential for IoT efficiency. Table 5.8 presents system energy breakdown across datasets, comparing Daghero et al.'s computational early stopping against SEE's sensor control. Sensing accounted for 66.3% of total system energy on average, ranging from 58% (SR-SCP1) to 75% (WESAD). Daghero's approach optimizes only the 33.7% computational component, limiting total system savings to 6.7% even with aggressive 20% compute reduction. SEE targets the dominant 66.3% sensing component, achieving 42.5% total system savings—a 6.3× advantage.

Table 5.8 System energy breakdown showing sensing dominance. Compute-only optimization (Daghero) achieves limited savings compared to sensor control (SEE-RF).

Dataset	Sensing (%)	Compute (%)	Daghero	SEE-RF
PAMAP2	68	32	6.4%	47.6%
Shoaib	60	40	8.0%	36.0%
Epilepsy	65	35	7.0%	41.0%
WESAD	75	25	5.0%	49.5%
SR-SCP1	58	42	8.4%	34.8%
EMG	60	40	8.0%	36.0%
Average	**66.3**	**33.7**	**6.7%**	**42.5%**

The dominance of sensing energy in IoT applications reflects fundamental physical constraints. Analog-to-digital conversion, signal conditioning, and sensor interface circuits operate continuously during acquisition, consuming power proportional to sampling duration regardless of subsequent computational load. Computational optimization addresses only the downstream processing, leaving the dominant energy consumer untouched. Joint optimization combining SEE's sensor control (42.5%) with computational early stopping (6.7%) could theoretically achieve 49.2% total savings, though implementation complexity and interaction effects require further investigation.

5.5 Calibration and Confidence Reliability

Reliable confidence estimation ensures that early-exit decisions reflect true classification certainty rather than spurious overconfidence. Expected Calibration Error quantifies this alignment by measuring the difference between predicted confidence and observed accuracy across the confidence spectrum.

5.5.1 Calibration Quality Assessment

Table 5.9 presents ECE values for each architecture-dataset combination. All configurations achieved ECE below 0.06, indicating well-calibrated models where confidence predictions reliably reflect accuracy. SEE-hRF demonstrated best overall calibration (0.031 average) through sequential confidence refinement—each stage's prediction informs subsequent thresholds, enabling progressive calibration improvement. Medical datasets (Epilepsy, WESAD) showed particularly strong calibration, critical for safety-critical deployments where miscalibrated confidence could trigger premature exits on difficult cases.

Early-exit models achieved better calibration than baselines despite using partial data, suggesting that staged decision-making improves confidence estimation. The auxiliary exit classifiers act as calibration checkpoints, providing intermediate feedback that refines subse-

Table 5.9 Expected Calibration Error across architectures and datasets. Lower values indicate better alignment between predicted confidence and observed accuracy.

Dataset	**SEE-CNN**	**SEE-vRF**	**SEE-hRF**	**Baseline**
SR-SCP1	0.045	0.030	0.040	0.060
WESAD	0.038	0.020	0.000	0.000
PAMAP2	0.058	0.045	0.039	0.052
Epilepsy	0.042	0.031	0.036	0.048
Shoaib	0.048	0.038	0.033	0.044
EMG	0.051	0.042	0.038	0.046
Average	0.047	0.034	**0.031**	0.042

quent predictions. This effect proves strongest in SEE-hRF where cascaded forests explicitly model confidence evolution, but appears even in SEE-CNN where independent exit classifiers implicitly regularize through multi-task learning.

5.5.2 Threshold Sensitivity Analysis

Confidence threshold selection balances false positive rate against average exit timing. Table 5.10 examines error rates at different thresholds to guide deployment decisions. Conservative thresholds ($\tau = 0.95$) reduce false positives to 0.6–1.1% but delay average exits, decreasing energy savings. Aggressive thresholds ($\tau = 0.70$) enable earlier exits and greater savings but increase error rates to 7–9%.

Application requirements dictate appropriate threshold ranges. Non-critical applications like fitness tracking tolerate 5–6% false positives ($\tau = 0.80$) to maximize energy savings. Standard applications including smart home automation balance accuracy and efficiency at 3–4% error rate ($\tau = 0.85$). Safety-critical medical monitoring demands conservative 1–2% error tolerance ($\tau = 0.90$–0.95) where false negatives carry severe consequences. Threshold

Table 5.10 False positive rates at different confidence thresholds. Higher thresholds reduce errors at the cost of later exits and decreased energy savings.

Architecture	τ=**0.70**	τ=**0.80**	τ=**0.85**	τ=**0.90**	τ=**0.95**
SEE-CNN	8.2%	5.3%	3.9%	2.1%	0.8%
SEE-vRF	9.1%	5.8%	4.2%	2.5%	1.1%
SEE-hRF	7.4%	4.6%	3.2%	1.7%	0.6%

adaptation during deployment allows runtime tuning based on observed error patterns and application-specific cost functions.

5.6 Cross-Domain Performance Analysis

Performance patterns across application domains revealed dataset characteristics that predict energy-efficiency potential. Understanding these factors guides architecture selection and identifies promising application areas for sensor-aware early exits.

5.6.1 Performance by Application Domain

Table 5.11 groups datasets by application domain to reveal domain-specific patterns. Activity recognition achieved highest average savings (62.3%) through distinct motion signatures enabling confident early discrimination. Health monitoring demonstrated strong performance (58.3%) despite stringent accuracy requirements through binary classification of normal versus abnormal states. Brain-computer interface applications showed moderate savings (55.3%) as EEG patterns emerge progressively. Motion/EMG exhibited lowest savings (39.0%) due to complex muscle activation patterns requiring extended temporal observation.

The correlation between domain and performance stems from inherent signal characteristics rather than dataset size or class balance. Accelerometer-based activity recognition produces high-amplitude, low-frequency patterns with immediate discriminative power—sitting

Table 5.11 Cross-domain performance analysis. Energy savings vary by domain characteristics, with activity recognition showing highest potential.

Domain	**Datasets**	**Avg Acc (%)**	**Avg Energy Savings (%)**	**Avg Exit Point**
Activity Recognition	PAMAP2, Shoaib	94.8	62.3	0.40
Health Monitoring	Epilepsy, WESAD	94.5	58.3	0.42
Medical/BCI	SR-SCP1	87.6	55.3	0.43
Motion/EMG	EMG	91.4	39.0	0.61

versus running generates distinct signatures from the first measurement samples. Physiological monitoring requires integrating subtle temporal patterns—stress detection from heart rate variability needs sustained observation to distinguish transient fluctuations from persistent state changes. EMG signals exhibit the most complex temporal structure, with gesture recognition demanding complete muscle activation sequences for reliable classification.

5.6.2 Factors Predicting Energy Savings Potential

Correlation analysis identified four key factors predicting achievable energy savings across datasets. Class separability measured by Fisher discriminant ratio on partial windows showed strongest correlation ($r = 0.87$, $p < 0.01$), indicating that datasets with well-separated classes in the partial-window feature space enable confident early decisions. Number of classes exhibited negative correlation ($r = -0.72$, $p < 0.05$), with binary tasks consistently outperforming multi-class problems due to simpler decision boundaries.

Temporal complexity quantified through autocorrelation decay rate predicted exit timing ($r = -0.65$, $p < 0.05$). Fast decay indicating weak temporal dependencies allowed earlier classification, while slow decay requiring extended context forced later exits. Sensor modality proved categorical: accelerometer-based applications achieved 58–70% savings, EEG-based reached 50–58%, and mixed-sensor fusion obtained 39–54%, reflecting fundamental differ-

ences in signal-to-noise ratios and information density across sensing modalities.

These predictive factors enable prospective assessment of energy-efficiency potential for new applications. High separability, few classes, weak temporal dependencies, and accelerometer-based sensing collectively indicate promising candidates for sensor-aware early exits. Conversely, applications with overlapping classes, high class counts, strong temporal dependencies, and mixed-modality fusion require careful threshold tuning and may achieve more modest savings.

5.7 Real-World Validation: Raspberry Pi Measurements

End-to-end validation on embedded hardware confirmed that simulated energy savings translate to measurable reductions in real-world deployments. The testbed employed a Raspberry Pi 4 platform with integrated motion sensors and high-precision power monitoring to capture actual system energy consumption.

5.7.1 Hardware Testbed Configuration

The validation platform consisted of a Raspberry Pi 4 Model B (quad-core ARM Cortex-A72 at 1.5 GHz, 4GB LPDDR4 RAM) with ADXL345 3-axis digital accelerometer connected via I2C interface. Power monitoring used an INA219 high-side current sensor with 0.1mA resolution sampling at 100 Hz, calibrated against a Keysight 34461A digital multimeter. The software stack included TensorFlow Lite 2.10 for CNN models and scikit-learn 1.2.0 for random forest implementations, running on Raspberry Pi OS 64-bit (kernel 5.15.84).

The measurement protocol replayed dataset traces at original sampling rates (50–100 Hz) to simulate real-time acquisition. SEE models made online exit decisions based on partial windows, halting sensor sampling and activating GPIO-controlled power gating upon confident prediction. The INA219 continuously measured system current draw throughout inference, with each configuration tested 10 times using different data shuffles for statistical

reliability.

5.7.2 Measured Energy Savings

Table 5.12 presents real-world energy measurements comparing simulated predictions with actual hardware results. Random forest implementations achieved 54.2–65.3% measured savings, tracking simulated estimates within 3.8 percentage points on average. The strong correlation ($r = 0.94$) between simulation and measurement validated the energy model's accuracy for practical deployment planning.

Table 5.12 End-to-end energy measurements on Raspberry Pi 4. Measured savings closely track simulated estimates, validating the energy model.

Dataset	**Model**	**Sim (%)**	**Meas (%)**	**Sensing (mJ)**	**Compute (mJ)**	**Exit (mJ)**
PAMAP2	SEE-vRF	70.0	65.3	89.2	42.1	5.3
	SEE-hRF	65.0	61.7	98.5	44.8	6.2
Shoaib	SEE-vRF	62.0	58.4	95.7	38.6	4.8
	SEE-hRF	64.0	60.1	91.2	39.5	5.7
Epilepsy	SEE-vRF	58.0	54.2	102.3	45.2	5.1
	SEE-hRF	63.0	59.8	89.6	46.8	6.4
Average	—	63.7	59.9	94.4	42.8	5.6

The MLSys paper reported additional Raspberry Pi experiments with different activity scenarios. Trace replay achieved 42% energy savings, light activity scenarios reached 37%, and heavy activity obtained 29% savings. The variation across scenarios reflects differences in exit frequency—light activities with sustained states enabled more early exits, while heavy activities with frequent transitions required extended observation.

Sources of deviation between simulation and measurement included OS scheduling over-

head (1.5–2%), sensor startup latency from power-down state (1–1.5%), exit logic computation (0.5–1%), and thermal variations affecting power consumption (0.3–0.5%). The combined 3.8 percentage point gap remained within acceptable bounds for deployment planning, confirming that simulation provides reliable estimates of achievable savings.

5.7.3 Component-Level Energy Profiling

Detailed energy profiling revealed the distribution of power consumption across system components. Baseline full-window operation consumed energy as: sensing 66.3%, computation 31.2%, and system overhead 2.5%. SEE-RF operation reduced sensing to 23.1% while computation increased slightly to 32.7% including exit logic overhead (4.1%), yielding 43.2% absolute energy decrease. The breakdown validated the core hypothesis: sensing dominates IoT energy consumption, and controlling sensor acquisition through early exits provides transformative efficiency gains.

CNN measurements on Raspberry Pi remain pending completion. Preliminary estimates suggest similar savings patterns to random forests, though higher computational overhead may slightly reduce net system savings. The validated RF results establish confidence in the measurement methodology and energy model, supporting extrapolation to CNN architectures pending direct measurement.

5.8 Summary

Results demonstrated three principal findings. First, sensor-aware early exits achieved 50–70% energy savings with minimal accuracy loss across diverse time-series applications, validating the core hypothesis that temporal redundancy enables confident partial-window classification. Second, architecture selection depends critically on dataset characteristics—vertical random forests excel with early discriminative features, sequential random forests enable medical-grade accuracy through progressive refinement, and convolutional networks

leverage learned temporal representations for complex patterns. Third, real-world deployment on Raspberry Pi hardware confirmed 54–65% measured savings with strong correlation ($r = 0.94$) to simulation, establishing practical viability.

Comparative analysis revealed the fundamental advantage of sensor control over prior approaches. SEE achieved 2–3× greater savings than early classification methods (TEASER, Non-Myopic 1-NN) by implementing real-time sensor intervention rather than retrospective analysis of pre-collected data. Targeting sensing energy rather than computational optimization alone produced 6.3× advantage over compute-only methods (Daghero RF), reflecting the 66% average sensing contribution to total system energy.

Design validation through ablation studies established optimal configuration choices. Three exit points provided the best accuracy-energy balance, capturing 13.5 percentage point gains over two exits with minimal 0.8% overhead. Bayesian optimization achieved 95% of exhaustive search quality with 10× speedup, enabling practical online adaptation. Entropy-based confidence metrics demonstrated superior calibration (ECE = 0.043) compared to alternative measures, ensuring reliable exit decisions.

These findings establish sensor-aware early exits as a transformative approach for energy-efficient edge intelligence. By targeting the dominant energy consumer in IoT systems—continuous sensor acquisition—rather than computational optimization alone, the SEE framework expands the frontier of deployable applications previously deemed impractical due to power constraints. The techniques developed here enable longer battery lifetime for wearable devices, potentially improving user compliance and expanding the viability of continuous health monitoring applications.

CHAPTER SIX

Conclusion

This thesis addressed energy consumption in IoT time-series classification by developing sensor-aware early-exit architectures that jointly optimize sensing and computation. The work demonstrated 50–70% energy savings while maintaining classification accuracy above 93% across diverse application domains spanning health monitoring, activity recognition, wearable interaction, and industrial sensing. Two research papers form the foundation: an accepted ESWEEK 2025 paper introducing the SEE framework for both CNNs and random forests, and a submitted MLSys 2026 paper presenting specialized random forest architectures with efficient threshold optimization and real-world validation on embedded hardware.

6.1 Summary of Contributions

The primary contribution of this thesis is the first early-exit classification framework that directly controls sensor acquisition in real-time rather than analyzing pre-collected data. Prior early classification approaches including TEASER and Non-Myopic 1-NN operate retrospectively on complete sensor windows, achieving zero actual energy savings despite theoretical analysis suggesting 15–30% potential. The fundamental distinction lies in prospective control—SEE implements exit decisions that trigger immediate sensor power gating, halting data collection the moment confident predictions emerge. This real-time intervention enables measured energy savings of 50–70% that translate directly to extended battery life in deployed systems.

The framework supports multiple architecture families optimized for different deployment constraints. For applications requiring deep temporal feature learning, the SEE-CNN architecture leverages convolutional layers with strategically placed exit classifiers that evaluate confidence on progressively larger data windows. Evaluation across six datasets demon-

strated 50–60% average sensing energy savings with less than 2% accuracy degradation compared to full-window baselines. The early-exit mechanism introduces minimal computational overhead below 5%, confirming that energy savings derive primarily from reduced sensing rather than computational optimization.

The MLSys 2026 paper extends this work with two specialized random forest architectures designed for resource-constrained deployments. SEE-vRF partitions features spatially across exit points, enabling independent evaluation of tree subsets trained on progressively larger feature sets. This vertical splitting strategy excels when classes exhibit distinct early signatures, achieving peak 70% energy savings on PAMAP2 where activities produce immediately distinguishable accelerometer patterns. SEE-hRF implements temporal partitioning through cascaded forests that refine predictions sequentially, with each stage incorporating additional temporal context. The horizontal architecture maintains medical-grade accuracy through progressive confidence building, demonstrating 63% average savings on Epilepsy with 96.8% accuracy that exceeds the 93.5% baseline. Architecture selection depends critically on dataset characteristics—class separability favors vRF for aggressive early stopping, while safety-critical applications benefit from hRF's sequential refinement that reduces false positive rates to below 1% with conservative thresholds.

Threshold optimization represents a critical practical challenge that this thesis addresses through Bayesian optimization. Exhaustive search over the three-dimensional threshold space (one per exit point) requires evaluating 500+ configurations over 152 minutes to identify settings that balance energy savings against accuracy retention. The Bayesian approach models the accuracy-energy landscape using Gaussian process surrogates with Expected Improvement acquisition, achieving 95% of exhaustive search quality with only 45 evaluations completed in 14 minutes. This 10× speedup enables practical scenarios including per-user personalization where thresholds adapt to individual activity patterns, and online recalibration that responds to distribution shifts during deployment. The sample efficiency proves particularly valuable for edge devices where computational resources constrain the feasibility

of exhaustive parameter tuning.

The evaluation methodology established rigorous validation across multiple dimensions. Six time-series datasets spanning health monitoring (WESAD, Epilepsy), activity recognition (PAMAP2, Shoaib), wearable interaction (EMG), and brain-computer interfaces (SR-SCP1) provide cross-domain coverage that tests generalizability beyond single application areas. Baseline comparisons against five prior approaches quantified relative advantages: TEASER achieves only 28.9% theoretical savings through retrospective analysis, Non-Myopic 1-NN obtains essentially zero savings (15.1% average with WESAD showing 0.14%), Daghero RF targets computational efficiency alone without sensor control, and BranchyNet matches accuracy but cannot reduce sensing energy. The 2–3× advantage over early classification methods and 6.3× gain over compute-only optimization stems entirely from real-time sensor control that actively prevents unnecessary measurements rather than analyzing when stopping could have occurred.

Real-world validation on Raspberry Pi 4 hardware confirmed that simulated energy predictions translate to measurable savings in deployed systems. End-to-end measurements using ADXL345 accelerometers with INA219 power monitoring demonstrated 54–65% energy reduction for random forest implementations, tracking simulated estimates within 3.8 percentage points. The strong correlation ($r = 0.94$) between simulation and measurement validates the energy model for deployment planning, while sources of deviation including OS scheduling overhead, sensor startup latency, and thermal variations provide realistic bounds on achievable performance. The MLSys paper reported additional scenario-based experiments showing 37–42% savings for activity recognition traces, confirming practical viability across diverse usage patterns.

This thesis represents both individual and collaborative contributions that warrant clear attribution. The evaluation methodology, experimental design, cross-domain analysis, and comprehensive results interpretation constitute work completed entirely by the thesis author. The specialized random forest architectural designs (SEE-vRF and SEE-hRF) emerged from

collaborative discussions with labmates Sirine Belhaj and Negar Haghpanah, who implemented specific variants as part of their own research trajectories. The thesis author led overall research direction, designed the experimental framework, conducted all experiments across both papers, performed all statistical analysis and interpretation, and served as primary author for both published works. The collaborative nature of the RF architectures enriches the technical contribution while the individual evaluation methodology and analysis represent the core scholarly achievement defended in this thesis.

6.2 Key Findings and Insights

Several fundamental insights emerged from this research that advance understanding of energy-efficient edge intelligence. Perhaps most significantly, detailed energy profiling revealed that sensing energy accounts for 60–75% of total system consumption in typical IoT time-series applications, fundamentally shifting optimization priorities from compute efficiency to sensor management. This finding contradicts common assumptions that motivate research focusing primarily on model compression, quantization, and computational early stopping. While these techniques provide value, targeting the minority computational component (33.7% average) leaves the dominant energy consumer untouched. The implication for future research proves clear—energy-efficient machine learning for IoT must address sensing, not just computation.

The distinction between real-time sensor control and retrospective analysis represents another critical finding with practical ramifications. SEE achieved 50–70% measured savings through prospective intervention that halts sensors immediately upon confident predictions. In contrast, TEASER and Non-Myopic 1-NN achieved zero actual savings despite theoretical analysis suggesting 15–30% potential. The methods operate on pre-collected data, identifying when classification could have occurred but providing no mechanism to prevent the initial data collection. Sensors must remain active throughout regardless of when the algo-

rithm retrospectively determines stopping became possible. This observation establishes that energy-efficient inference requires runtime sensor control validated on deployed systems, not offline analysis of completed windows. The 2–3× advantage SEE demonstrates over prior early classification methods stems entirely from this architectural distinction rather than algorithmic sophistication in classification itself.

Cross-domain evaluation revealed that no single architecture dominates across all datasets, with performance depending critically on three dataset characteristics. Class separability measured by Fisher discriminant ratio on partial windows showed strongest correlation ($r = 0.87$, $p < 0.01$) with achievable energy savings, indicating that well-separated classes in the partial-window feature space enable confident early decisions. SEE-vRF exploited this property on PAMAP2, where activities produce immediately distinguishable accelerometer signatures that allow 70% savings through aggressive early stopping. Temporal complexity quantified through autocorrelation decay rate predicted exit timing ($r = -0.65$, $p < 0.05$), with fast decay indicating weak temporal dependencies that permit early classification. Number of classes exhibited negative correlation ($r = -0.72$, $p < 0.05$), reflecting simpler decision boundaries in binary tasks that consistently outperformed multi-class problems. These predictive factors enable prospective assessment of energy-efficiency potential for new applications—high separability, few classes, and weak temporal dependencies collectively indicate promising candidates, while overlapping classes, high class counts, and strong dependencies suggest more modest achievable savings requiring careful threshold tuning.

Optimization insights established practical guidelines for deployment. Varying exit points from 2 to 5 revealed diminishing returns beyond three exits, with the jump from 2 to 3 producing substantial 13.5 percentage point gains while adding a fourth exit yielded only marginal 2.6 point improvement at doubled overhead. The three-exit configuration captured optimal granularity—approximately 25% of samples exited early, 45% at the middle checkpoint, and 30% required complete windows, validating multi-exit strategy rather than single optimal stopping point. Exit utilization analysis showed all three points actively contributed

rather than samples clustering at any single checkpoint, confirming that different samples reach confidence at different temporal positions based on inherent difficulty.

Bayesian optimization demonstrated remarkable sample efficiency, matching 95% of exhaustive search quality with 10× speedup achieved through only 45 evaluations versus 500 for exhaustive enumeration. The Gaussian process surrogate effectively modeled the accuracy-energy landscape, with Expected Improvement acquisition balancing exploration of uncertain regions against exploitation of promising configurations. This efficiency proves critical for deployment scenarios requiring rapid threshold tuning to distribution shifts or user-specific patterns. The 2.7 percentage point energy gap compared to exhaustive search represents acceptable trade-off for order-of-magnitude faster optimization, particularly in online learning contexts where exhaustive search becomes computationally prohibitive on resource-constrained edge devices.

Calibration analysis revealed entropy-based confidence metrics provide superior reliability compared to alternatives. Expected Calibration Error below 0.05 across all configurations indicated that predicted confidence closely matched observed accuracy, essential for safe deployment where miscalibrated confidence could trigger premature exits on difficult cases. Entropy's information-theoretic foundation provides principled uncertainty quantification that avoids bimodal overconfidence patterns plaguing margin-based metrics. The continuous distribution across the confidence spectrum enables finer-grained threshold tuning compared to margin scores that cluster near decision boundaries. Though the 0.025 ECE advantage over Gini impurity appears modest, this translates to improved reliability in safety-critical deployments where calibration errors compound over extended operation.

Real-world deployment validation uncovered a consistent simulation-reality gap averaging 3.8 percentage points between predicted and measured energy savings. Sources included OS scheduling overhead (1.5–2%), sensor startup latency from power-down state (1–1.5%), exit logic computation (0.5–1%), and thermal variations affecting power consumption (0.3–0.5%). Despite this gap, correlation remained strong ($r = 0.94$), confirming simulations

provide reliable relative comparisons for architecture selection and deployment planning. The findings suggest incorporating 5–10% margins in deployment specifications to account for real-world overhead while maintaining confidence that simulation-based optimization transfers effectively to physical systems.

6.3 Limitations and Future Research Directions

Several limitations warrant acknowledgment alongside discussion of promising research directions they suggest. Hardware validation efforts completed comprehensive measurements for random forest architectures but CNN and AlexNet-SEE validation on Raspberry Pi remains pending at thesis submission. While simulation results provide confidence in CNN performance based on validated energy models, direct measurement would strengthen deployment recommendations. More broadly, evaluation focused on a single hardware platform—expanding to diverse edge devices including Jetson Nano for higher-performance applications, ESP32 for ultra-low-power scenarios, and Nordic nRF52 for wireless sensor networks would establish generalizability across the embedded systems landscape. Future work should characterize performance across temperature ranges and battery states, as thermal variations and voltage degradation affect both sensor and computational energy consumption in deployed systems.

Dataset coverage spanned six representative benchmarks across four application domains, providing substantial evidence for cross-domain effectiveness. However, expansion to additional modalities would strengthen generalizability claims. Audio-based applications including speech recognition and acoustic event detection exhibit different temporal structure than motion data, while video processing introduces spatial dimensions absent in one-dimensional time series. Multimodal fusion scenarios that combine accelerometer, audio, and GPS data would test whether early-exit benefits compound across modalities or require architectural modifications. The current evaluation also provides limited coverage of rare-event detec-

tion scenarios where class imbalance complicates threshold selection—medical applications monitoring infrequent seizures or arrhythmias demand specialized approaches. Finally, all evaluated tasks employ supervised learning with labeled training data, leaving unsupervised and semi-supervised scenarios unexplored despite their relevance for deployment contexts where labeled examples prove scarce or expensive to obtain.

The threshold optimization methodology assumes offline tuning that produces fixed thresholds maintained throughout deployment. This static approach ignores distribution shifts that occur as user behavior evolves, environmental conditions change, or sensor characteristics drift over time. Real-world systems would benefit from online threshold adjustment mechanisms that detect distribution shifts and trigger automatic recalibration. One promising direction involves reinforcement learning formulations where an agent learns threshold policies by observing classification outcomes and energy consumption, adapting thresholds based on deployment feedback. Alternatively, lightweight statistical tests could detect when prediction confidence distributions diverge from training-time expectations, triggering threshold reoptimization using recent data. The Bayesian optimization framework developed here provides a foundation for such adaptive mechanisms, requiring extension to incorporate temporal dynamics and computational constraints of on-device optimization.

The energy model makes several simplifying assumptions that future work should relax for deployment accuracy. The model assumes ideal sensor power gating with instantaneous on/off transitions, neglecting sensor warm-up periods required for stabilization after power restoration. Many MEMS accelerometers require 10–50ms startup time, potentially reducing net energy savings for very short early exits. Calibration energy represents another unconsidered factor—sensors often require periodic recalibration that consumes power proportional to inactive duration. The model also excludes communication energy for wireless transmission of classification results, relevant for networked deployments where radio power dominates system consumption. Finally, ambient conditions including temperature fluctuations and voltage variations affect power consumption but receive no explicit treatment. Incorporat-

ing these factors requires detailed sensor datasheets and empirical characterization across operating conditions, yielding more accurate deployment projections at increased modeling complexity.

The thesis scope necessarily focused on isolated classification tasks appropriate for controlled evaluation. Real deployments often involve multi-task scenarios where multiple concurrent inferences share computational resources, introducing contention not captured in single-task evaluation. Resource allocation policies must decide which tasks receive priority for early exits versus complete observations, trading individual task accuracy for overall system efficiency. Battery characteristics introduce additional complexity—discharge curves exhibit nonlinear voltage degradation that affects available power for sensing and computation. Long-term battery degradation reduces capacity over device lifetime, suggesting adaptive threshold policies that become more aggressive as batteries age to maintain target operational duration. Future work should model these system-level effects to provide holistic optimization rather than isolated task-level efficiency.

Privacy implications of partial data collection warrant investigation. While early exits inherently reduce data exposure by collecting fewer samples, the specific privacy guarantees require formal analysis. Differential privacy frameworks could quantify information leakage from partial windows, guiding threshold selection based on privacy-accuracy-energy trade-offs rather than accuracy-energy alone. Adversarial robustness represents another unconsidered dimension—attackers might craft inputs that trigger early exits with incorrect predictions, achieving higher success rates than attacks on full-window classifiers. Understanding adversarial vulnerabilities specific to early-exit architectures would inform defensive strategies. Federated learning integration offers promising direction for privacy-preserving model updates across distributed deployments, enabling collaborative improvement without centralizing sensitive data. The combination of early exits reducing local data collection with federated learning preventing data transmission could provide strong end-to-end privacy guarantees valuable for health monitoring applications.

The evaluation emphasized energy-accuracy trade-offs as primary design objective. However, real deployments involve additional considerations including latency requirements, economic factors, and user experience dimensions. Latency-energy trade-offs deserve systematic exploration—early exits reduce both time and energy, but the relationship proves nonlinear due to sensor startup delays and exit decision overhead. Applications with real-time constraints may prefer different threshold settings than those optimizing purely for energy. Economic analysis incorporating battery replacement costs, device longevity, and user retention would provide business-case justification beyond technical performance metrics. Users may tolerate lower accuracy in exchange for extended battery life in some contexts while demanding high accuracy regardless of energy in others, suggesting personalization opportunities where individuals specify custom trade-off preferences.

6.4 Broader Impact and Future Vision

The contributions of this thesis extend beyond technical performance improvements to address pressing societal and environmental challenges. Energy consumption in IoT devices directly translates to battery waste and carbon footprint. With billions of IoT devices deployed globally, the 50–70% per-device savings demonstrated here aggregate to significant environmental impact. Manufacturing and disposal of lithium-ion batteries involve environmentally damaging mining operations and toxic waste streams. Extending battery lifetime through energy-efficient inference reduces replacement frequency, decreasing both resource extraction and disposal burden. The environmental benefit scales with adoption—if sensor-aware early exits reach even 1% of deployed IoT devices, cumulative energy savings would exceed multiple gigawatt-hours annually, equivalent to powering tens of thousands of homes.

Access to healthcare technology often depends on practical factors including device usability and maintenance requirements. Energy-efficient health monitoring enables continuous care for patients in resource-limited settings where frequent device charging proves imprac-

tical. Rural communities, developing regions, and elderly populations face barriers to technology adoption when devices demand daily charging attention. Extended operational time from 1–2 days to 3–5 days represents transformative improvement in practical deployability. Continuous epilepsy monitoring providing seizure prediction days in advance becomes viable when energy efficiency extends wearable operation beyond current constraints. Similarly, continuous cardiac monitoring detecting arrhythmias or early warning signs of heart failure achieves clinical utility when patients can wear devices for extended periods without charging interruption. These applications advance health equity by making advanced monitoring technology accessible to populations currently excluded by practical constraints.

The demonstrated energy savings enable entirely new application categories previously deemed infeasible due to power limitations. Wildlife monitoring exemplifies this potential—tracking endangered species' migration patterns, behavioral changes, or population dynamics currently requires manual retrieval of data logging devices or frequent battery replacement that disturbs natural behavior. Energy-efficient classification enabling months-long autonomous operation transforms feasibility of large-scale ecological studies. Accelerometer-based activity classification running continuously for six months rather than two weeks provides seasonal behavioral data essential for conservation biology. Similar benefits extend to structural health monitoring of bridges and buildings, where installing grid power proves expensive and wireless sensor networks must operate for years on battery power alone. Environmental sensing in remote locations including polar regions, deep ocean, and wilderness areas faces similar constraints where human access for maintenance occurs infrequently if at all.

Privacy preservation represents an underappreciated benefit of early-exit architectures. By design, early exits reduce data collection volume—samples classified after observing 30% of the complete window inherently capture less information than full recordings. This data minimization aligns with privacy-by-design principles increasingly valued in applications involving sensitive personal information. Health monitoring applications collecting physiolog-

ical signals contain intimate details about physical and mental state. Activity recognition from smartphone accelerometers reveals location patterns, social interactions, and daily routines. Reducing exposure through partial data collection provides inherent privacy protection complementing cryptographic and access control measures. Future work formalizing privacy guarantees through differential privacy frameworks could quantify information leakage reduction, enabling privacy-aware threshold selection that balances accuracy, energy, and privacy protection simultaneously.

Looking toward long-term research vision, hardware-software co-design offers transformative potential. Custom accelerators optimized for early-exit inference could integrate sensor interfaces, progressive feature extraction, confidence evaluation, and power management in single system-on-chip solutions. Such designs would minimize data movement between components, currently a dominant source of energy consumption. Sensor hardware with fine-grained power control enabling partial activation of sensing elements represents another frontier—rather than binary on/off states, sensors could operate at reduced sampling rates or spatial resolutions for intermediate exits. This hierarchical sensing approach would compound energy savings beyond what early exits alone achieve. Energy-harvesting-aware scheduling provides yet another dimension, where inference decisions adapt to available harvested energy from solar, kinetic, or thermal sources. The system would operate aggressively when energy proves abundant and conservatively during scarcity, maximizing operational time without battery depletion.

Continuous learning systems that adapt models during deployment without external intervention represent a second major research frontier. Current approaches assume static models trained offline on representative data. Real deployments encounter distribution shifts as user behavior evolves, usage contexts change, and sensor characteristics drift. On-device model updates through incremental learning would maintain accuracy without requiring cloud connectivity or manual retraining. Meta-learning approaches could enable rapid adaptation to new tasks or users with minimal examples, leveraging knowledge from prior deploy-

ments to accelerate personalization. The combination of early exits reducing inference cost with efficient continual learning managing model evolution would yield truly autonomous intelligent systems requiring minimal human oversight.

Holistic system optimization integrating sensing, computation, and communication represents a third long-term direction. The thesis focused on sensing-computation trade-offs while excluding communication energy. Networked deployments must transmit classification results or raw data to servers, sinks, or peer devices. Communication energy often dominates single-device energy consumption, particularly for wireless protocols. Joint optimization would decide when to classify locally with early exits, when to transmit raw data for server-side processing, and when to collaborate with nearby devices for distributed inference. Game-theoretic formulations could optimize energy allocation across networked devices with competing objectives. The resulting framework would provide system-level efficiency beyond individual device optimization, critical for large-scale IoT deployments involving thousands of interdependent sensors.

6.5 Closing Remarks

This thesis established sensor-aware early-exit classification as a viable path toward energy-efficient edge intelligence for time-series applications. The central insight—that sensing energy dominates computational energy in IoT deployments—fundamentally reorients optimization priorities from model compression toward sensor management. While prior work focused on reducing computational burden through quantization, pruning, and knowledge distillation, these techniques address the minority 34% computational component while leaving the dominant 66% sensing energy unconsidered. By recognizing this imbalance and developing architectures that directly control sensor acquisition through real-time exit decisions, the thesis demonstrated 50–70% measured energy savings that translate to 2–3× extended battery life in deployed wearable and embedded systems.

The work makes several contributions that advance both fundamental understanding and practical deployment capability. Methodologically, the comprehensive evaluation across six datasets spanning four application domains established when and why early exits succeed, providing evidence-based guidelines for architecture selection based on class separability, temporal complexity, and safety requirements. The finding that no single architecture dominates across all scenarios contrasts with typical machine learning research seeking universal solutions, instead embracing the reality that deployment constraints and dataset characteristics demand matched approaches. Technically, the specialized random forest architectures and efficient Bayesian optimization methodology provide practitioners with actionable tools ready for deployment. The RF designs make strategic trade-offs—SEE-vRF sacrifices some accuracy for maximum energy efficiency through aggressive early stopping, while SEE-hRF maintains medical-grade reliability through sequential refinement—enabling selection aligned with application priorities.

Real-world validation on Raspberry Pi hardware confirmed that simulated energy predictions translate to measurable savings in physical systems, a validation step frequently overlooked in machine learning research that remains simulation-bound. The strong correlation ($r = 0.94$) between predicted and measured savings provides confidence for deployment planning, while the consistent 3.8 percentage point deviation quantifies realistic margins to incorporate in system specifications. This simulation-reality gap analysis contributes methodological value beyond the specific architectures evaluated, establishing precedent for honest characterization of model-to-deployment transfer that the field should adopt more broadly.

Significant challenges remain in realizing the full potential of sensor-aware inference. Adaptive thresholds that respond to distribution shifts during deployment would eliminate current reliance on static offline optimization. Multi-modal sensing scenarios integrating accelerometer, audio, and physiological signals would test whether early-exit benefits compound across modalities. Hardware-software co-design optimizing sensor interfaces, feature

extraction accelerators, and power management in integrated solutions would achieve efficiency gains beyond what software-only approaches enable. These directions represent natural extensions of the foundation established here, offering promising avenues for researchers building on this work.

The thesis also contributes to broader conversations about sustainable computing and responsible AI deployment. Energy efficiency directly impacts environmental footprint through reduced battery manufacturing, disposal, and charging energy. The techniques developed here enable applications advancing scientific research—long-term wildlife monitoring, infrastructure health assessment, environmental sensing—that prove impractical under current power constraints. Health equity benefits emerge from extended operational time that makes continuous monitoring viable for patients in resource-limited settings. Privacy protection follows naturally from data minimization inherent in early exits. These societal benefits underscore that technical innovation in machine learning serves purposes beyond benchmark accuracy improvements, contributing meaningfully to challenges facing communities and ecosystems.

As IoT devices proliferate across homes, cities, industries, and natural environments, energy efficiency transitions from optimization goal to deployment necessity. Battery-powered devices cannot simultaneously provide rich sensing, continuous operation, and frequent human interaction for charging. The techniques developed in this thesis relax this fundamental tension by recognizing that not all data proves equally necessary for confident predictions. Temporal redundancy in time-series observations means early measurements often suffice, enabling systems to stop sensing once confidence emerges. This insight—seemingly simple yet systematically overlooked in prior work—opens new possibilities for pervasive intelligence operating within stringent power budgets.

The future of edge AI depends not only on making models smaller through compression but also on making them smarter about when and how much to sense. This thesis demonstrated that the latter proves more consequential for IoT energy efficiency, achieving savings

that exceed what computational optimization alone can provide by an order of magnitude. As the field matures from research demonstrations toward scaled deployment, this reorientation from compute to sensing will prove increasingly critical. The sensor-aware inference paradigm established here provides a foundation for this evolution, showing that fundamental insights about system energy characteristics enable transformative efficiency gains that algorithmic refinements cannot match.

APPENDIX

APPENDIX A

Hyperparameter Configurations

This appendix provides complete hyperparameter specifications for all models evaluated in this thesis, ensuring reproducibility of results.

A.1 CNN Architecture Hyperparameters

A.2 Random Forest Hyperparameters

A.3 Bayesian Optimization Configuration

Table A.1 Hyperparameters for SEE-CNN architecture (ESWEEK 2025)

Parameter	**Value**
Architecture	
Number of convolutional layers	4
Convolutional filters	[64, 128, 128, 256]
Kernel size	3
Pooling type	Max pooling
Pooling size	2
Fully connected layers	[512, 256, num_classes]
Dropout rate	0.5
Activation function	ReLU
Output activation	Softmax
Early Exit Configuration	
Number of exits	3
Exit locations	After layers [1, 2, 4]
Exit classifier hidden units	[128, 64, num_classes]
Exit threshold optimization	Bayesian Optimization
Confidence metric	Entropy
Training	
Optimizer	Adam
Learning rate	0.001
Learning rate schedule	ReduceLROnPlateau (factor=0.5, patience=5)
Batch size	32
Epochs	100
Early stopping patience	10
Loss function	Categorical cross-entropy
Weight initialization	He normal

Table A.2 Hyperparameters for SEE-RF, SEE-vRF, and SEE-hRF architectures

Parameter	**SEE-RF (ESWEEK)**	**SEE-vRF/hRF (MLSys)**
Forest Configuration		
Number of trees	100	100
Max depth	20	20
Min samples split	2	2
Min samples leaf	1	1
Max features	sqrt(n_features)	sqrt(n_features)
Bootstrap	True	True
Random state	42	42
Early Exit Configuration		
Number of exits	3	3
Exit placement (vRF)	–	Feature partitions: [1/3, 2/3, 1.0]
Exit placement (hRF)	–	Tree partitions: [33, 67, 100]
Confidence metric	Entropy	Entropy
Threshold optimization	Grid Search	Bayesian Optimization
Threshold Search Space		
Confidence threshold range	[0.70, 0.95]	[0.70, 0.95]
Threshold discretization	0.05	0.01

Table A.3 Bayesian optimization hyperparameters for threshold selection (MLSys 2026)

Parameter	**Value**
Surrogate model	Gaussian Process
Kernel	Matérn 5/2
Acquisition function	Expected Improvement (EI)
Number of initial random points	5
Total number of evaluations	45
Exploration-exploitation parameter	$\xi = 0.01$
Noise level	$\alpha = 1e - 6$
Normalization	Z-score standardization
Objective Function	
Metric	Weighted sum: $0.5 \times \text{accuracy} + 0.5 \times \text{energy_savings}$
Constraints	Accuracy $\geq 90\%$

APPENDIX B

Detailed Dataset Statistics

This appendix provides comprehensive statistics for all six datasets used in experimental evaluation.

B.1 Dataset Characteristics Summary

Table B.1 Complete dataset characteristics

Property	**PAMAP2**	**UCI-HAR**	**Epilepsy**	**Gesture**	**Heartbeat**	**FD-A**
Domain	Activity	Activity	Medical	Wearable	Medical	Industrial
Sensor Type	Accel	Accel+Gyro	EEG	Accel	ECG	Vibration
Sampling Rate	100 Hz	50 Hz	173.6 Hz	50 Hz	125 Hz	64 Hz
Window Size	512	128	178	64	187	128
Window Duration	5.12 s	2.56 s	1.03 s	1.28 s	1.50 s	2.0 s
Channels	9	9	1	3	1	1
Features/Window	4608	1152	178	192	187	128
Classes	12	6	2	8	2	3
Train Samples	15,420	7,352	8,244	1,200	1,638	1,800
Test Samples	3,856	2,947	2,060	300	410	600
Class Balance	Imbalanced	Balanced	Imbalanced	Balanced	Balanced	Imbalanced
Energy Profile						
Sensing (mW)	42.3	38.7	52.1	35.2	28.4	45.8
Compute (mW)	18.6	15.2	22.8	19.4	12.1	24.3
% Sensing	69%	72%	70%	64%	70%	65%

B.2 Class Distribution Details

Table B.2 Per-class sample counts and balance ratios

Dataset	Class Label	Train Count	Test Count
PAMAP2			
	Lying	1,842	461
	Sitting	2,103	526
	Standing	1,287	322
	Walking	2,456	614
	Running	1,634	409
	Cycling	891	223
	Nordic Walking	1,245	311
	Ascending Stairs	987	247
	Descending Stairs	856	214
	Vacuum Cleaning	734	184
	Ironing	698	175
	Rope Jumping	687	170
UCI-HAR			
	Walking	1,226	496
	Walking Upstairs	1,073	471
	Walking Downstairs	986	420
	Sitting	1,286	491
	Standing	1,374	532
	Laying	1,407	537
Epilepsy			
	Normal	5,634	1,408
	Seizure	2,610	652
Gesture			
	Wave	150	38
	Swipe Left	150	37
	Swipe Right	150	38

APPENDIX C

Energy Model Derivations

This appendix provides detailed mathematical formulations for energy consumption modeling used throughout this thesis.

C.1 Sensing Energy Model

The energy consumed by sensor acquisition for a time-series window of length W with sampling rate f_s is:

$$E_{\text{sense}} = P_{\text{sensor}} \times \frac{W}{f_s} \tag{C.1}$$

where:

- P_{sensor} is the sensor power consumption (mW)
- W is the window length (number of samples)
- f_s is the sampling frequency (Hz)

For multi-channel sensors (e.g., 3-axis accelerometer), the total sensing energy is:

$$E_{\text{sense}}^{\text{total}} = \sum_{c=1}^{C} P_{\text{sensor}}^{(c)} \times \frac{W}{f_s} \tag{C.2}$$

where C is the number of channels.

C.2 Early-Exit Sensing Energy

For an early-exit system with exit point at fraction $\alpha \in [0, 1]$ of the full window, the sensing energy becomes:

$$E_{\text{sense}}^{\text{exit}}(\alpha) = P_{\text{sensor}} \times \frac{\alpha W}{f_s} \tag{C.3}$$

The **sensing energy savings** relative to full-window baseline is:

$$\text{Savings}_{\text{sense}}(\alpha) = \frac{E_{\text{sense}} - E_{\text{sense}}^{\text{exit}}(\alpha)}{E_{\text{sense}}} = 1 - \alpha \tag{C.4}$$

C.3 Computational Energy Model

C.3.1 CNN Inference Energy

For a CNN with L layers, the computational energy is:

$$E_{\text{compute}}^{\text{CNN}} = \sum_{\ell=1}^{L} \left(E_{\text{conv}}^{(\ell)} + E_{\text{pool}}^{(\ell)} + E_{\text{act}}^{(\ell)} \right) \tag{C.5}$$

where:

- $E_{\text{conv}}^{(\ell)}$ = energy for convolutional operations in layer ℓ
- $E_{\text{pool}}^{(\ell)}$ = energy for pooling operations
- $E_{\text{act}}^{(\ell)}$ = energy for activation functions

For a convolutional layer with K filters of size $k \times k$:

$$E_{\text{conv}}^{(\ell)} = P_{\text{MAC}} \times N_{\text{MAC}}^{(\ell)} \times t_{\text{MAC}} \tag{C.6}$$

where:

- P_{MAC} = power per multiply-accumulate operation
- $N_{\text{MAC}}^{(\ell)}$ = number of MAC operations in layer ℓ
- t_{MAC} = time per MAC operation

C.3.2 Random Forest Inference Energy

For a Random Forest with T trees and maximum depth D:

$$E_{\text{compute}}^{\text{RF}} = T \times E_{\text{tree}} \tag{C.7}$$

where E_{tree} is the energy per tree traversal:

$$E_{\text{tree}} = P_{\text{CPU}} \times d_{\text{avg}} \times t_{\text{compare}} \tag{C.8}$$

where:

- P_{CPU} = CPU power consumption
- d_{avg} = average tree depth traversed (typically $\approx \log_2(N_{\text{leaves}})$)
- t_{compare} = time per node comparison

C.4 Early-Exit Overhead

The additional energy cost of early-exit mechanisms includes:

$$E_{\text{overhead}} = E_{\text{confidence}} + E_{\text{decision}} + E_{\text{gate}} \tag{C.9}$$

where:

- $E_{\text{confidence}}$ = energy to compute confidence score (entropy calculation)
- E_{decision} = energy for threshold comparison
- E_{gate} = energy to control sensor power gating

For entropy-based confidence with K classes:

$$E_{\text{confidence}} = P_{\text{CPU}} \times K \times (t_{\log} + t_{\text{mult}}) \tag{C.10}$$

C.5 Total System Energy

The total system energy for an early-exit inference at exit point α is:

$$E_{\text{total}}(\alpha) = E_{\text{sense}}^{\text{exit}}(\alpha) + E_{\text{compute}}^{\text{exit}}(\alpha) + E_{\text{overhead}} \tag{C.11}$$

For a 3-exit system, the expected energy across the test set is:

$$\mathbb{E}[E_{\text{total}}] = \sum_{i=1}^{3} p_i \times E_{\text{total}}(\alpha_i) \tag{C.12}$$

where p_i is the fraction of samples exiting at exit point i with position α_i.

C.6 Energy Savings Calculation

The percentage energy savings relative to baseline is:

$$\text{Savings}_{\%} = \frac{E_{\text{baseline}} - \mathbb{E}[E_{\text{total}}]}{E_{\text{baseline}}} \times 100\% \tag{C.13}$$

APPENDIX D

Author Contribution Statements

This appendix provides detailed attribution for collaborative work, fulfilling Washington State University's requirements for multi-author manuscripts in alternative-format theses.

D.1 ESWEEK 2025 Paper Contributions

Paper Title: Energy-Efficient Time Series Applications on IoT Devices with Sensor-Aware Early-Exit Classifiers

Authors: [Your Name], [Co-author 1], [Co-author 2], [Advisor Name]

D.1.1 Individual Contributions

[Your Name] (Thesis Author):

- Designed and implemented SEE-CNN architecture (100%)
- Designed and implemented basic SEE-RF architecture (100%)
- Designed all experiments and evaluation methodology (100%)
- Conducted all experimental evaluations (100%)
- Performed all data analysis and generated figures (100%)
- Wrote the complete manuscript (100%)
- Submitted to conference and handled revisions (100%)

[Co-author 1]:

- Provided feedback on experimental design

- Reviewed manuscript and provided editorial suggestions

[Co-author 2]:

- Assisted with initial literature review
- Provided feedback on presentation

[Advisor Name]:

- Supervised research direction
- Provided guidance on methodology
- Reviewed manuscript drafts

Thesis Author Contribution Percentage: 95%

D.2 MLSys 2026 Paper Contributions

Paper Title: Sensor-Aware Random Forests with Early Exits for Energy-Efficient Time Series Applications on IoT Devices

Authors: [Your Name], [Labmate 1], [Labmate 2], [Advisor Name]

D.2.1 Individual Contributions

[Your Name] (Thesis Author):

- Conceptualized sensor-aware early-exit approach for RFs (shared)
- Designed evaluation methodology framework (100%)
- Implemented Bayesian optimization for threshold selection (100%)
- Conducted all experiments across 6 datasets (100%)
- Performed all data analysis and interpretation (100%)

- Generated all figures and tables (100%)
- Wrote the complete manuscript (100%)
- Conducted real-world validation on Raspberry Pi (100%)

[Labmate 1]:

- Conceptualized sensor-aware early-exit approach for RFs (shared)
- Designed SEE-vRF (vertical split) architecture (primary)
- Implemented SEE-vRF architecture (100%)
- Reviewed manuscript and provided technical feedback

[Labmate 2]:

- Conceptualized sensor-aware early-exit approach for RFs (shared)
- Designed SEE-hRF (horizontal/sequential split) architecture (primary)
- Implemented SEE-hRF architecture (100%)
- Reviewed manuscript and provided technical feedback

[Advisor Name]:

- Supervised research direction
- Provided guidance on methodology and optimization approach
- Reviewed manuscript drafts

Thesis Author Contribution Percentage: 70%

D.2.2 Clarification on Thesis Scope

The thesis author's contributions to the MLSys 2026 paper focus on:

1. **Evaluation Methodology:** Complete design of experimental framework, metrics, ablation studies, and cross-domain analysis

2. **Optimization:** Design and implementation of Bayesian optimization for threshold selection (10× speedup over exhaustive search)

3. **Execution and Analysis:** All experimental execution, data collection, statistical analysis, and interpretation

4. **Validation:** Real-world deployment and energy measurements on embedded hardware

The RF architectures (vRF, hRF) were collaboratively designed with labmates. This collaborative work is clearly attributed and does not diminish the thesis contribution, which emphasizes methodological rigor and comprehensive evaluation.

APPENDIX E

Additional Experimental Results

This appendix presents supplementary experimental results that support findings in Chapter 5 but were omitted from the main text for brevity.

E.1 Per-Dataset Ablation Studies

Table E.1 Complete ablation study results: number of exits per dataset

Dataset	**2 Exits**	**3 Exits**	**4 Exits**	**5 Exits**	**Best**
PAMAP2					
Energy Savings	58.2%	70.0%	71.3%	71.8%	3 exits
Accuracy	94.1%	94.5%	94.6%	94.6%	
Overhead	1.2%	2.1%	3.8%	5.2%	
UCI-HAR					
Energy Savings	52.4%	64.0%	66.1%	66.5%	3 exits
Accuracy	95.3%	95.9%	95.9%	96.0%	
Overhead	1.4%	2.3%	4.1%	5.8%	
Epilepsy					
Energy Savings	48.7%	63.0%	64.8%	65.2%	3 exits
Accuracy	96.8%	97.5%	97.6%	97.6%	
Overhead	1.1%	1.9%	3.5%	4.9%	

Table E.2 Detailed calibration metrics for different confidence measures

Dataset	Metric	ECE	MCE	Brier Score	NLL
PAMAP2					
	Entropy	0.058	0.142	0.089	0.234
	Gini	0.067	0.156	0.098	0.251
	Margin	0.083	0.189	0.124	0.287
Epilepsy					
	Entropy	0.031	0.098	0.042	0.156
	Gini	0.039	0.112	0.051	0.178
	Margin	0.052	0.145	0.073	0.203

E.2 Confidence Metric Detailed Comparison

ECE = Expected Calibration Error, MCE = Maximum Calibration Error, NLL = Negative Log Likelihood

E.3 Exit Point Distribution Histograms

[TODO: Add detailed histograms for each dataset showing exit distributions]

APPENDIX F

Implementation Details

This appendix provides implementation specifics to facilitate reproducibility.

F.1 Software Dependencies

F.2 Pseudocode for Key Algorithms

F.2.1 Early-Exit Inference Algorithm

Algorithm 1 Sensor-Aware Early-Exit Inference

Model M with exits $E = \{e_1, e_2, ..., e_k\}$ Thresholds $T = \{\tau_1, \tau_2, ..., \tau_k\}$ Window size W, sampling rate f_s Prediction y, energy consumption E_{total} $E_{sense} \leftarrow 0$ $E_{compute} \leftarrow 0$ $i = 1$ to k $\alpha_i \leftarrow$ exit position for e_i e.g., 0.33, 0.67, 1.0 $w_i \leftarrow \alpha_i \times W$ Window length at exit i **Sensor Acquisition:** $x_i \leftarrow$ collect samples from w_{i-1} to w_i $E_{sense} \leftarrow E_{sense} + P_{sensor} \times (w_i - w_{i-1})/f_s$ **Inference:** $p_i \leftarrow M.forward(x_{1:i})$ Forward pass to exit e_i $E_{compute} \leftarrow E_{compute} + \text{compute_energy}(e_i)$ **Confidence Check:** $c_i \leftarrow \text{entropy}(p_i)$ Compute confidence $E_{compute} \leftarrow E_{compute} + \text{confidence_energy}(c_i)$ $c_i \geq \tau_i$ $y \leftarrow \arg\max(p_i)$ **break** Early exit $E_{total} \leftarrow E_{sense} + E_{compute}$ y, E_{total}

F.2.2 Bayesian Optimization for Threshold Selection

F.3 Hardware Setup Details

F.3.1 Raspberry Pi Configuration

- **Model:** Raspberry Pi 4 Model B
- **RAM:** 4GB LPDDR4-3200
- **CPU:** Broadcom BCM2711, Quad-core Cortex-A72 @ 1.5GHz

Table F.1 Software environment and dependencies

Component	Version
Python	3.10.8
Deep Learning	
TensorFlow	2.10.0
Keras	2.10.0
PyTorch	1.13.1 (for comparisons)
Machine Learning	
scikit-learn	1.2.0
scipy	1.9.3
Optimization	
scikit-optimize	0.9.0
bayesian-optimization	1.4.2
Data Processing	
numpy	1.23.5
pandas	1.5.2
Visualization	
matplotlib	3.6.2
seaborn	0.12.1
Hardware Interface	
Adafruit-Blinka	8.9.0
board	1.0

Algorithm 2 Threshold Optimization via Bayesian Optimization

Model M, validation set D_{val} Number of exits k, evaluation budget n_{eval} Optimal thresholds $T^* = \{\tau_1^*, \tau_2^*, ..., \tau_k^*\}$ Initialize Gaussian Process surrogate GP Initialize acquisition function $\alpha(T) = EI(T)$ **Random Initialization:** $i = 1$ to 5 $T_i \leftarrow$ sample random thresholds from $[0.70, 0.95]^k$ $f_i \leftarrow$ evaluate(M, D_{val}, T_i) Compute accuracy-energy $GP \leftarrow GP \cup \{(T_i, f_i)\}$ **Iterative Optimization:** $i = 6$ to n_{eval} $T_i \leftarrow \arg\max_T \alpha(T)$ Maximize expected improvement $f_i \leftarrow$ evaluate(M, D_{val}, T_i) $GP \leftarrow GP \cup \{(T_i, f_i)\}$ Update GP posterior $T^* \leftarrow \arg\max_T f(T)$ Select best observed configuration T^*

- **Storage:** SanDisk 32GB Class 10 microSD
- **OS:** Raspberry Pi OS (64-bit), Debian version 11 (Bullseye)
- **Kernel:** Linux 5.15.84-v8+
- **Power Supply:** Official 5V 3A USB-C

F.3.2 Power Measurement Setup

- **Sensor:** INA219 High-Side DC Current Sensor
- **Measurement Range:** 0-3.2A, 0-26V
- **Resolution:** 0.1mA (current), 4mV (voltage)
- **Sampling Rate:** 100 Hz
- **Interface:** I2C (address 0x40)
- **Calibration:** Verified against Keysight 34461A DMM